



FIDATO HOTELS

Hospitality Platform

Service Manual

BUILD

Phase 2.5 — deployed, in production use

DATE OF RECORD

14 August 2026

CONTENTS

Roadmap · 19 volumes · 2 appendices

AUDIENCE

Engineers, operators and reviewers joining the project

Internal document. Commercial figures quoted from Phase 1 are simulated and labelled where they appear.

Before you start

This manual describes a system that exists and runs. Everything in it can be opened, clicked and checked against the code. If a statement here disagrees with the running application, the application is right and this manual has a bug — please report it.

What you are looking at

Fidato Hotels sells room nights into **32 partner properties it does not own**. This platform is the system that runs that business: the CRM, the reservations, the money and the reporting. It is not a property management system — each partner hotel already has one of those.

That one fact explains a surprising number of decisions in this manual, so it is worth holding on to.

Phase 1 is the frontend only

There is **no backend and no login**. Every screen is finished and every interaction works, but the data comes from a simulation running in the browser rather than a database. This was deliberate: it lets the shape of the product be settled before any backend work is committed to. Volume I explains the reasoning; Volume III argues it against the alternatives.

Two consequences you will notice immediately:

- **Data resets when you refresh.** By design — every review starts from identical data.
- **There is no sign-in.** Instead, a “*Viewing as...*” switcher in the top bar lets you become any of the eight roles in one second. Use it constantly; it is the fastest way to understand the product.

How this manual is written

It is written the way a service manual for an engine is written, not the way a product tour is written:

- **Every part is described on its own** — inputs, outputs, failure modes — before it is described in context.
- **Every decision carries the options that lost.** A decision without the roads not taken is just an assertion. Volume III is nothing but decisions and reasoning.
- **Diagnostics are organised by symptom**, because when something is wrong you do not yet know which part it is.
- **The code is real.** Every listing is quoted from the running source at the path given above it.

If you are new, read in this order

DAY	READ	THEN DO THIS
1	Volumes I and II	Run the app. Switch between all eight roles and notice what changes.
2	Volume X, skimming	Open every screen it describes, in order. Do not read the code yet.
3	Volumes IV and V	Open <code>/design-system</code> in the app and match it against the volume.
4	Volume IX	Open <code>/admin/roles</code> . Then try to edit a rate plan as a Hotel Manager.
5	Volumes VI, VII, VIII	Follow one query from a screen down to the store and back.
6	Volume XII	The nine defects. This is the most instructive volume in the manual.

Volumes XI (diagnostics), XIII (what was tested), XIV (Phase 2) and the two appendices are reference. Read them when you need them, not front to back.

Conventions

<code>src/path/file.ts</code>	A real file in the repository
BR-01	A numbered business rule — Volume IX
ADR-01	A numbered decision — Volume III

D-01	A numbered defect — Volume XII
⚠️	A trap: something that looks correct and is not
🔧	Something that must change in Phase 2

Two honest warnings

Twelve interactive flows were never click-tested. The reservation wizard end-to-end, the approve and cancel dialogs, the merge action and the import commit are built and typechecked, but the automated browser used for verification could not deliver real input events. Volume XIII, section 13.4 lists exactly what this leaves unproven. Do not read the rest of this manual as claiming otherwise.

There is no test suite. Volume III, ADR-24 explains why, and says plainly that it is the weakest decision in the log. If you are joining to work on Phase 2, writing tests over the rules layer is the first task and it is already scoped in Volume XIV.

Contents

VOLUME 0	Programme roadmap
VOLUME I	System overview
VOLUME II	Architecture
VOLUME III	Decision log
VOLUME IV	Design system
VOLUME V	Component reference
VOLUME VI	Data model
VOLUME VII	The seed engine
VOLUME VIII	The repository layer
VOLUME IX	Permissions and business rules
VOLUME X	Screen teardown
VOLUME XI	Diagnostics
VOLUME XII	Defect log
VOLUME XIII	Verification record
VOLUME XIV	Phase 2 handover
VOLUME XVI	Phase 2 as built
VOLUME XVII	Automation and n8n
VOLUME XVIII	The booking register
VOLUME XIX	Launch and operations
VOLUME XV	Glossary and index
APPENDIX A	Complete data dictionary
APPENDIX B	Complete component props reference

VOLUME 0

Programme roadmap

Where this started, what was built, where it stands, and what is left.

Every other volume in this manual describes a *part* of the system. This one describes the *programme* — the sequence of decisions and builds that produced it, in the order they happened, with the dates they happened on.

Record date: 14 August 2026 · **Commits:** 62 · **Elapsed:** 31 July – 14 August 2026

0.1 The problem this exists to solve

Fidato Hotels sells room nights into **32 partner properties it does not own**.

That sentence is the origin of almost every design decision in this manual, so it is worth stating what it rules out. Fidato is not a hotel chain and this is not a property management system — every partner already runs one. Fidato is a distributor: it negotiates rates, holds corporate relationships, takes bookings, issues vouchers, invoices the corporate account and earns a commission from the property.

Before this platform, that ran on spreadsheets. The booking register was one of them — a file called `FH Booking Register.xlsx` with 6,626 rows accumulated over years, maintained by hand by whoever took the booking.

The consequences of that were the brief:

PROBLEM	WHAT IT CAUSED
One spreadsheet, many editors	563 cancellations spelled four different ways; 868 distinct values in a column that should hold about five
No enforced structure	Bank and UTR reference numbers typed into a money column. Summing it gives 1.37 quadrillion against 7.2 crore of real revenue
No permissions	Everyone who could open the file could see the commission Fidato negotiated with each property
No record of <i>why</i>	A rate, once changed, had no history and no author
Nothing automatic	Every voucher hand-made, every confirmation hand-sent

⚠ **The register was not merely untidy — it was actively misleading.** A system that produces a confident wrong number is worse than one that produces none, and that principle recurs throughout this manual because it was learned here first.

0.2 The phase plan as originally drawn

Four phases, decided before any code was written.

PHASE	SCOPE	REASON FOR THE BOUNDARY
1	The entire frontend, with simulated data. No backend, no login	Prove the product is <i>right</i> before making it <i>real</i> . A wrong screen is cheap to change with no database behind it
2	Firebase — authentication, Firestore, security rules	Make it real
2.5	n8n consumes an event queue — vouchers, email, Drive, WhatsApp	Automate the delivery that was done by hand
3	Further automation	—
4	Final testing and launch	—

The single most consequential decision in the programme was made at this point and is recorded as ADR-1 in Volume III: **build the whole frontend against a repository interface first, with simulated data behind it, and swap the implementation in Phase 2**.

That is why Phase 2 changed *one file* rather than 34 screens. Volume VIII documents the seam; §0.4 below records what it actually cost when the time came.

0.3 Phase 1 — the frontend, 31 July 2026

Delivered: every screen of the product, working, against a deterministic seed engine that generated 1,100 reservations across the 32 real properties.

Routes	38
Roles	8 (6 active, 2 dormant)

UI primitives	28
Collections modelled	18
Backend	None
Authentication	None — a role switcher stood in for identity

The properties were real, extracted from the partner fact sheets. The commercial figures were simulated, and labelled as such on every page of the record.

What this phase bought: nine defects found and fixed before a database existed, four of them *silent* — screens that rendered perfectly and reported wrong answers. Volume XII is the full account. Zero of the nine were caught by TypeScript or by the build, which is the origin of the verification discipline in Volume XIII and in `docs/TEST-CHECKLIST.md`.

The most instructive was **occupancy**. Computing it as reservations ÷ total rooms produced a figure under 1% and looked like a data problem. It was not: Fidato books a *slice* of each partner hotel, so that ratio is meaningless by construction. The reports now show Fidato's share of sellable room nights and say so on the label.

0.4 Phase 2 — Firebase, 31 July – 1 August 2026

Delivered: authentication, Firestore, real security rules, and the removal of every trace of simulated data.

The phase opened with a finding that contradicted its own plan. Volume XIV had been written assuming Cloud Functions; the project runs on the Firebase **Spark** plan, which has none. Four of the handover's recommendations had to be redesigned:

ASSUMED	SPARK REALITY	WHAT WAS BUILT INSTEAD
Cloud Function roll-up counters	No functions	Counters updated inside the client transaction
Server-side merge	No functions	Transactional merge in the repository
Server-issued invoice numbers	No functions	A <code>counters</code> document, incremented transactionally
Custom claims for roles	No Admin SDK	The role lives in <code>users/{uid}</code> and every rule re-reads it

⚠️ **That last row is load-bearing and appears throughout this manual.** A role in a Firestore document, not a token claim, means the rules perform a document read to authorise a document read. It also means the `users` document id **must** be the auth uid, which in turn is why invitations are a separate collection keyed by email — an invited person has no uid yet.

The seam held. Swapping `repositories/mock/` for `repositories/firestore/` changed the barrel file in `src/data/repositories/index.ts` and nothing above it. That was the wager of Phase 1 and it paid.

The bootstrap problem was discovered here and is by design: only an Owner or Admin may create an invitation, and Spark has no Admin SDK to break the cycle from a script. The first Owner is written by hand in the Firebase console, which bypasses rules. `docs/RUNBOOK.md` carries the procedure.

Phase 2 closed with **59 security-rules tests executing against the real rules engine** in the emulator — not assertions about a permission table, but the engine itself refusing what it should refuse.

0.5 Phase 2.5 — automation, 1 – 5 August 2026

Delivered: a booking raises an event; n8n turns that event into a voucher in a guest's inbox.

The architectural rule was set first and has not been broken since:

React UI → repository layer → Firestore → automationQueue → n8n → the outside world

The React app never talks to Drive, WhatsApp, email or accounting. It cannot: Spark has no server, and a credential in a browser bundle is a published credential. It writes a row to `automationQueue` and posts a webhook; n8n does everything else. Volume XVII is the full account.

This phase was the longest and the most rewritten, because it kept meeting the outside world:

ATTEMPT	WHY IT WAS ABANDONED
Voucher as HTML in the email body	Gmail strips <code><svg></code> and blocks <code>data:</code> URIs, so the logo vanished
PDF rendered by a Gotenberg container	The n8n host had no Docker access
Base64 PDF over the webhook	Payload size, and n8n handling it badly

ATTEMPT	WHY IT WAS ABANDONED
Interakt for WhatsApp	Superseded when a Meta Cloud API app with an approved template became available

What settled: the app generates a real vector PDF with jsPDF, the email is itself a branded voucher rather than a covering note, and WhatsApp goes through the Meta Cloud API by uploading the PDF to obtain a media id — which avoids publishing a guest's voucher at a public URL.

⚠️ **jsPDF's built-in fonts are WinAnsi only.** No ₹, no en dash, no middot. They are stripped at render, because a blank box on a document about money reads as a broken system.

Two defects from this phase are worth carrying forward because both were *intermittent by data shape*, which is the hardest kind to see:

- **A reservation for a customer linked to a company failed; one without a company worked.** The company read sat below the write inside the transaction, and Firestore aborts a transaction that reads after writing. With no company there is only one read and the ordering happens to be legal. Now pinned by an emulator test.
- **A undefined field anywhere in the document was rejected outright by Firestore,** which surfaced as "Could not create / Nothing was saved" with no indication of which field.

0.6 The booking register — 6 August 2026

Delivered: the 6,626-row spreadsheet, digitised, queryable, filterable and editable, in its own database.

This was not in the original plan. It was added because the historical record was the one thing the new platform could not replace by going forward — it had to be carried across.

⚠️ **It is a separate Supabase project and shares no domain model with the CRS.** A register entry is a historical record; a Reservation is a live booking. Conflating them would produce two systems that disagree about the same stay. Volume XVIII is the full account.

The phase produced the single most instructive security finding in the programme:

A Postgres view runs as its owner and bypasses RLS entirely unless `security_invoker = true`.

The SQL test queried the *table*, which was correctly protected, and passed. The browser queried the *view* and received all 6,626 rows while signed out. It is now Constraint 12, and the lesson generalises: **a test that exercises a different path from production proves nothing about production.**

0.7 Hardening and launch preparation — 8 – 14 August 2026

Not a phase in the original plan; the work that turns a built system into an operable one.

DATE	WORK
8 Aug	GSTIN made optional; properties can be deleted, with a guard that refuses while any reservation references them
10 Aug	Bank details added to the property import; Import promoted to its own place in the navigation
14 Aug	The register's filters and reports fixed; the documentation set built; the launch wipe

Two permission faults were found and closed in this period, both of the same shape: **a navigation entry gated on a grant that had nothing to do with what the screen does**. A salesperson was shown Import — a screen on which they could import nothing — because the entry was gated on "can view customers". The sidebar and the route guard now read one shared predicate, and a test asserts they agree for every role.

⚠️ **One more of the same kind is known and not yet fixed:** "Duplicates" is gated on customer access but needs the `merge` action. It is recorded in `docs/HANDOVER.md`.

The register fix on 14 August is worth recording in full because it was two faults wearing one symptom — see Volume XVIII §5 and `docs/traces/bug-register-filters.md`.

0.8 Where the programme stands today

14 August 2026. Deployed, verified, and wiped clean for launch.

Live	https://crstest-9a0c5.web.app — current build
Firestore rules	Deployed
Tests	153 unit · 101 rules · typecheck and build clean
Repository	github.com/hotelsfidato-ai/Crs-Test — public, <code>main</code> current

Routes	51
Firestore contents	The Owner account and two settings documents. Nothing else

Every business collection was cleared on 14 August at the owner's instruction so the platform starts on real data. Booking references restart at `FH-2026-00001` .

`settings` was deliberately kept: it holds the company details printed on every voucher and the n8n webhook configuration. Deleting it would have silently disarmed the automation on launch day.

Verified, and not

VERIFIED	HOW
The permission model	101 rules tests against the real engine
Business logic	153 unit tests — GST bands, import mapping, permissions, register folds, navigation
Voucher delivery	Three real bookings, all recorded <code>sent</code> · <code>Delivered</code> with the PDF
The deployed build	Bundle hash compared against the local build
NOT VERIFIED	WHY
The interactive flows end to end	Wizard, approve/cancel, merge and import commit are tested at the logic layer, never click-tested
The register under a real token	Blocked on a Supabase configuration step
Email deliverability	Blocked on DNS

0.9 What is left

Blocked on configuration outside this repository — none of it is code:

ITEM	NEEDS
The register returns nothing	Supabase → Authentication → Third-Party Auth → trust Firebase project <code>crstest-9a0c5</code> , then the person's address in <code>register_access</code>
Register still holds its rows	<code>docs/supabase/clear-register.sql</code> , run from the Supabase dashboard
Email lands in spam	SPF, DKIM and DMARC for <code>fidatohotels.com</code> , aligned to the sender
n8n register insert refused	The n8n Supabase credential holds the publishable key; it needs <code>service_role</code>

Known and deliberately not fixed:

- Nothing ever closes an `automationQueue` row, so Automation → Run history will report a 0% success rate while vouchers deliver correctly. The fix is one line but decides whether the app or n8n owns the queue's lifecycle.
- "Duplicates" is gated on the wrong permission.
- Register aggregation would be better done in Postgres than folded in the browser.

The launch sequence is in Volume XIX and in `docs/HANDOVER.md` .

0.10 What this programme learned

Recorded here because each one cost something and each one generalises beyond this codebase.

1. **A confident wrong number is worse than no number.** Occupancy, the +757% growth report, and 1.37 quadrillion of "money received" were all systems answering fluently and wrongly. Every one is now either corrected or labelled.
2. **Test the path production uses.** The RLS view, the `--noEmit` typecheck that compiles nothing, and the "Send test" button that forced the webhook enabled were all green signals from a path nobody actually took.
3. **The seam paid for itself.** One file changed at the Phase 1 → 2 boundary.
4. **Silent defects outnumber loud ones.** Four of Phase 1's nine rendered perfectly.
5. **Two permission systems will drift.** The client matrix and `firestore.rules` cannot see each other; only tests asserting they agree keep them together.

6. **Documentation that is trusted and wrong is worse than none.** `CLAUDE.md` described an undeployed project for weeks after it shipped. State now lives in one file that is rewritten every session.

VOLUME I

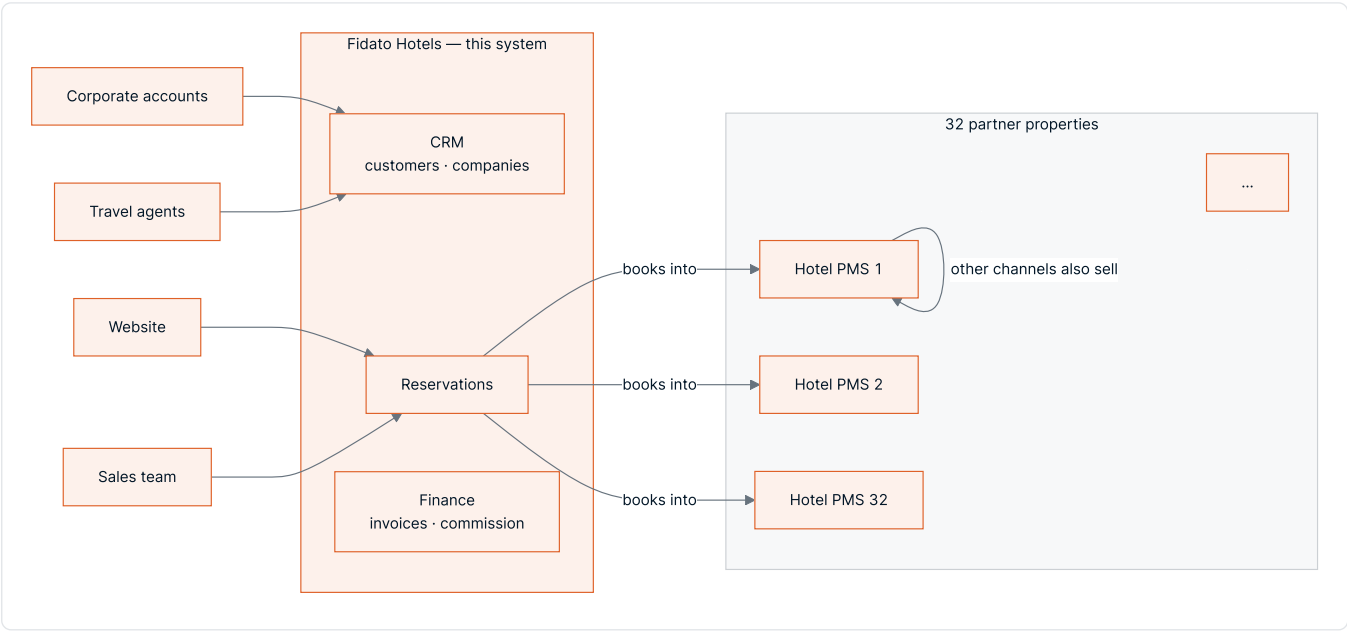
System overview

1.1 What this system is

Fidato Hotels is a hospitality management company that does **not own** the properties it sells. It holds commercial agreements with 32 partner hotels across India and sells room nights into them through a sales team, corporate accounts, travel agents and its own website.

That business shape determines everything about this software.

A property management system (PMS) runs *one hotel* — it knows every room, every housekeeping status, every walk-in. Fidato does not need that; each partner already has one. What Fidato needs is a **central reservation system (CRS) with a CRM and a back office bolted to it**: a system that knows *its own* customers, *its own* bookings, *its own* commission, spread across 32 properties it does not control.



The consequence that catches everyone out

Fidato sells **a slice** of each partner hotel. Other channels — the hotel's own website, OTAs, walk-ins, other agents — sell the rest. This single fact invalidates the most obvious metric in the product:

Occupancy is **not** Fidato's reservations divided by the hotel's total rooms.

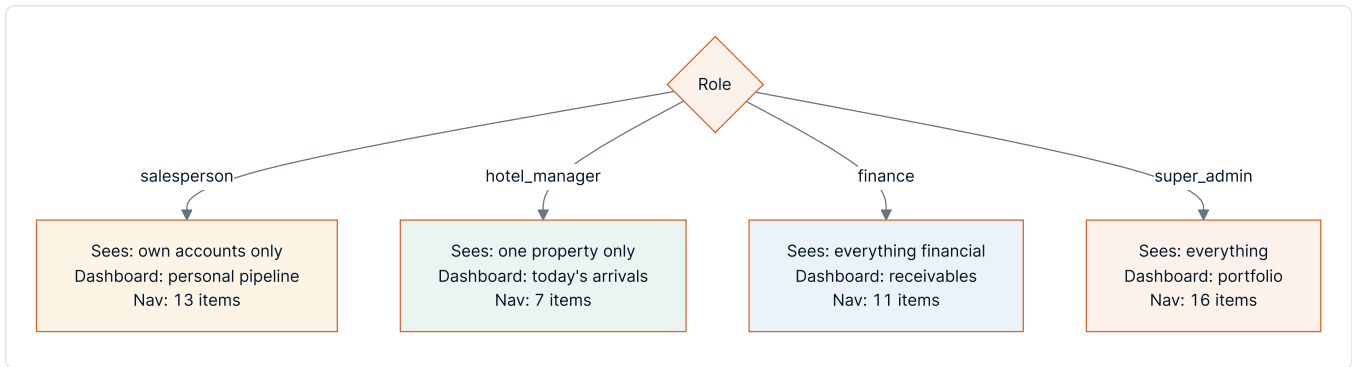
That ratio, computed over this portfolio, comes out below one percent. It is not a low number; it is a *meaningless* number, because the denominator describes something Fidato does not control. Volume VI §6.9 and ADR-19 cover how occupancy is actually derived.

1.2 Who uses it

Eight roles, and they want genuinely different things from the same data.

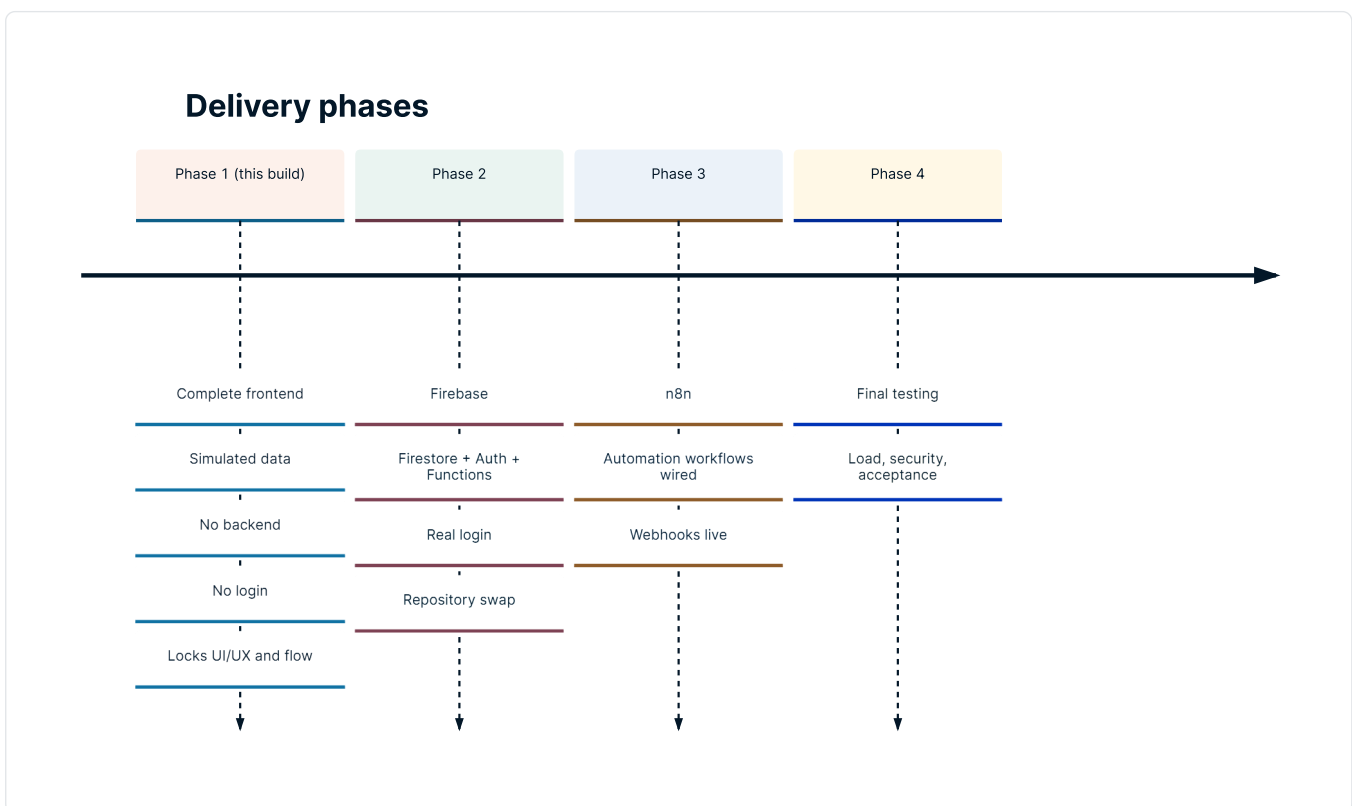
ROLE	POPULATION	PRIMARY QUESTION THEY OPEN THE APP TO ANSWER
Super Admin	1	"Is anything broken, and who did what?"
Admin	2	"What needs administering today?"
Sales Manager	3	"How is my team performing, and what needs my approval?"
Salesperson	8	"What's in my book, and what do I need to chase?"
Hotel Manager	5	"Who is arriving at <i>my</i> property today?"
Finance	3	"What have we billed, what's outstanding, what's overdue?"
Support	1	"What happened on this booking?"
Viewer	1	"Show me, don't let me touch."

These are not cosmetic differences. A salesperson and a sales manager looking at `/crm/companies` see **different rows**, not the same rows with different buttons. A hotel manager's dashboard is a different dashboard, with different KPIs, not the portfolio dashboard filtered.



1.3 The phase plan

The build is deliberately staged. Phase 1 exists so that Phases 2–4 are plumbing rather than redesign.



Why front-end first, and why this is not the usual advice

The conventional order is data model → API → UI. It was inverted here on purpose, for three reasons specific to this project:

- The requirements were a description, not a specification.** The brief was a twenty-document product engineering specification that did not yet exist. Building the screens *is* the fastest way to discover what the fields actually need to be. Several fields in the data model exist only because a screen demanded them.
- The expensive mistake here is a wrong flow, not a wrong query.** Rewriting a Firestore query is an afternoon. Rewriting a reservation wizard after the sales team has been trained on it is a quarter. Phase 1 puts the reversible work first.
- Simulation is cheap and honest.** Because the repository layer was written to Firestore's shape from the first line (Volume VIII), the simulation is not throwaway. It is the same interface with a different implementation behind it.

The risk of this order — that the UI assumes data the backend cannot cheaply provide — is managed by ADR-06: the mock repository is deliberately restricted to operations Firestore can actually perform. There are no joins in the repository layer, because Firestore has none.

1.4 What Phase 1 includes

AREA	INCLUDED
Screens	All 38 routes, fully built, no stubs
Data	1,100 reservations, 32 real properties, 180 customers, 40 companies, 450 invoices, and 13 more collections
Interactions	Create, edit, cancel, approve, merge, import, filter, sort, paginate, print
Roles	All 8, switchable, genuinely affecting navigation, scoping and available actions
Business rules	All 8 encoded and enforced in the UI
States	Empty, loading, error and no-results on every list
Responsive	Down to tablet; drawer navigation below <code>lg</code>
Accessibility	Keyboard navigation, focus management, ARIA on all custom controls
Print	Invoices and report covers
Documentation	This manual, plus four working documents and two self-documenting screens

1.5 What Phase 1 deliberately excludes

EXCLUDED	WHY	ARRIVES IN
Login / authentication	A login screen proves nothing about the product. The role switcher demonstrates the permission model <i>better</i> than a login would, because you can move between roles in one second	Phase 2
Firebase	Committing to a schema before the screens exist is the mistake this phase avoids	Phase 2
n8n	The workflow definitions are written; only execution is deferred	Phase 3
Real email / PDF / WhatsApp	Delivery is an integration concern, not a UX one. Buttons show what <i>would</i> happen	Phase 2–3
File uploads	Needs storage; nothing in Phase 1 depends on it	Phase 2
Server	There is nothing to serve	Phase 2
A real language model	The assistant computes from live data instead — see ADR-22	Phase 2
Data persistence across refresh	Deliberate. Every review starts from an identical state	Phase 2

⚠️ **The exclusions are not gaps to be apologised for.** Each is a decision with a stated reason. If a reason stops holding, the decision should be revisited — that is what Volume III is for.

1.6 The demo date

The entire seeded world is built around a fixed "today":

```
// src/data/seed/index.ts
export const TODAY = new Date(2026, 6, 28); // 28 July 2026
```

Everything is positioned relative to it: reservations span −330 to +150 days around it, invoices age from it, audit entries trail behind it, and the inventory grid projects forward from it.

Why fix the date at all? Because a demo that drifts is a demo that decays. If the seed used the real clock, then in three months the "upcoming arrivals" panel would be empty, every forward booking would have become historical, and the approval queue would be stale. A fixed date means the manual you are reading now describes the screens you will see next year.

🔧 **Phase 2 removes this.** Once data comes from Firestore, `TODAY` is replaced by `new Date()` and the seed module is deleted. There are 43 references to `TODAY` across the codebase; Volume XIV §14.6 lists them.

1.7 The single most important thing to understand

The product has **one idea** running through every screen:

Show people what they cannot do, and tell them why.

A blocked action is never hidden. The Edit button on a rate plan does not disappear for a hotel manager — it is replaced by the word **Locked**, and a banner at the top of the page explains that pricing is owned centrally by the revenue team.

This costs more to build than hiding things. It is worth it because:

- A product that hides things looks **broken** to the person who cannot see them.
- A rule that is never surfaced is a rule nobody learns, so it gets asked about forever.
- When a permission is wrong, a visible restriction gets reported. A hidden one does not.

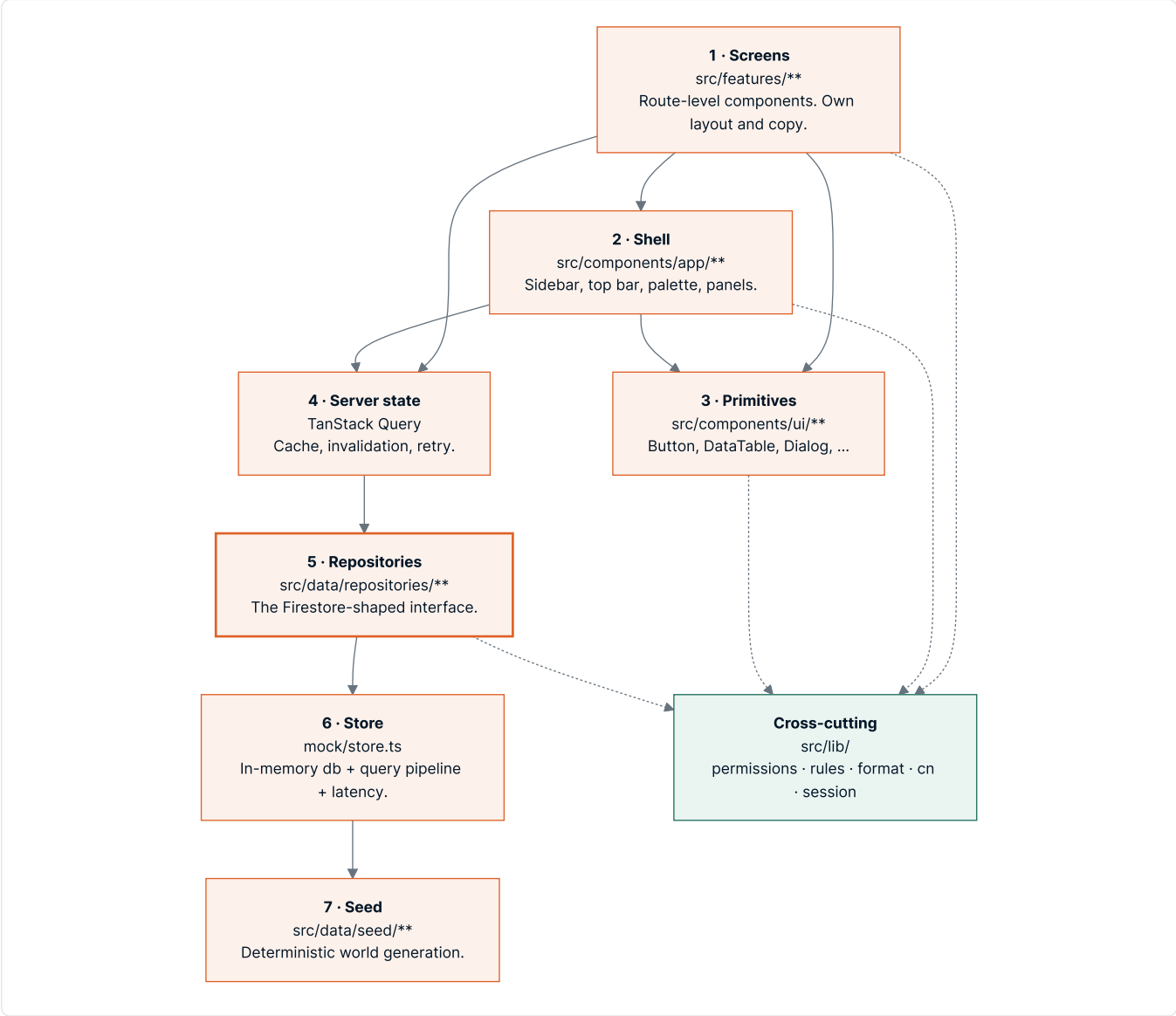
The same principle produces the **Forbidden** screen instead of a 404 (ADR-11), the disabled Cancel button with a tooltip on completed reservations, and the explicit statement on the Occupancy report that the number does not mean what you might assume.

VOLUME II

Architecture

2.1 The layer model

Seven layers. Each may only call downward. That constraint is what makes the Phase 2 swap a one-folder change instead of a rewrite.



The rules of the layer model

RULE	ENFORCED BY	WHY
Screens never import from <code>repositories/mock/</code>	Convention + review	Phase 2 replaces <code>mock/</code> . Any screen that reaches past <code>repositories/index.ts</code> breaks the swap
Primitives never import from <code>features/</code>	Convention	A primitive that knows about reservations is not a primitive
Primitives never import repositories	Convention	A component that fetches is not reusable
<code>lib/</code> imports nothing from <code>components/</code> or <code>features/</code>	Convention	It is the bottom of the dependency graph and must stay there
Business rules live only in <code>lib/rules.ts</code>	Review	A rule duplicated in a component is a rule that will drift

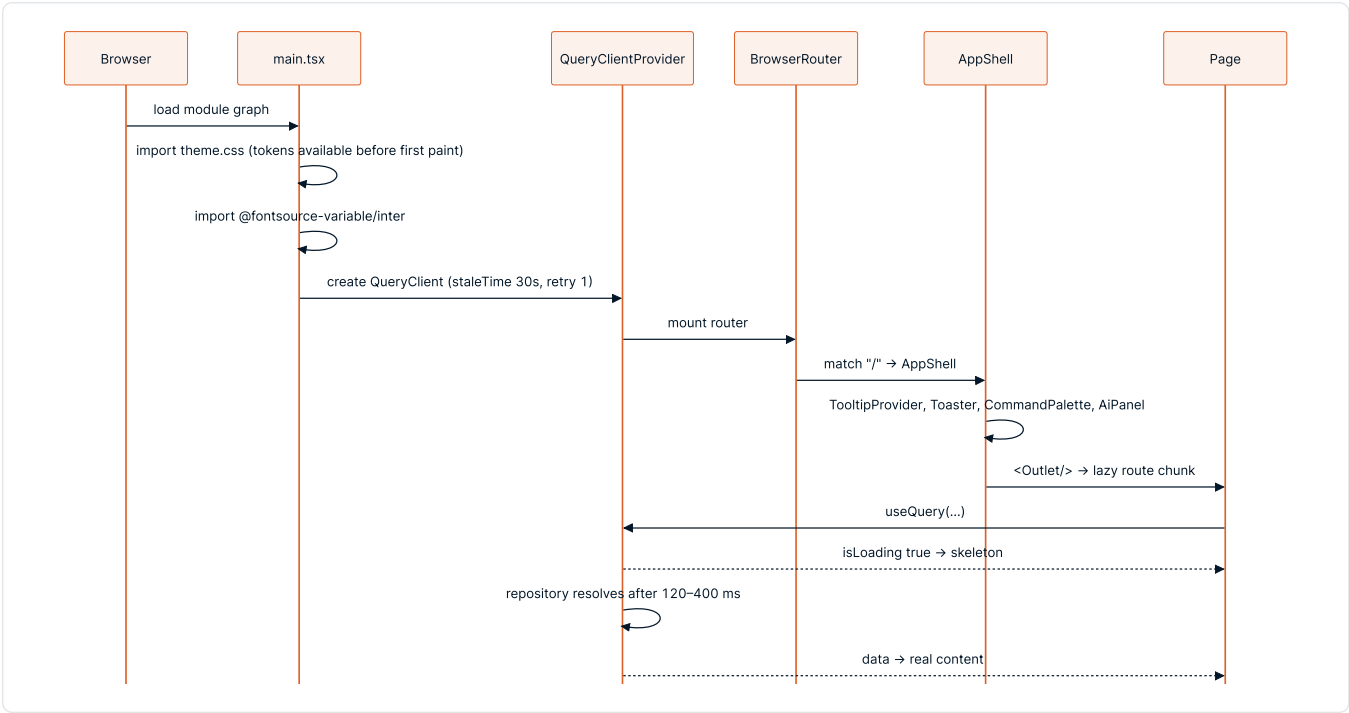
⚠ **The one deliberate exception:** several screens import `db` directly from `@/data/repositories` for cheap synchronous derived counts (for example, the invoice totals strip on `/finance/invoices`). This is a Phase 1 shortcut for values that do not warrant a query. Volume XIV §14.4 lists all 11 occurrences, because each becomes a real aggregate in Phase 2.

2.2 Directory map

```
D:\fidato crs\
├─ index.html           Entry document; sets <title> and favicon
├─ vite.config.ts       Aliases, chunking strategy
├─ tsconfig.app.json    strict: true, noUnusedLocals, noUncheckedIndexedAccess
├─ .npmrc + win-ca-bundle.pem  Local CA trust for this machine's TLS proxy
├─ docs/
│   └─ design-system.md  Working doc
│   └─ data-model.md     Working doc
│   └─ screen-inventory.md Working doc
│   └─ role-matrix.md    Working doc
│   └─ manual/           ← this manual
├─ src/
│   └─ main.tsx          Root: providers, router, query client
│   └─ routes.tsx        All 38 routes + permission guard
│   └─ styles/theme.css  Every design token. Single source of truth
│   └─ assets/brand/     logo-full.svg, logo-mark.svg
│   └─ lib/
│       └─ cn.ts         clsx + tailwind-merge
│       └─ format.ts     Money, dates, numbers, text – 20 functions
│       └─ permissions.ts Roles, resources, actions, matrix, scoping
│       └─ rules.ts       The 8 business rules
│       └─ session.ts     Zustand: current role + UI state
│   └─ components/
│       └─ ui/
│           └─ app/       AppShell, Sidebar, TopBar, RoleSwitcher,
│                           CommandPalette, AiPanel, navigation.ts
│   └─ features/
│       └─ shared/        Forbidden, NotFound, RouteFallback, useListState
│       └─ dashboard/     1 screen
│       └─ reservations/  5 screens
│       └─ crm/           8 screens
│       └─ hotels/        4 screens
│       └─ finance/       4 screens
│       └─ reports/       6 screens + ReportShell
│       └─ automation/    3 screens
│       └─ notifications/ 2 screens
│       └─ ai/            1 screen + responses.ts
│       └─ admin/         5 screens
│       └─ design-system/ 1 screen (living style guide)
├─ data/
│   └─ types.ts          46 exported types
│   └─ seed/
│       └─ index.ts       The generator
│       └─ hotels.data.ts 32 real properties from the fact-sheet PDFs
│       └─ names.ts       Name pools, industries, preferences
│       └─ random.ts      Seeded PRNG
│   └─ repositories/
│       └─ index.ts       ← the only import path screens use
│       └─ mock/
│           └─ store.ts    db, latency, runQuery
│           └─ index.ts    11 repositories
```

2.3 Application bootstrap

`src/main.tsx` is the whole startup sequence. Four providers, in this order, for reasons:



Why the query client sits outside the router: cache must survive navigation. Move it inside and every back-navigation refetches, which in a data-dense internal tool feels broken.

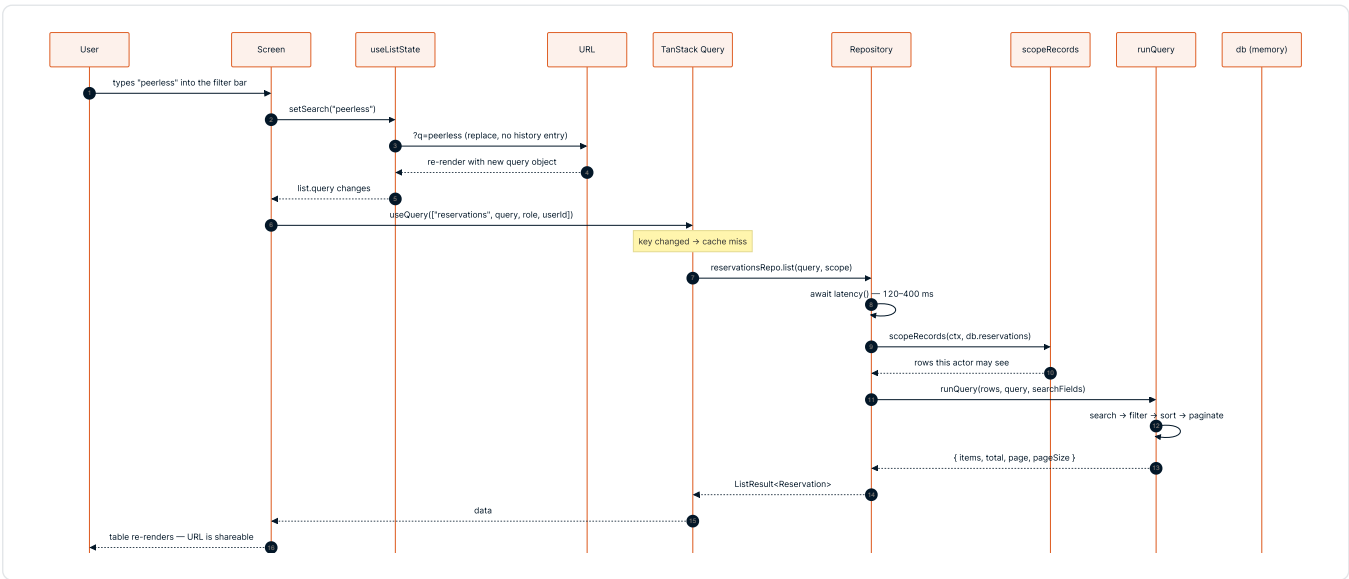
Why `theme.css` is imported first: tokens must exist before the first component renders, or the first paint flashes unstyled.

Query client configuration and its reasoning

OPTION	VALUE	REASONING
<code>staleTime</code>	30 s	Internal tool. Data changes slowly; refetching on every focus is noise. Long enough to make tab-switching feel instant, short enough that a colleague's change appears within a coffee refill
<code>retry</code>	1	The mock layer does not fail randomly. In Phase 2 one retry absorbs a transient Firestore hiccup without hiding a real outage behind four
<code>refetchOnWindowFocus</code>	false	An accountant with the invoice list open and a spreadsheet beside it should not see rows shuffle every time they alt-tab

2.4 The request lifecycle

This is the path every piece of data on screen has taken. Understanding it is most of understanding the system.



Two properties worth noticing

Scope is applied before search. A salesperson searching for a company they do not own finds nothing — not "access denied", just nothing, because the record was removed from the set before the search ever ran. This is the correct behaviour: it does not leak the existence of records the actor may not see.

The URL is the state. Search, filters, sort and page all live in the query string (§2.6). The React state is a mirror of the URL, not the other way round.

2.5 Routing and code splitting

All 38 routes are declared in `src/routes.tsx`. Every feature screen is `React.lazy`.

```
// src/routes.tsx
const ReservationsPage = lazy(() => import("@/features/reservations/ReservationsPage"));
// ... 33 more
```

The route tree has an unusual shape — a nested `<Routes>` inside a `path="*"` element:

```
<Routes>
  <Route element={AppShell} />
  <Route path="*" element={
    <Suspense fallback={RouteFallback} />
    <Routes>
      { /* all 38 routes */ }
    </Routes>
  } />
</Route>
</Routes>
```

Why nested rather than flat? Because the `<Suspense>` boundary must sit *inside* the shell, not around it. With a flat tree, every lazy route transition would unmount the sidebar and top bar while the chunk loaded — the whole frame would flash. Nesting puts the boundary below the persistent chrome, so only the content area shows the skeleton.

The permission guard

```
function Guard({ resource, children }: { resource: Resource; children: ReactNode }) {
  const role = useSession((s) => s.role);
  if (!canAccess(role, resource)) return <Forbidden resource={resource} />;
  return <>{children}</>;
}
```

Eleven lines, wrapping every protected route. It renders `Forbidden`, never a redirect and never a 404 — see ADR-11.

Bundle output

CHUNK	RAW	GZIPPED	CONTENTS
<code>index</code>	390.6 kB	120.4 kB	React, Router, Query, Zustand, shell
<code>BarChart</code>	359.1 kB	103.9 kB	Recharts — loaded only when a chart-bearing route is opened
<code>ui</code>	257.6 kB	81.4 kB	Radix primitives + the component library
<code>schemas</code>	100.4 kB	29.4 kB	Zod + react-hook-form resolvers
Largest screen	37.0 kB	11.3 kB	<code>RevenueReportPage</code>
Median screen	~6 kB	~2.2 kB	—

Recharts being its own chunk is the point of the splitting: it is the single heaviest dependency, and eleven of the thirty-eight routes never touch it.

2.6 URL as state

`src/features/shared/useListState.ts` puts list state in the query string.

```
/reservations?q=peerless&status=confirmed&sort=totalAmount&dir=desc&page=3
```

PARAMETER	MEANING
<code>q</code>	Free-text search
<i>(named keys)</i>	One per filter, e.g. <code>status</code> , <code>channel</code> , <code>tier</code>
<code>sort</code>	Field to sort by
<code>dir</code>	<code>asc</code> or <code>desc</code>
<code>page</code>	1-based; omitted when 1

Why not component state? Three reasons, in order of how often they matter:

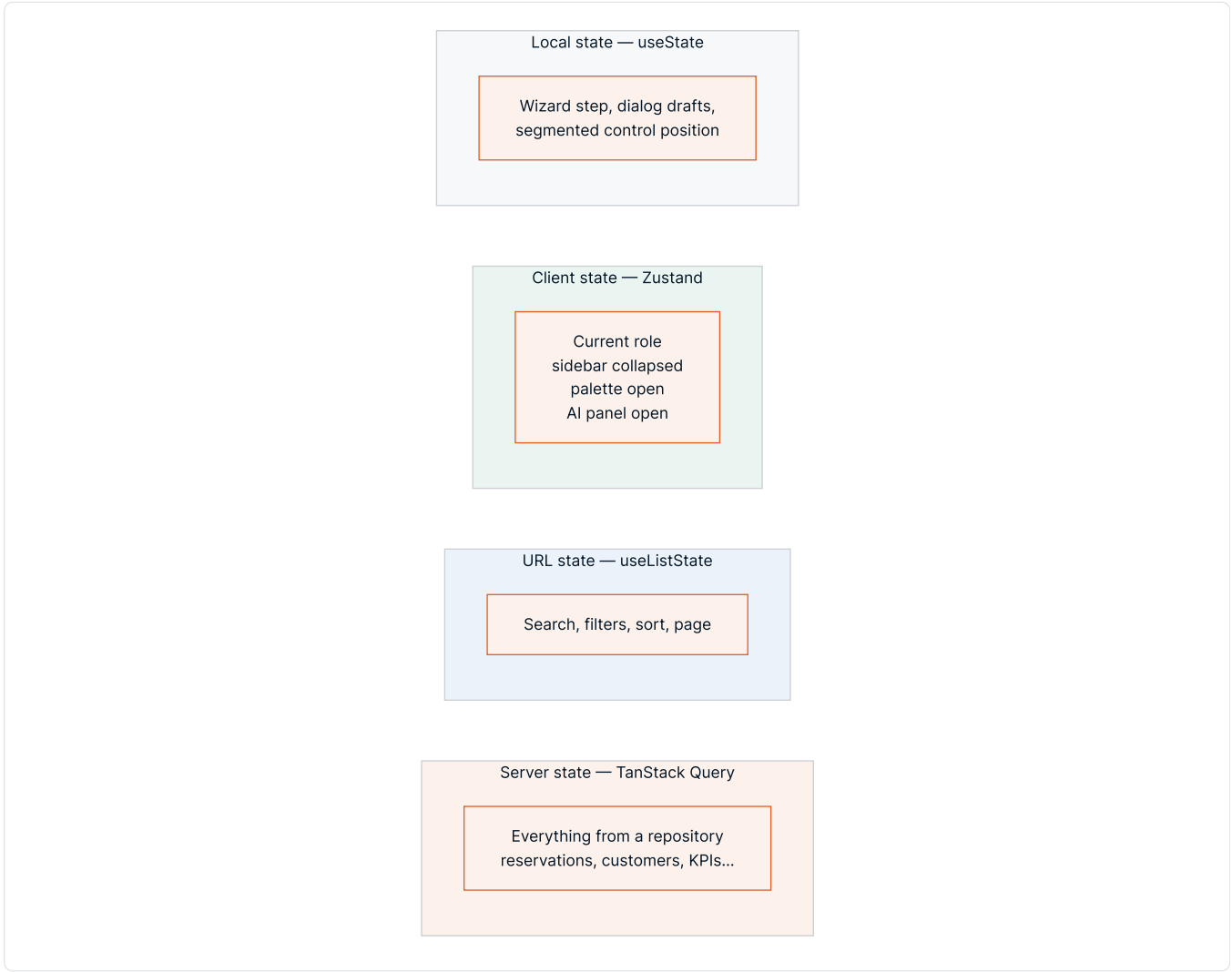
1. **A filtered view is a thing people send each other.** "Look at this" with a link is worth more than "filter by overdue then sort by due date".
2. **Refresh should not lose your place.** In a tool people keep open all day, losing a filter to an accidental reload is a real irritation.
3. **Back should work.** Browser back returning to the previous filter is what every user already expects.

The implementation uses `replace: true` , so typing in the search box does not create thirty history entries — only navigation does.

One nuance: the search input is *mirrored* in local state (`searchDraft`) so typing feels instant, while the URL is the authority. Without the mirror, every keystroke would round-trip through the router and the input would feel laggy.

2.7 State ownership

Three kinds of state, three different mechanisms. Choosing the wrong one is the most common architectural mistake in React applications, so the boundaries here are explicit.



STATE	MECHANISM	TEST FOR "DOES IT BELONG HERE?"
Server	TanStack Query	Does it come from a repository?
URL	<code>useListState</code>	Would you want to send it to a colleague?
Global client	Zustand	Do two distant components both need it?
Local	<code>useState</code>	Does anything outside this component care?

Zustand holds exactly two stores and deliberately no more:

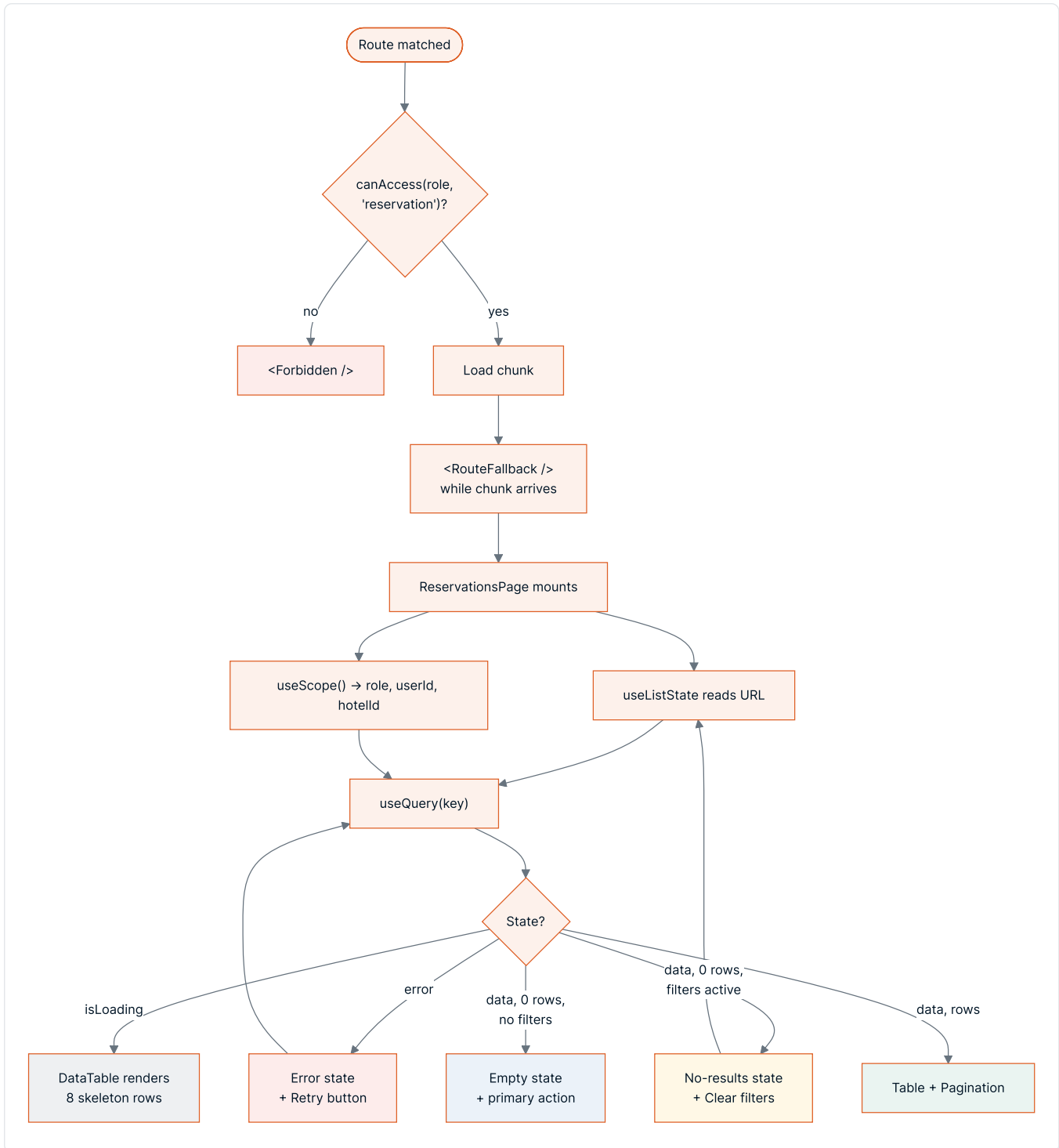
```
// src/lib/session.ts
export const useSession = create<SessionState>(){
  persist((set) => ({ role: "sales_manager", setRole: (role) => set({ role }) }),
    { name: "fidato.session" }),
};

export const useUi = create<UiState>(){
  persist((set) => ({ /* sidebar, mobile nav, palette, AI panel */ }), {
    name: "fidato.ui",
    partialize: (state) => ({ sidebarCollapsed: state.sidebarCollapsed }) as UiState,
  }),
};
```

⚠️ `partialize` matters. Without it, `commandOpen: true` would persist to localStorage, and the next page load would open with the command palette already up. Only the durable preference — sidebar collapsed — is worth surviving a reload.

2.8 The rendering pipeline for a typical screen

Taking `/reservations` as the worked example.



The distinction between **empty** and **no-results** is not pedantry. They need different words and different exits:

- *Empty* — "No reservations yet. Bookings raised by the sales team, the website or a travel agent all land here." Exit: **Raise the first booking**.
- *No results* — "No reservation matches the current search and filters." Exit: **Clear filters**.

Offering "Raise the first booking" to someone who has simply mistyped a filter is a small insult. Offering "Clear filters" to a genuinely empty system is confusing. Every list in the application distinguishes the two.

2.9 Build configuration

```
// vite.config.ts - the parts that matter
export default defineConfig({
  plugins: [react(), tailwindcss()],
  resolve: { alias: { "@": path.resolve(__dirname, "./src") } },
  build: {
    rollupOptions: {
      output: {
        manualChunks: {
          ui: [/* radix packages */],
          schemas: ["zod", "@hookform/resolvers", "react-hook-form"],
        },
      },
    },
  },
});
```

TypeScript runs at `strict: true` with two additional flags that caught real bugs during the build:

FLAG	WHAT IT CAUGHT
<code>noUnusedLocals</code>	Four dead imports left after refactors
<code>noUncheckedIndexedAccess</code>	Array access assumed non-empty in the calendar packing algorithm — would have thrown on an empty month

VOLUME III

Decision log

Twenty-four decisions. Each states the problem, the options that were genuinely considered, what was chosen, and — most importantly — **what it costs**. A decision record without a stated cost is marketing, not engineering.

#	DECISION	VOLUME
ADR-01	Build the frontend before the backend	I
ADR-02	React + Vite over Next.js	II
ADR-03	Tailwind v4 over CSS Modules or styled-components	IV
ADR-04	Hand-built components on Radix, not shadcn/ui	V
ADR-05	TanStack Query even with no server	II
ADR-06	Repository interface shaped to Firestore from line one	VIII
ADR-07	Simulated latency of 120–400 ms	VIII
ADR-08	Deterministic seed, reset on refresh	VII
ADR-09	Real property data from the fact-sheet PDFs	VII
ADR-10	Role switcher instead of a login page	IX
ADR-11	Forbidden, not 404, not redirect	IX
ADR-12	Show blocked actions, never hide them	I
ADR-13	Permissions and scoping as separate concepts	IX
ADR-14	Inter instead of the brand guide's Georgia + Arial	IV
ADR-15	Adding a success colour to the brand palette	IV
ADR-16	Hairline borders instead of shadows	IV
ADR-17	URL as the home of list state	II
ADR-18	Zustand for client state, not Context	II
ADR-19	Occupancy from the inventory model, not from reservations	VI
ADR-20	Reservations are cancelled, never deleted	IX
ADR-21	Duplicate detection warns; it never blocks	X
ADR-22	The assistant computes; it does not generate	X
ADR-23	Recharts over D3, Visx or Chart.js	X
ADR-24	No test framework in Phase 1	XIII

ADR-01

Build the complete frontend before any backend

Status: Accepted · Volume: I

Problem. The brief described a 20-document product engineering specification covering 1,200 pages that did not yet exist, and a system spanning CRM, reservations, finance, reporting and automation. Something had to be built first.

Options considered.

OPTION	FOR	AGAINST
Write the full PES first, then build	Complete specification; no rework	1,200 pages written against zero feedback. Most of it would be wrong, and being wrong on paper is invisible until you build

OPTION	FOR	AGAINST
Data model → API → UI (conventional)	Solid foundations; UI is thin	The expensive mistake in this project is a wrong <i>flow</i> , not a wrong query. This order defers the risky part
Frontend first with simulated data ✓	Discovers the real requirements by building them; flows can be reviewed in week one	Risk of assuming data the backend cannot cheaply supply
Thin vertical slice through all layers	Proves the stack end to end	Answers "can we build it" — never in doubt — not "what should it be"

Chosen: frontend first.

How the risk is managed. The stated risk — a UI that assumes impossible data — is controlled by ADR-06. The repository layer was written to Firestore's capabilities from the first line. There are no joins anywhere, because Firestore has no joins. Denormalised fields (`reservation.hotelName` , `reservation.customerName`) exist on the documents exactly as they will in Firestore.

Cost accepted. The seed engine (`src/data/seed/index.ts` , ~1,180 lines) is throwaway work. That is roughly one day, spent to make every subsequent review meaningful.

ADR-02

React + Vite over Next.js

Status: Accepted · **Volume:** II

Problem. Choose the application framework.

OPTION	FOR	AGAINST
React + Vite SPA ✓	Dev server starts in 600 ms; no server runtime to host; the product is behind a login and gains nothing from SSR	No SSR, no file-based routing, no built-in data layer
Next.js App Router	Server components, streaming, built-in routing	An internal tool behind auth has no SEO and no cold-visitor problem. Server components would fight the client-heavy, filter-heavy nature of every screen. Adds a Node runtime to host in Phase 2
Remix / React Router 7 framework mode	Good data loading story	Loaders assume a server. In Phase 1 there is none, and in Phase 2 Firestore is queried from the client
Vue / Svelte	Smaller, faster	The stated deliverable was "every React component". Team familiarity is React

Chosen: React 19 + Vite 8 SPA.

The deciding argument. Every screen in this product is a dense, filterable, interactive table or form behind authentication. That is the exact workload SSR helps least with, and the exact workload a client-side SPA with a good cache handles best.

Cost accepted. First load ships ~120 kB gzipped before any route chunk. For an internal tool opened once and kept open all day, this is the right trade.

ADR-03

Tailwind CSS v4 over CSS Modules or styled-components

Status: Accepted · **Volume:** IV

OPTION	FOR	AGAINST
Tailwind v4 ✓	<code>@theme</code> makes design tokens first-class CSS variables; no runtime; utilities keep spacing and colour honest by making deviation visible in review	Verbose clas
CSS Modules	Scoped, plain CSS	Tokens live ir
styled-components / Emotion	Co-located, dynamic	Runtime cost React 19 fric
Vanilla Extract	Type-safe, zero runtime	Build comple

Chosen: Tailwind v4.

What settled it. In v4, `@theme` is the token system:

```
@theme {
  --color-brand-orange: #df6128;
  --text-base: 0.875rem;
  --radius-md: 10px;
}
```

That single block generates the utilities *and* exposes the tokens as CSS variables for Recharts, inline styles and print rules. One source of truth, no synchronisation step. With CSS Modules the same tokens would exist twice.

Cost accepted. `className` strings get long. Mitigated by `cn()` (clsx + tailwind-merge) and by extracting variants into lookup objects (see `Button.tsx`'s `VARIANTS` / `SIZES` maps) rather than inlining conditionals.

ADR-04

Hand-built components on Radix primitives, not shadcn/ui

Status: Accepted · **Volume:** V

OPTION	FOR	AGAINST
Radix primitives, styled by us ✓	Full accessibility for free; zero opinion about appearance; every visual decision is deliberate	~28 components to write
shadcn/ui	Fast start; same Radix underneath	Its default look is <i>recognisable</i> . The brief asked for a system carrying Fidato's identity — shipping a widely-used default is the opposite. Restyling it to the brand costs about as much as writing it
MUI / Ant Design	Enormous component set	Both carry a strong, foreign design language. Fighting Material Design to look like Fidato is more work than not using it
Fully hand-rolled, no Radix	Total control	Would mean re-implementing focus trapping, roving tabindex, dismissable layers and typeahead. This is where accessibility bugs live

Chosen: Radix primitives + our own styling.

The reasoning. The hard part of a component library is behaviour, not appearance. Radix gives away the hard part. Appearance is precisely what should *not* be inherited when the deliverable is a branded design system.

Cost accepted. 28 components, roughly 2,400 lines. Volume V documents each.

ADR-05

TanStack Query, despite there being no server

Status: Accepted · **Volume:** II

Problem. Phase 1 has no network. `useState` + `useEffect` would work.

OPTION	FOR	AGAINST
TanStack Query ✓	Phase 2 becomes a change of fetcher, nothing else. Caching, dedupe, invalidation, retry and loading/error states all arrive already built	A dependency doing "nothing" in Phase 1
<code>useState</code> + <code>useEffect</code>	No dependency	Every screen hand-rolls loading and error state. Phase 2 would then rewrite all 34 screens
SWR	Lighter	Weaker mutation and invalidation story, which this app leans on hard (approve, cancel, merge, import all invalidate several keys)
Redux Toolkit Query	Integrated	Pulls in Redux for an app with almost no global client state

Chosen: TanStack Query.

The argument. The dependency is not doing nothing — it is defining the *shape* of every screen's data handling. Because `useQuery` is already there, `isLoading` already drives a skeleton and `error` already drives a retry button on all 34 screens. In Phase 2 the only change is what `queryFn` calls.

Evidence it was right. Query key design already encodes the scope:

```
queryKey: ["reservations", list.query, scope.role, scope.userId]
```

Switching role changes the key, which invalidates automatically. No manual cache clearing anywhere in the codebase.

ADR-06

The repository interface is shaped to Firestore from the first line

Status: Accepted · Volume: VIII

Problem. A mock data layer can be anything. If it is shaped conveniently, the Phase 2 migration becomes a rewrite.

Chosen. Every repository method mirrors an operation Firestore can actually perform.

CONSTRAINT	CONSEQUENCE IN THE CODE
Firestore has no joins	Documents are denormalised. <code>reservation</code> carries <code>customerName</code> , <code>companyName</code> , <code>hotelName</code> , <code>hotelCity</code> , <code>ownerName</code>
Firestore filters are equality/range on indexed fields	<code>applyFilters()</code> does equality only. No <code>LIKE</code> , no computed predicates
Firestore has no full-text search	<code>matchesSearch()</code> is a client-side substring scan over named fields — exactly what Phase 2 will do until Algolia or Typesense is added
Firestore paginates by cursor	<code>paginate()</code> is offset-based, which is the one honest deviation. Documented in Volume XIV §14.3 as requiring change
Firestore writes are per-document	Every write method touches one document, plus explicit denormalisation updates

Cost accepted. Some screens are less efficient than a SQL backend would allow. The Occupancy report aggregates in the client because Firestore cannot `GROUP BY` . That is not a flaw in the mock — it is an accurate preview of Phase 2, which is the point.

⚠️ If this ADR is ever violated for convenience, the Phase 2 estimate in Volume XIV becomes worthless.

ADR-07

Simulated latency of 120–400 ms

Status: Accepted · Volume: VIII

```
// src/data/repositories/mock/store.ts
const MIN_LATENCY = 120;
const MAX_LATENCY = 400;
```

OPTION	CONSEQUENCE
No latency	Skeletons never appear. Loading states are written, never seen, and rot. The app feels unrealistically instant, so nobody notices that a screen fires six sequential queries
Fixed 200 ms	Better, but uniform. Real networks jitter, and jitter is what exposes race conditions
Random 120–400 ms ✓	Skeletons are exercised on every load. Out-of-order resolution surfaces stale-closure bugs during development, not in production
1,000 ms+	Realistic for a bad connection, but makes the app tedious to review

Chosen: uniform random 120–400 ms, plus an extra 80 ms on writes to reflect a real round trip.

What this caught during the build. The dashboard fires four independent queries. With zero latency they appeared to resolve together. With jitter it became obvious that the KPI row, chart, day sheet and approvals each needed their own skeleton rather than one shared loading flag.

ADR-08

Deterministic seed, and data resets on refresh

Status: Accepted · Volume: VII

Two decisions in one.

Deterministic: a fixed-seed PRNG (`src/data/seed/random.ts`) rather than `Math.random()` .

Resets: the store is plain memory, not localStorage or IndexedDB.

OPTION	FOR	AGAINST
Fixed seed + memory ✓	Identical data on every load. Screenshots stay comparable. A bug reported on "the Peerless Inn booking" is reproducible by anyone	Work is lost on refresh
<code>Math.random()</code>	Feels varied	Nobody can reproduce anything. "It looked wrong yesterday" becomes unfalsifiable
Persist to localStorage	Work survives	Every reviewer's copy diverges within a day. Demos become unpredictable, and stale persisted data outlives schema changes
Persist to IndexedDB	Larger, structured	All of the above, plus migration code for a database being deleted in Phase 2

Chosen: deterministic generation into memory.

The argument for losing work on refresh. In a review build, *reproducibility beats persistence*. Every reviewer sees the same 1,100 reservations, the same four duplicate groups, the same 66 pending approvals. That property is worth far more than remembering a test booking someone made five minutes ago.

This is stated on-screen so it is never a surprise — the README and Volume I both say it plainly.

ADR-09

The 32 properties are real data extracted from the fact sheets

Status: Accepted · **Volume**: VII

Problem. Seed a portfolio of hotels.

OPTION	FOR	AGAINST
Extract from the 32 fact-sheet PDFs ✓	Reviewers recognise the properties. Room counts, room mixes, amenities and distances are all true. Bugs surface against real distributions	An extraction pass over 32 PDFs
Generate plausible fake hotels	Fast	"Grand Plaza Hotel, Mumbai — 100 rooms" teaches nobody anything. A reviewer cannot tell whether a screen handles their actual inventory
A handful of real ones, rest generated	Cheaper	The inconsistency is worse than either extreme; you never know which rows to trust

Chosen: all 32 extracted into `src/data/seed/hotels.data.ts` .

What came out of the PDFs: name, city, state, star rating, category, description, total rooms, room-mix breakdown, features, facilities, amenities, things to do, and road distances to named landmarks.

So *Marigold Banquets 'n' Conventions, Pune* really is 150 rooms — 5 Suite, 115 Deluxe, 30 Business Class — and Shaniwar Wada really is 11.4 km away.

The payoff, concretely. The real portfolio spans a 17-key Udaipur property and a 236-key Agra hotel. That 14× spread immediately exposed that ranking properties by revenue flatters scale, which is why the Property Performance report offers *revenue per room* as an equal sort (Volume X §10.24).

ADR-10

A role switcher instead of a login page

Status: Accepted · **Volume**: IX

Problem. The permission model is the most important thing to review, and Phase 1 has no authentication.

OPTION	FOR	AGAINST
"Viewing as..." role switcher ✓	Move between 8 roles in one second. Reviewing the permission model becomes trivial	Not how the real product behaves

OPTION	FOR	AGAINST
Fake login page with 8 hardcoded accounts	Looks like the real thing	Eight sign-outs and sign-ins to review eight roles. Everyone stops after two
No role concept in Phase 1	Simplest	The permission model is a third of the product's complexity. Deferring it means discovering its consequences in Phase 2, which is exactly backwards
Real Firebase Auth now	Real	Drags the entire Phase 2 backend forward

Chosen: the role switcher, in the top bar, on every screen.

Why this is better than a login page for this purpose. A login page proves you can authenticate — never in doubt. The switcher proves the *authorisation model works*, which is the actual risk. Reviewing "does a hotel manager correctly lose access to rate editing" becomes a one-second operation instead of a two-minute one.

Verified behaviour: switching from Super Admin to Hotel Manager takes navigation from 16 items to 7, changes the dashboard to a property day-sheet, and turns 12 Edit buttons on the rate-plan screen into 12 "Locked" markers.

🔗 In Phase 2 the same Zustand store is fed by Firebase Auth, and the switcher becomes dev-only. The store's interface does not change.

ADR-11

Forbidden, not 404, and never a silent redirect

Status: Accepted · **Volume:** IX

Problem. What should a route the current role cannot access render?

OPTION	CONSEQUENCE
Redirect to dashboard	The user's click appears to do nothing. Feels broken. The most common choice and the worst
404 Not found	Actively misleading. The page exists; you cannot see it. Sends people to look for a bug that is not there
Forbidden, naming the role and the resource ✓	Truthful, and teaches the permission model at the moment it becomes relevant

Chosen: a dedicated `Forbidden` screen:

Not available for your role You are viewing the platform as **Hotel Manager**, which has no access to **invoices**. Switch role from the top bar to see how the platform looks for another team.

The security objection, addressed. Revealing that a resource exists is a real consideration in a public product. This is an internal tool where the *existence* of an invoices module is not secret — it is on the org chart. What is protected is the data, and none is shown.

ADR-12

Show blocked actions; never hide them

Status: Accepted · **Volume:** I

The product-wide principle. Applied consistently:

SITUATION	HIDDEN APPROACH	WHAT THIS PRODUCT DOES
Hotel manager on rate plans	Edit buttons absent	Banner explains pricing is owned by revenue; each row shows Locked
Completed reservation	Cancel button absent	Button present, disabled, tooltip: "Completed reservations are locked"
Salesperson viewing approvals	Page hidden	Page visible; a note explains approving is limited to sales managers and admins
Viewer on settings	Form absent	Form visible, inputs disabled, note explains why

Cost. More markup, more copy, more conditional rendering. Every restriction needs a sentence written for it.

Why it is worth it. Three reasons, in order of how much they matter:

1. A product that hides things **looks broken** to whoever cannot see them.
2. A rule never surfaced is a rule nobody learns — so it gets asked about forever.

3. **A visible restriction gets reported when it is wrong.** A hidden one does not. This is the strongest argument: the design makes permission bugs *self-reporting*.

ADR-13

Permissions and row-level scoping are separate concepts

Status: Accepted · Volume: IX

Problem. "A salesperson can edit customers, but only their own" mixes two different questions.

Chosen. Split them:

```
can(role, action, resource)    // May this role touch this kind of thing?
scopeRecords(ctx, records)    // Which records may this actor see?
```

Why not one combined check? Because they run at different times and different granularity. `can()` is synchronous, cheap and drives *rendering* — nav items, buttons, route guards. `scopeRecords()` runs inside the repository against a record set and drives *data*. Fusing them would mean either fetching everything and filtering in the UI (leaky and slow) or making every permission check async (unusable for rendering).

The security property this gives. Scope is applied *before* search and filtering, inside the repository. A salesperson searching for a company they do not own gets no results — without any "access denied" message, because from their view the record simply is not there. It does not leak existence.

ADR-14

Inter, not the brand guide's Georgia + Arial

Status: Accepted, flagged for brand review · Volume: IV

Problem. The visual identity guide (p.12) specifies two font duos: ITC Charter + Helvetica Neue for design software, Georgia + Arial for Office. The brief asked for the feel of premium Apple software.

OPTION	FOR	AGAINST
Strict Georgia + Arial	100% guide-compliant; no licensing question	Reads as an Office document, not a product. Georgia has no variable axis, so weight control is coarse. Arial at 13 px in a dense table is noticeably less legible than a modern UI face
ITC Charter + Helvetica Neue	Also guide-compliant	Both are commercially licensed for web use. Neither was supplied
Inter Variable everywhere, Georgia for print ✓	Closest free analogue to SF Pro; designed for UI at small sizes; variable weight and optical sizing; tabular figures	A documented departure from the guide
System font stack	Zero payload	Renders Segoe UI on Windows, SF on macOS. The product would look materially different per machine — unacceptable for a brand system

Chosen: Inter Variable, self-hosted via `@fontsource-variable/inter`.

Where Georgia survives. Print. Invoice headers and report covers use `.print-serif`:

```
@media print {
  .print-serif { font-family: var(--font-serif); }
}
```

This is not a token gesture. Printed documents are exactly where a serif still earns its keep, and it keeps the brand's typographic voice on the artefacts that leave the building.

Why self-hosted rather than Google Fonts CDN. Three reasons: it works offline, it removes a third-party request from an internal tool's critical path, and it is not subject to a CDN being blocked by the corporate proxy on this network.

⚠️ **This is a real deviation from an approved brand document.** It is recorded in `docs/design-system.md` and must be signed off by the brand team, not quietly absorbed.

ADR-15

Adding a success colour to the brand palette

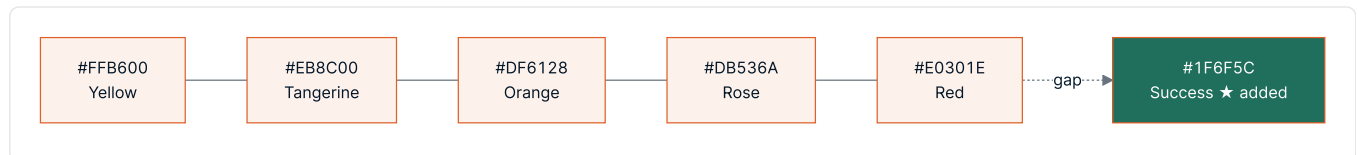
Status: Accepted, flagged for brand review · **Volume:** IV

Problem. The brand palette is five warm colours — yellow, tangerine, orange, rose, red — plus greys. There is no green. An ERP must express *confirmed*, *paid*, *reconciled*, *synced*, *active*, *success*.

OPTION	CONSEQUENCE
Use tangerine for success	<i>Pending</i> and <i>confirmed</i> become the same colour. The status vocabulary collapses — the single most-used signal in the product stops signalling
Use grey for success	"Confirmed" reads as disabled or inactive. Semantically backwards
Use only iconography, no colour	Forces reading every row. Defeats the purpose of a status column, which exists to be scanned
Add one green ✓	Status vocabulary works. One documented addition

Chosen: `--color-success: #1f6f5c`.

How the value was chosen. Not an off-the-shelf green. A saturated `#22c55e` beside `#df6128` looks like a traffic light and fights the warm palette. `#1f6f5c` is desaturated and pushed toward teal so it reads as a *quiet confirmation* rather than a competing accent. It carries 5.8:1 contrast on white, clearing WCAG AA for text.



A second addition, `--color-info: #2b6cb0`, was made on the same reasoning for *in progress* and *checked in*. It is drawn from the ink family rather than the warm set so it reads as neutral information rather than a fifth brand colour.

ADR-16

Hairline borders instead of shadows

Status: Accepted · **Volume:** IV

Chosen. Cards, tables, inputs and panels use a 1 px `--color-grey-200` border. Shadows are reserved for things that genuinely float: dialogs, popovers, dropdown menus, toasts, the command palette.

Why. A shadow is a claim about elevation. When every card casts one, the claim is meaningless and the interface acquires a soft, undifferentiated haze — the visual signature of a Bootstrap admin template. Reserving shadows for overlays means that when something *does* cast a shadow, it reads immediately as sitting above the page.

Three shadow tokens exist, and only three:

```
--shadow-overlay: 0 12px 32px -8px rgb(3 23 40 / 0.18), 0 2px 8px -2px rgb(3 23 40 / 0.08);
--shadow-popover: 0 8px 24px -6px rgb(3 23 40 / 0.14), 0 1px 4px rgb(3 23 40 / 0.06);
--shadow-raise:   0 1px 2px rgb(3 23 40 / 0.06);
```

All three are tinted with the brand ink (`rgb(3 23 40)`) rather than pure black. Black shadows on a warm palette read as grey smudge; ink-tinted ones read as depth.

ADR-17

The URL owns list state

Covered in Volume II §2.6. Summary of the alternatives:

OPTION	AGAINST
Component <code>useState</code>	Filters lost on refresh; views cannot be shared; back button does nothing

OPTION	AGAINST
Zustand store	Global state for something inherently per-screen; still not shareable
URL query string ✓	Verbose keys; needs care to avoid flooding history (solved with <code>replace: true</code>)

ADR-18

Zustand for client state, not Context

OPTION	AGAINST
React Context	Every consumer re-renders when any part of the value changes. The role lives in context consumed by the sidebar, top bar, every guard and most screens — a sidebar toggle would re-render the entire tree
Redux Toolkit	Substantial ceremony for two stores holding five values
Jotai / Valtio	Fine choices; Zustand's selector API and <code>persist</code> middleware fit exactly what was needed
Zustand ✓	One more dependency (1.2 kB)

The deciding feature is selector subscriptions:

```
const role = useSession((s) => s.role); // re-renders only when role changes
```

ADR-19

Occupancy comes from the inventory model, not from reservations

Status: Accepted · Volume: VI · Origin: defect D-04

Problem. The first implementation computed occupancy as $\text{Fidato room-nights} \div (\text{total rooms} \times \text{days})$. It reported **1%**, and the dashboard looked broken.

The diagnosis. The number was not wrong arithmetically; the *metric* was wrong. Fidato books a slice of each partner hotel — other channels sell the rest. Dividing Fidato's bookings by the hotel's entire inventory answers a question nobody asked.

OPTION	CONSEQUENCE
Inflate the seed until the number looks good	Dishonest. The portfolio is 1,969 rooms; a realistic 60% occupancy over 450 days is ~530,000 room-nights, needing ~90,000 reservations. Not generatable client-side, and faking it would misrepresent the business
Remove occupancy from the product	It is a genuine operational metric. Hotel managers need it
Relabel it "Fidato share of inventory"	Honest but useless — a number nobody acts on
Take occupancy from the inventory model; label the Fidato slice separately ✓	Both numbers are true and both are useful

Chosen: occupancy reads from `buildInventory()`, which models each property's real position across *all* channels at 55–82%. The Fidato-booked portion is surfaced beside it as **"Fidato room nights"**.

```
function occupancyForHotel(hotelId: string, days = 30): number {
  const rows = buildInventory(hotelId, days);
  const capacity = rows.reduce((s, r) => s + r.totalRooms, 0);
  const booked = rows.reduce((s, r) => s + r.booked, 0);
  return capacity > 0 ? (booked / capacity) * 100 : 0;
}
```

The generalisable lesson. When a metric reads absurdly, check whether the *denominator* describes something you control before you assume the numerator is too small.

ADR-20

Reservations are cancelled, never deleted

Status: Accepted · **Business rule** BR-01 · **Volume:** IX

There is **no delete control for a reservation anywhere in the product**. The rule is enforced by absence, not by a confirmation dialog.

Why. The commercial history of a booking has to outlive the booking:

- A guest disputes a charge six months later. The cancelled reservation is the evidence.
- Commission is reconciled with the property quarterly. A deleted booking silently changes what is owed.
- The cancellation report is a management metric. Deletion would make cancellations *invisible*, which inverts the metric's purpose.

Consequence in the data model. `status: "cancelled"` plus `cancellationReason`, `cancelledBy`, `cancelledAt`. The record stays in every list, greyed, filterable.

ADR-21

Duplicate detection warns; it never blocks

Status: Accepted · **Volume:** X

Problem. BR-06 says customer email and phone must be unique. What happens when someone enters a duplicate?

OPTION	CONSEQUENCE
Hard block on save	The real world has genuine collisions — two colleagues sharing a company switchboard number, a couple sharing an email. A hard block leaves the user stuck with no path forward
Silent allow	The rule is not a rule
Warn inline, allow save, resolve on the merge screen ✓	Honest about the real world; keeps the data clean via a dedicated tool

Chosen: live inline warning under the field, with a link to `/crm/merge` :

⚠ Another customer already has this email. You can still save, then resolve it on the duplicates screen.

The import wizard applies the same philosophy with one sharpening: duplicates *inside the uploaded file* are hard errors (a file containing the same person twice is a mistake in the file), while collisions with *existing* records are warnings.

ADR-22

The assistant computes; it does not generate

Status: Accepted · **Volume:** X

Problem. Phase 1 has no backend, therefore no API key, therefore no language model.

OPTION	CONSEQUENCE
Omit the AI screen	The module is in the brief. Omitting it leaves the biggest UX question — how does an assistant fit into this product — unanswered until Phase 2
Hardcoded lorem-ipsum responses	Reviewers cannot judge usefulness. Worse, quoted figures would contradict the dashboards, undermining trust in both
Ship an API key in the client	Unacceptable
Compute real answers from live seed data ✓	Every figure the assistant quotes matches Reports exactly, because both read the same store

Chosen: `src/features/ai/responses.ts` computes answers from `db`.

Ask "How did we perform this month?" and it returns real portfolio figures — live reservation count, booked value, room nights, average booking value, cancellation rate, active properties — all traceable to the same numbers on `/reports/revenue`.

Every response carries a footnote:

Generated from live platform data. Phase 1 uses scripted analysis, not a language model.

🔗 The component contract is what Phase 2 keeps. `answerFor(question): ChatTurn` becomes an async call to a real completion; nothing in `AiPage.tsx` changes shape.

ADR-23

Recharts over D3, Visx or Chart.js

OPTION	AGAINST
Recharts ✓	104 kB gzipped — the heaviest dependency. Mitigated by isolating it in its own chunk, loaded only by chart-bearing routes
D3 directly	Maximum control, but every chart becomes bespoke. Six report screens would each grow an axis implementation
Visx	Lower-level D3 primitives for React; excellent, but more assembly per chart than this project needs
Chart.js	Canvas-based — charts are then invisible to the accessibility tree and to text selection, and do not print cleanly. This product prints reports

The deciding factor. Recharts renders SVG. That makes charts printable, selectable and inspectable — and the print path is a real requirement here (report covers, invoices).

ADR-24

No test framework in Phase 1

Status: Accepted, with reservations · Volume: XIII

OPTION	FOR	AGAINST
No framework; verify via typecheck, build and browser walkthrough ✓	Phase 1's deliverable is a UI for review, and the UI changed constantly. Tests written against churning markup are rewritten more often than they catch anything	No regression safety net
Vitest + Testing Library from the start	Real safety	Most value would be in testing the rules layer, which is small and stable — a narrow win for a broad cost during heavy churn
Playwright E2E	Tests the real flows	Slowest to write; most brittle against a UI in flux

Chosen: none in Phase 1 — but this is the weakest decision in this log, and it is flagged as such.

What stands in for tests today: `tsc --noEmit` at `strict` with `noUnusedLocals` and `noUncheckedIndexedAccess`; a clean production build; and the browser walkthrough recorded in Volume XIII.

What this does not cover, honestly: the reservation wizard end-to-end, the approve/cancel dialogs, and the merge action were **not** click-tested, because the automated browser used for verification did not deliver trusted input events to the React root (Volume XIII §13.4). They are typechecked and structurally verified, not behaviourally verified.

🔗 **Recommendation for Phase 2, first task, before any Firebase work:** Vitest over `src/lib/rules.ts` and `src/lib/permissions.ts`. That is ~160 assertions covering the permission matrix and all 8 business rules, against code that is now stable. It is the highest value-per-hour testing available in this codebase.

VOLUME IV

Design system

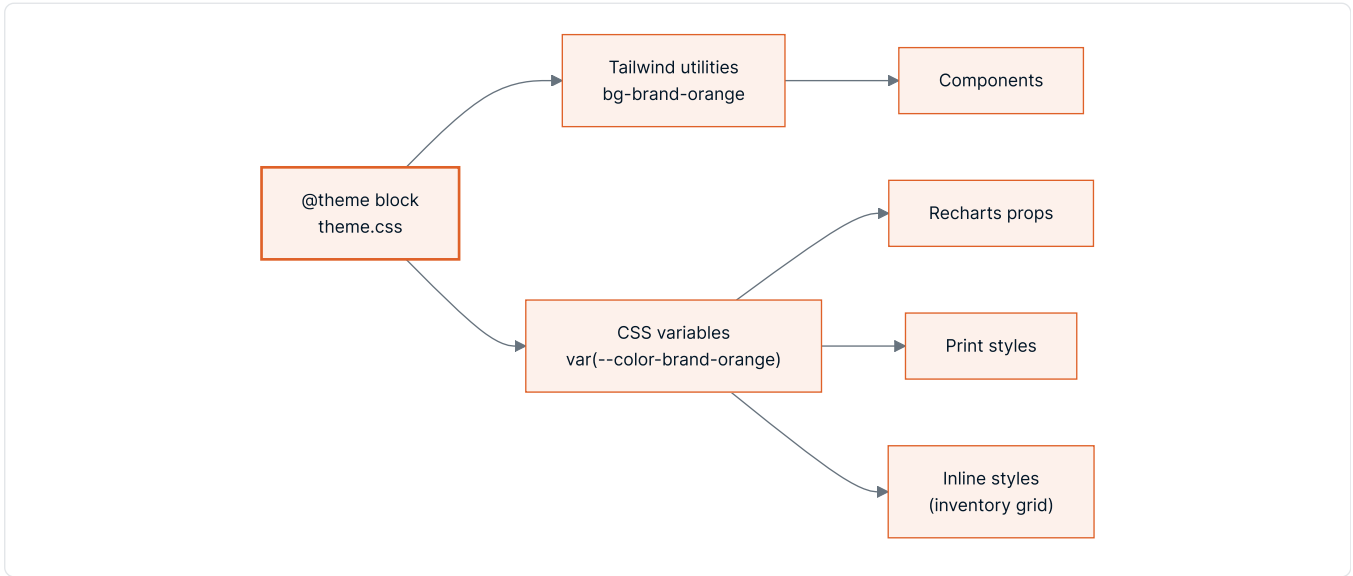
Source of truth: `src/styles/theme.css` . Living proof: `/design-system` in the running app, which renders every token and component from the real modules and therefore cannot drift.

4.1 How the token system works

Tailwind v4 makes `@theme` the token layer. One declaration produces both a utility class and a CSS custom property:

```
@theme {
  --color-brand-orange: #df6128;
}
```

gives you `bg-brand-orange` , `text-brand-orange` , `border-brand-orange` and `var(--color-brand-orange)` for Recharts, inline styles and print rules.



This is the whole reason for ADR-03. With CSS Modules these two representations would be separate systems needing manual synchronisation, and they would drift.

4.2 Colour — complete reference

Brand ink (Fidato Black family)

TOKEN	HEX	SWATCH	USED FOR
ink-950	#010d17		Modal backdrops at 25% opacity
ink-900	#031728		Fidato Black. Primary text, headings, sidebar
ink-800	#0b2135		Sidebar hover states
ink-700	#142b3a		The logo wordmark's own colour (see §4.9)
ink-600	#24384a		Sidebar borders

Grey ramp

TOKEN	HEX	FROM THE GUIDE	USED FOR
grey-700	#354552	✓ Dark Grey	Body text on white
grey-600	#4c5b67	derived	Secondary body
grey-500	#67737e	✓ Medium Grey	Supporting text, hints

TOKEN	HEX	FROM THE GUIDE	USED FOR
grey-400	#9aa2a9	✓ Grey	Tertiary text, icons, placeholders
grey-300	#ccd0d4	✓ Light Grey	Input borders, disabled
grey-200	#e2e5e8	derived	The hairline border colour
grey-100	#eef0f2	derived	Table row dividers, chart gridlines
grey-50	#f7f8f9	derived	Page background

Four of these are directly from the guide (p.13). The other four are interpolated because an interface needs more steps than a print palette does — specifically a border colour lighter than Light Grey and a page background lighter still.

The five warm brand colours

TOKEN	HEX	GUIDE NAME	SEMANTIC ROLE IN THIS PRODUCT
brand-yellow	#fffb600	Yellow	Pending, warning, awaiting approval
brand-tangerine	#eb8c00	Tangerine	Secondary accent, second chart series
brand-orange	#df6128	Orange	Primary action, active navigation
brand-rose	#db536a	Rose	Attention, no-show
brand-red	#e0301e	Red	Destructive, error, overdue

Why orange became the primary action colour rather than red or yellow: it sits in the middle of the warm range, so it is distinguishable from both the warning (yellow) and the destructive (red) ends. A primary action adjacent to the destructive colour is a design that produces mis-clicks.

Tints

Soft surfaces for badges and banners. Each is its parent colour at roughly 8% and 20% over white.

TOKEN	HEX	USED FOR
brand-orange-50	#fdf1eb	Active nav background, accent banners
brand-orange-100	#f9dccc	Selection highlight, progress track
brand-orange-600	#c4521d	Primary button hover
brand-orange-700	#a34314	Primary button active
brand-yellow-50	#fff8e6	Approval-needed banners
brand-yellow-100	#ffedbf	Approval banner borders
brand-rose-50 / -100	#fdeef0 / #f8d4da	No-show pills
brand-red-50 / -100	#fdeceb / #f9d0cc	Error banners, overdue pills
brand-tangerine-50	#fd4e6	Secondary accent surfaces

The two documented additions

```
/* The brand guide has no success colour. An ERP cannot express
   confirmed / paid / synced without one. */
--color-success: #1f6f5c;
--color-success-50: #eaf4f1;
--color-success-100: #cde5de;
--color-success-600: #1a5f4f;

/* Info - neutral blue drawn from the ink family, not the warm set */
--color-info: #2b6cb0;
--color-info-50: #ebf2f9;
--color-info-100: #d2e2f2;
```

Rationale in [ADR-15](#). Both are flagged for brand review.

The logo gradient

```
--color-accent-from: #fe611f;
--color-accent-to:   #f4bf54;
```

Sampled from the flame mark in `FH SVG.svg`. Exposed as the `brand-gradient` utility.

Where it may appear: the logo mark itself, the collapsed sidebar rail, the AI assistant avatar, and exactly one hero stat on the dashboard.

Where it may never appear: buttons. A gradient primary button would compete with the flat `brand-orange` primary and destroy the single-primary-action-per-view rule.

4.3 Contrast audit

All values against the surface they actually sit on.

FOREGROUND	BACKGROUND	RATIO	WCAG AA
ink-900	white	16.8:1	✓ AAA
grey-700	white	9.2:1	✓ AAA
grey-600	white	6.9:1	✓ AAA
grey-500	white	4.9:1	✓ AA
grey-400	white	2.8:1	⚠ decorative / large only
brand-orange	white	3.6:1	⚠ large text and UI only
white	brand-orange	3.6:1	⚠ ≥16 px or bold
success	white	5.8:1	✓ AA
brand-red	white	4.6:1	✓ AA
info	white	5.4:1	✓ AA
white	ink-900	16.8:1	✓ AAA

⚠ **Two known constraints, handled deliberately:**

- `grey-400` is used only for icons, placeholders and disabled text — never for content a user must read.
- `brand-orange` on white clears AA for *large* text and UI components but not for body copy. It is therefore used for links and 14 px+ medium-weight labels, never for paragraphs. White on `brand-orange` (primary buttons) is at 14 px medium, which qualifies as UI component text under WCAG 1.4.11.

4.4 Typography

The stack

```
--font-sans: "Inter Variable", "Inter", -apple-system, BlinkMacSystemFont,
             "Segoe UI", Roboto, sans-serif;
--font-serif: Georgia, "ITC Charter", "Times New Roman", serif;
--font-mono: ui-monospace, "SF Mono", "Cascadia Mono", Menlo, monospace;
```

Self-hosted via `@fontsource-variable/inter`. Rationale in [ADR-14](#).

The scale

Eleven steps. Deliberately more granular than a typical 6-step scale, because dense data interfaces need fine control between 11 px and 15 px — the range where most of this product lives.

TOKEN	REM	PX	LINE HEIGHT	USED FOR
text-2xs	0.6875	11	16	Pills, eyebrows, table micro-labels
text-xs	0.75	12	18	Captions, footnotes, timestamps
text-sm	0.8125	13	20	Secondary text, hints, sub-rows

TOKEN	REM	PX	LINE HEIGHT	USED FOR
text-base	0.875	14	22	Body, table cells, form inputs
text-md	0.9375	15	24	Card titles, section headings
text-lg	1.0625	17	26	Sub-page headings
text-xl	1.25	20	28	Page titles
text-2xl	1.5	24	32	KPI figures
text-3xl	1.875	30	36	Reserved
text-4xl	2.25	36	40	Reserved

Why 14 px body and not 16 px. 16 px is the correct default for reading text on the web. This is not reading text — it is an operational interface where a table row must show eight columns without truncation and a user scans rather than reads. 14 px with a 22 px line height is the density point at which an internal tool stops feeling like a website. Every major professional tool in this category sits at 13–14 px for the same reason.

Letter spacing

```
h1, h2, h3, h4 { letter-spacing: -0.014em; font-weight: 600; }
h1 { letter-spacing: -0.021em; }
```

Tightening headings is most of what separates "premium software" from "default web page" typographically. Inter is drawn slightly wide for small-size legibility; at heading sizes that width reads as loose, so it is pulled back. The effect is subtle and cumulative.

Font features

```
body { font-feature-settings: "cv02", "cv03", "cv04", "cv11"; }
```

Four Inter character variants, chosen for a data-dense interface:

FEATURE	EFFECT	WHY IT MATTERS HERE
cv02	Open-tailed g	Distinguishes g from q at 11–13 px
cv03	Open l with tail	Separates l from 1 and I — critical in reference codes like FH-2026-04498
cv04	Open i	Same family of confusion
cv11	Single-storey a	Cleaner at small sizes; closer to SF Pro's texture

Tabular figures — the most important typographic rule in the product

```
.tabular,
th[data-numeric],
td[data-numeric] {
  font-variant-numeric: tabular-nums;
  font-feature-settings: "tnum";
}
```

Applied to **every number a user might compare**: money, dates, counts, references, percentages, durations.

Without it:

```
₹1,28,400    ₹1,28,400
₹64,800      ₹64,800
₹9,120       ₹9,120
↑ proportional: ragged    ↑ tabular: aligned
```

In a 25-row invoice table, proportional figures make the amount column impossible to scan for magnitude. This single property does more for the perceived quality of a data table than any amount of border styling.

4.5 Spacing

An 8-point grid, expressed through Tailwind's default 4 px step. Permitted values in this codebase:

CLASS	PX	USED FOR
gap-1 / p-1	4	Icon-to-label inside a pill
gap-1.5	6	Tight inline groups
gap-2 / p-2	8	Button internals, chip padding
gap-2.5	10	Menu item padding
gap-3 / p-3	12	Card internals, list rows
gap-3.5	14	Timeline gutters
gap-4 / p-4	16	Grid gaps, compact card padding
p-5	20	Standard card padding
gap-6	24	Section gaps
p-6 / py-6	24	Page padding (mobile)
gap-8 / mt-8	32	Major section separation
py-8	32	Page padding (desktop)

The page container:

```
px-5 py-6 sm:px-7 sm:py-8 max-w-[1600px] mx-auto
```

The 1600 px cap exists because a 12-column data table on a 3440 px ultrawide becomes unreadable — the eye cannot track a row across that distance. Capping is a legibility decision, not an aesthetic one.

4.6 Radius

TOKEN	PX	APPLIED TO
radius-xs	4	Focus rings, chart bar tops, micro-chips
radius-sm	6	Small buttons, menu items, inline code
radius-DEFAULT	8	—
radius-md	10	House radius. Cards, inputs, buttons, panels
radius-lg	12	Dialogs, the command palette
radius-xl	16	Reserved

10 px is the deliberate house value. 4 px reads as utilitarian and dated; 16 px reads as consumer-friendly and soft. 10 px is the point where a card looks considered without looking playful — and it matches the radius language of the macOS/iOS design era the brief pointed at.

4.7 Elevation

Three shadows, all tinted with brand ink rather than black.

```
--shadow-overlay: 0 12px 32px -8px rgb(3 23 40 / 0.18), 0 2px 8px -2px rgb(3 23 40 / 0.08);
--shadow-popover: 0 8px 24px -6px rgb(3 23 40 / 0.14), 0 1px 4px rgb(3 23 40 / 0.06);
--shadow-raise: 0 1px 2px rgb(3 23 40 / 0.06);
```

SHADOW	APPLIED TO	NEVER APPLIED TO
overlay	Dialogs, drawers, command palette	Anything in normal flow
popover	Dropdown menus, comboboxes, tooltips, toasts	Cards
raise	Reserved for drag states	Currently unused

Each is a two-layer shadow — a wide soft ambient plus a tight contact shadow. Single-layer shadows look artificial because real light produces both.

Everything else uses a 1 px grey-200 border. Rationale in ADR-16.

4.8 Motion

One easing curve for the entire application:

```
--ease-out-quiet: cubic-bezier(0.16, 1, 0.3, 1);
```

A strong ease-out: fast departure, gentle settle. No overshoot, no bounce, no spring.

Duration table

INTERACTION	IN	OUT	REASONING
Colour / hover transitions	150 ms	150 ms	Below ~100 ms feels instantaneous and loses the sense of response; above ~200 ms feels sluggish
Backdrop fade	150 ms	120 ms	Exits are faster than entrances — leaving should never feel like waiting
Dialog pop	160 ms	120 ms	Scale 0.97 → 1, not 0.9 → 1. A large scale delta reads as theatrical
Menu / popover	140 ms	110 ms	Fastest, because menus are opened repeatedly
Drawer slide	200 ms	160 ms	Longest, because the distance travelled is longest
Toast	200 ms	—	Enters from below-right with a slight scale

All eight keyframe sets are driven by Radix's [data-state] attribute, so no animation library is needed:

```
.motion-menu[data-state="open"] { animation: fidato-menu-in 140ms var(--ease-out-quiet); }
.motion-menu[data-state="closed"] { animation: fidato-menu-out 110ms var(--ease-out-quiet); }
```

Reduced motion

```
@media (prefers-reduced-motion: reduce) {
  *, *::before, *::after {
    animation-duration: 0.01ms !important;
    animation-iteration-count: 1 !important;
    transition-duration: 0.01ms !important;
  }
}
```

Applied globally rather than per-component, so a component added later cannot forget it.

4.9 The logo

```
src/assets/brand/
├─ logo-full.svg  3825 × 1082 – the supplied lockup, byte-for-byte
└─ logo-mark.svg  the flame glyph alone, extracted
```

ASSET	USED IN
logo-full.svg	Expanded sidebar, print headers, design-system page
logo-mark.svg	Collapsed rail, favicon, AI assistant avatar

Rules observed, from the guide (p.10–11):

- One logo. No alternates, no product-specific variants.
- Never distorted, recoloured or redrawn.
- Clear space of one "O" from "Fidato" on all sides.
- "Hotels" is set in Malgun Gothic within the mark — which is another reason the file is used as-is rather than rebuilt.

⚠ **Unresolved brand question.** The supplied SVG draws the wordmark in #142B3A . The guide (p.13) specifies Fidato Black #031728 . These are different colours.

The rule "don't alter the logo" outranks the colour specification, so the file is used untouched and `#031728` is used for UI ink only. The discrepancy is visible if a logo sits directly beside a heading. **This needs a decision from the brand team** — either the SVG is out of date or the guide is.

4.10 Focus

One treatment, applied globally:

```
:focus-visible {
  outline: 2px solid var(--color-brand-orange);
  outline-offset: 2px;
  border-radius: var(--radius-xs);
}
```

`:focus-visible` rather than `:focus` means mouse users never see a ring, keyboard users always do. The 2 px offset keeps the ring clear of the element's own border so it stays legible on inputs, which already have one.

Defining it once at base level means no component can forget it, and no component can accidentally style it differently.

4.11 Scrollbars

```
.scrollbar-quiet::-webkit-scrollbar { width: 10px; height: 10px; }
.scrollbar-quiet::-webkit-scrollbar-thumb {
  background-color: var(--color-grey-300);
  border-radius: 999px;
  border: 3px solid transparent;
  background-clip: content-box;
}
```

The transparent border plus `background-clip: content-box` is the trick that produces a macOS-style inset thumb rather than a full-width bar. Applied to every scrollable region: main content, tables, the inventory grid, dropdown lists, the AI transcript.

Windows default scrollbars are visually heavy and would undercut the intended feel on the platform this product is actually used on.

4.12 Print

```
@media print {
  body { background: #fff; }
  .no-print { display: none !important; }
  .print-serif { font-family: var(--font-serif); }
}
```

Print is a real output of this product — tax invoices and report covers. Three rules:

1. Page background goes white (screen grey wastes toner and looks wrong on paper).
2. `.no-print` removes navigation, action buttons and side rails.
3. `.print-serif` switches document headers to Georgia — the one surviving use of the brand serif ([ADR-14](#)).

Applied on `/finance/invoices/:id` and every report via `ReportShell`.

4.13 Applying the system — worked example

A status pill, showing how the layers combine:

```
<StatusPill tone="success">Confirmed</StatusPill>
```

renders

```
<span class="inline-flex items-center gap-1.5 px-2 py-0.5 rounded-full
  text-2xs font-medium bg-success-50 text-success">
  <span class="size-1.5 rounded-full bg-success"></span>
  Confirmed
</span>
```

DECISION	TOKEN	WHY
Text size	<code>text-2xs</code> (11 px)	A pill is a label, not content. It must not compete with the row's primary text
Weight	<code>font-medium</code> (500)	Enough to hold at 11 px; 600 would shout
Radius	<code>rounded-full</code>	Fully round distinguishes a status from a rectangular chip or button
Background	<code>success-50</code>	8% tint. Legible without becoming a coloured block in a 25-row table
Text	<code>success</code>	5.8:1 on the tint — AA
Dot	6 px, solid	Carries the signal at a glance; also the only cue that survives greyscale printing

The dot is the detail worth noting. It means the status column is scannable by *position and shade* before any word is read, and it remains distinguishable when the page is printed in black and white.

VOLUME V

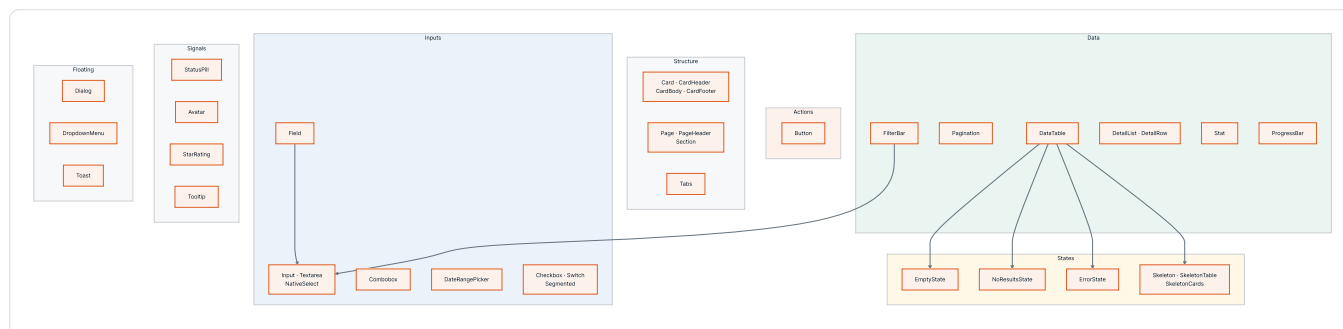
Component reference

28 primitives in `src/components/ui/`, exported through one barrel:

```
// src/components/ui/index.ts — one import surface for the whole library
export * from "./Button";      export * from "./Input";      export * from "./Card";
export * from "./StatusPill";  export * from "./Overlays";  export * from "./States";
export * from "./DataTable";  export * from "./Toast";      export * from "./Misc";
export * from "./Combobox";    export * from "./DatePicker"; export * from "./PageHeader";
export * from "./FilterBar";
```

Why a barrel. A screen imports twelve primitives. Twelve import lines is noise; one is readable. The cost — the barrel defeats per-file tree-shaking — is irrelevant here because the entire library is used somewhere in the app and it lives in its own `ui` chunk anyway.

5.1 Library map



5.2 Button

Source: `src/components/ui/Button.tsx`

Props

PROP	TYPE	DEFAULT	NOTES
<code>variant</code>	<code>primary</code> <code>secondary</code> <code>ghost</code> <code>danger</code> <code>link</code>	<code>secondary</code>	
<code>size</code>	<code>sm</code> <code>md</code> <code>lg</code> <code>icon</code>	<code>md</code>	
<code>loading</code>	<code>boolean</code>	<code>false</code>	Replaces the leading icon with a spinner and disables the button
<code>asChild</code>	<code>boolean</code>	<code>false</code>	Renders the child element instead of a <code><button></code> — used for router links
<code>leadingIcon</code>	<code>ReactNode</code>	—	
<code>trailingIcon</code>	<code>ReactNode</code>	—	Suppressed while <code>loading</code>
...plus all native <code><button></code> attributes			

Variants and their intent

VARIANT	APPEARANCE	RULE OF USE
<code>primary</code>	Solid <code>brand-orange</code> , white text	One per view. The single most likely next action
<code>secondary</code>	White, <code>grey-300</code> hairline border	The workhorse. Everything that is not the primary action
<code>ghost</code>	Transparent, grey text	Tertiary — Cancel in a dialog, Back in a wizard
<code>danger</code>	Solid <code>brand-red</code>	Destructive confirmation only. Never the trigger that opens the dialog
<code>link</code>	Orange text, underline on hover	Inline within prose

⚠️ **The `danger` rule is easy to get wrong.** The button that *opens* a cancel dialog is `secondary`; the button that *performs* the cancellation inside the dialog is `danger`. A red button in a table row invites the mis-click it is trying to protect against.

Size table

SIZE	HEIGHT	PADDING	TEXT	RADIUS
sm		32 px px-3	text-xs	rounded-sm
md		36 px px-3.5	text-base	rounded-md
lg		44 px px-5	text-md	rounded-md
icon		36 × 36 px —	—	rounded-md

The `asChild` implementation — and the bug it fixed

Radix's `Slot` accepts **exactly one child**. The naïve implementation passes three:

```
// X WRONG - this crashed the entire application (defect D-01)
<Slot>
  {leadingIcon}
  {children}
  {trailingIcon}
</Slot>
```

The correct approach folds the icons *inside* the child element:

```
// src/components/ui/Button.tsx
if (asChild) {
  const child = Children.only(children) as ReactElement<{ children?: ReactNode }>;
  const merged = isValidElement(child)
    ? cloneElement(child, undefined, leading, child.props.children, !loading && trailingIcon)
    : child;

  return (
    <Slot ref={ref} className={styles} {...props}>
      {merged}
    </Slot>
  );
}
```

Note also that `disabled` is **not** forwarded in the `asChild` branch — `disabled` is not a valid attribute on an anchor, and React would warn. See [D-01](#).

Usage

```
<Button variant="primary" leadingIcon={<Plus className="size-4" />}>
  New reservation
</Button>

<Button asChild variant="secondary">
  <Link to="/reservations">All reservations</Link>
</Button>

<Button variant="danger" loading={cancel.isPending} onClick={confirm}>
  Cancel reservation
</Button>
```

5.3 DataTable

Source: `src/components/ui/DataTable.tsx` · The most-used component in the product.

The column contract

```
export interface Column<T> {
  key: string;
  header: ReactNode;
  cell: (row: T, index: number) => ReactNode;
  numeric?: boolean;           // right-align + tabular figures
  sortable?: boolean;
  width?: string;             // Tailwind width utility
  hideBelow?: "sm" | "md" | "lg" | "xl"; // responsive column dropping
  className?: string;
}
```

It owns all four states

This is the component's most important property. A screen does not write loading, error, empty and no-results markup — it passes flags:

```
<DataTable
  columns={columns}
  rows={data?.items ?? []}
  rowKey={(r) => r.id}
  loading={isLoading}
  error={error}
  onRetry={refetch}
  hasFilters={list.hasFilters}
  onClearFilters={list.clear}
  empty={<EmptyState ... />}
/>
```

Internally:

```
if (error)      return <ErrorState onRetry={onRetry} />;
if (loading)    return <SkeletonTable columns={columns.length} />;
if (!rows.length) return hasFilters
                  ? <NoResultsState onClear={onClearFilters} />
                  : empty ?? <EmptyState compact title="Nothing here yet" />;
```

Why this belongs in the component and not the screen. Thirteen list screens use it. If each wrote its own state handling, there would be thirteen chances to forget the no-results case — and forgetting it produces the single most confusing empty screen in any admin tool: a filtered table showing "no records exist" when in fact many do.

Responsive column dropping

```
const HIDE_BELOW = {
  sm: "hidden sm:table-cell",
  md: "hidden md:table-cell",
  lg: "hidden lg:table-cell",
  xl: "hidden xl:table-cell",
};
```

Why drop columns rather than scroll horizontally. A horizontally scrolling table on a tablet hides the fact that data exists off-screen. Dropping columns by declared priority means the visible table is always complete-looking and always the *most important* columns. The table still scrolls if the remaining columns overflow — but that is the fallback, not the strategy.

The convention across the product:

BREAKPOINT	TYPICALLY DROPPED
always visible	Identity column (name/reference), status, primary value
hideBelow="md"	Dates, secondary counts
hideBelow="lg"	Location, category, owner
hideBelow="xl"	Timestamps, computed ratios, tertiary metadata

Numeric columns

```
<th data-numeric={col.numeric || undefined} className={col.numeric ? "text-right" : "text-left"}>
<td data-numeric={col.numeric || undefined} className={col.numeric && "text-right tabular"}>
```

The `data-numeric` attribute is picked up by the global CSS rule in `theme.css`, so tabular figures apply even to cells whose content is rendered by a custom `cell` function that forgot the class.

Sort indicators on numeric columns render with `flex-row-reverse`, putting the chevron on the left of the label so the label itself stays flush against the right-aligned column edge.

Row interaction

```
<tr
  onClick={onRowClick ? () => onRowClick(row) : undefined}
  tabIndex={onRowClick ? 0 : undefined}
  onKeyDown={onRowClick ? (e) => { if (e.key === "Enter") onRowClick(row); } : undefined}
  className={onRowClick && "cursor-pointer hover:bg-grey-50 focus:bg-grey-50 outline-none"}
/>
```

Clickable rows are focusable and respond to Enter. A row that is clickable by mouse only is a row keyboard users cannot reach.

⚠️ **Known limitation.** A clickable `<tr>` is not announced as interactive by screen readers — there is no valid ARIA role for "clickable table row". The mitigation is that every clickable row also contains a real link or has an equivalent action reachable elsewhere. Recorded in Volume XIII §13.6 as an accepted Phase 1 limitation.

Deliberately absent

NOT IMPLEMENTED	WHY
Zebra striping	A 1 px <code>grey-100</code> divider is enough. Striping adds visual weight to no benefit and makes status tints harder to read
Column resizing	No screen has columns whose width users would want to change; it adds meaningful complexity
Row selection / bulk actions	The reservations screen advertises bulk actions in its description but Phase 1 does not implement selection. Recorded as a gap in Volume XIII §13.7
Virtualisation	Page size is 25. Virtualisation matters past ~200 rendered rows

5.4 Pagination

Source: same file.

Windowed page numbers: `1 ... 4 5 [6] 7 8 ... 20`

```
const near = (n: number) => Math.abs(n - page) <= 1;

for (let n = 1; n <= pages; n++) {
  if (n === 1 || n === pages || near(n)) push(n);
  else if (window[window.length - 1] !== "gap") push("gap");
}
```

Always shows: first page, last page, current page ± 1. Collapses everything else into a single ellipsis, never two adjacent ones.

The range readout uses `en-IN` grouping and tabular figures: `26-50 of 1,100`.

Returns `null` when `total === 0`, so an empty table is not followed by an orphaned "0-0 of 0".

5.5 StatusPill

Source: `src/components/ui/StatusPill.tsx`

Six tones, each a soft tint with readable ink — never a solid block:

tone	background	text	meaning
success	success-50	success	Confirmed, paid, active, reconciled
warning	brand-yellow-50	#8a6300	Pending, awaiting, partially paid
danger	brand-red-50	brand-red	Cancelled, overdue, failed
info	info-50	info	Checked in, running, in progress
accent	brand-orange-50	brand-orange-700	VIP, key account, "you"

TONE	BACKGROUND	TEXT	MEANING
neutral	grey-100	grey-600	Draft, inactive, categorical labels

⚠️ The warning text colour is #8a6300, not brand-yellow. #ffb600 on #fff8e6 is 1.9:1 — illegible. The darkened amber gives 6.1:1. This is the one place where a status colour is not simply its brand token, and it is deliberate.

Domain tone maps

Rather than each screen deciding what colour "checked_in" should be, the mapping is declared once per domain:

```
export const RESERVATION_TONES: Record<string, Tone> = { ... };
export const INVOICE_TONES: Record<string, Tone> = { ... };
export const HOTEL_TONES: Record<string, Tone> = { ... };
export const CUSTOMER_TONES: Record<string, Tone> = { ... };
export const COMPANY_TONES: Record<string, Tone> = { ... };
export const AUTOMATION_TONES: Record<string, Tone> = { ... };
export const RUN_TONES: Record<string, Tone> = { ... };
export const COMMISSION_TONES: Record<string, Tone> = { ... };
export const INTEGRATION_TONES: Record<string, Tone> = { ... };
```

Usage is then uniform everywhere a reservation status appears — dashboard, list, calendar legend, detail page, customer history, company history:

```
<StatusPill tone={RESERVATION_TONES[r.status] ?? "neutral"}>
  {labelFor(r.status)}
</StatusPill>
```

Why nine maps instead of one. The same word means different things in different domains. `active` on a customer is `success`; `active` on an automation workflow is also `success`; but `draft` on a reservation is `neutral` while `draft` on an invoice is also `neutral` — and if they ever need to differ, they can, without a shared map forcing them to agree.

The dot

A 6 px solid dot precedes the label unless `dot={false}`. It carries the signal at a glance and — importantly — survives greyscale printing, where the tint backgrounds all collapse to near-identical light grey.

5.6 Field, Input, Textarea, NativeSelect

Source: `src/components/ui/Input.tsx`

`Field` uses a render-prop, which is unusual enough to justify:

```
<Field label="Email" required error={errors.email?.message} hint="Used for confirmations">
  ({ id, describedBy, invalid }) => (
    <Input id={id} aria-describedby={describedBy} invalid={invalid}
      {...form.register("email")} />
  )
</Field>
```

Why a render prop rather than `<Field><Input/></Field>`? Because `Field` must generate the `id`, wire `aria-describedby` to whichever of hint/error is present, and pass down `invalid`. With children-as-elements it would have to clone and inject props — fragile, and it breaks the moment a field wraps two controls. The render prop makes the contract explicit and typed.

What `Field` guarantees:

- A unique `id`, and a `<label htmlFor>` pointing at it.
- `aria-describedby` referencing the hint, the error, or both.
- `aria-invalid` when there is an error.
- The required marker, and `aria-required`.
- Error replaces hint rather than stacking — two lines of guidance under one input is noise.

Input

PROP	EFFECT
invalid	Red border and focus ring
numeric	Applies <code>.tabular</code> — for amounts, phone numbers, GSTINs

PROP	EFFECT
<code>leadingIcon</code>	Icon inside the field, left, with padding compensation

5.7 Combobox

Source: `src/components/ui/Combobox.tsx`

A searchable single-select on a Radix Popover. Used for customer and company pickers, where a `<select>` with 180 options is unusable.

PROP	NOTES
<code>options</code>	<code>{ value, label, description?, disabled? }[]</code>
<code>footer</code>	Rendered below the list — used for "Create a new customer"
<code>emptyMessage</code>	Shown when the filter matches nothing

Filtering is a case-insensitive substring match on label and description, capped at 100 rendered results.

Accessibility

The trigger is a Popover, so Radix reports `aria-haspopup="dialog"`. Without explicit roles a screen reader would announce *a dialog containing buttons* rather than *a set of choices*. Fixed:

```
<div role="listbox" className="max-h-64 overflow-y-auto scrollbar-quiet p-1">
  {filtered.map((opt) => (
    <button role="option" aria-selected={opt.value === value} ...>
```

The search input carries an `aria-label` matching its placeholder.

5.8 States — the four-state contract

Source: `src/components/ui/States.tsx`

COMPONENT	WHEN	CONTAINS
<code>EmptyState</code>	Dataset genuinely empty	Icon, title, explanation of what <i>will</i> appear here, primary action
<code>NoResultsState</code>	Filters exclude everything	Title, "Clear filters" action
<code>ErrorState</code>	Query rejected	Title, "Try again" action
<code>Skeleton</code>	Loading	Pulsing <code>grey-100</code> block
<code>SkeletonTable</code>	Loading a table	8 rows × N columns, matching real row height
<code>SkeletonCards</code>	Loading a card grid	N cards at real card height

The rule the skeletons follow: they must occupy the *same height* as the content that replaces them. A skeleton shorter than its content produces a layout jump at the exact moment the user starts reading — the most irritating loading bug there is, and the reason `SkeletonTable` hardcodes the same `py-3` row padding as `DataTable`.

Empty-state copy convention

Every empty state answers two questions:

1. **What would be here?** — "Bookings raised by the sales team, the website or a travel agent all land here."
2. **What do I do now?** — a primary action, *if the role can perform it*.

Role-awareness matters. The companies empty state for a salesperson reads differently:

No accounts assigned to you A sales manager assigns accounts. Switch role in the top bar to see the full list.

Offering "Add a company" to someone whose problem is *assignment*, not absence, would be unhelpful.

5.9 Overlays

Source: `src/components/ui/Overlays.tsx` — Dialog, DropdownMenu, Tooltip, Popover, Tabs.

All thin wrappers over Radix, adding only Fidato styling and the `motion-*` classes.

Dialog

```
<Dialog>
  <DialogTrigger asChild><Button>Cancel booking</Button></DialogTrigger>
  <DialogContent
    title="Cancel FH-2026-04498?"
    description="The reservation is kept and marked cancelled."
    footer={<>
      <DialogClose asChild><Button variant="ghost">Keep it</Button></DialogClose>
      <DialogClose asChild><Button variant="danger" onClick={confirm}>Cancel reservation</Button></DialogClose>
    </>
  >
  ...body...
</DialogContent>
</Dialog>
```

Four sizes: `sm` 420 px, `md` 560 px, `lg` 760 px, `xl` 980 px. The body scrolls; header and footer are fixed, so the actions are always reachable in a long dialog.

⚠ **The scroll-lock trap.** Radix sets `pointer-events: none` on `<body>` while a modal is open. If a modal unmounts *while still open* — which happens when a route change is triggered from inside it — the style can be stranded and **the entire application becomes unclickable with no visible cause**. Two mitigations are in place; see [D-02](#) and §5.13.

Tooltip

`TooltipProvider` wraps the app with `delayDuration={400}`.

Tooltips in this product explain restrictions, not decorate icons:

```
<Tooltip content={cancelCheck.reason}>
  <span>
    <Button variant="secondary" disabled leadingIcon={<Ban className="size-4" />}>
      Cancel
    </Button>
  </span>
</Tooltip>
```

Note the wrapping `<span>`. A disabled button fires no pointer events, so the tooltip would never trigger — the span is what receives them. This pattern appears wherever a disabled control needs to explain itself.

5.10 Toast

Source: `src/components/ui/Toast.tsx`

An imperative API, deliberately:

```
toast.success("Reservation confirmed", "FH-2026-04498 has been created.");
toast.warning("Sent for approval", "This booking is above ₹50,000.");
toast.error("Could not save", "Nothing was changed. Try again.");
toast.info("Preview only", "Document delivery is wired up in Phase 3.");
```

Why imperative rather than declarative state. A toast is fired from a mutation callback, often several components away from where it renders. Threading toast state through props or context would touch every mutation site. A module-level store subscribed to by a single `<Toaster>` is the correct shape for this one case.

Convention: **title states what happened; body gives the specifics**. Never "Success!" — that tells the user nothing they did not already assume.

Toasts appear bottom-right, auto-dismiss after 5 s (7 s for errors, which need longer to read), and stack to a maximum of three.

5.11 Misc

Source: `src/components/ui/Misc.tsx`

Component	Purpose	Detail worth knowing
Avatar	Initials in a tinted circle	Colour is passed in, not hashed from the name — so a VIP customer can be orange while others are grey
StarRating	Property star rating	Renders filled/empty stars with an <code>aria-label</code> giving the numeric value
ProgressBar	Utilisation, occupancy, relative magnitude	Four tones; used inside table cells at 4 px height
Segmented	Small view switcher	An accessible tab-like control for 2–3 mutually exclusive views
Stat	Label / value / hint triple	The KPI unit, used ~40 times
DetailList / DetailRow	Label-value pairs	Renders a real <code><dl></code>
Checkbox / Switch	Radix-backed	Switch is for immediate effect (pause a workflow); Checkbox is for form values

Checkbox vs Switch is a real distinction, not a style choice. A Switch takes effect the moment it is flicked and needs no save. A Checkbox is part of a form and takes effect on submit. Using a Switch inside a form implies changes save immediately, which is a lie the user will discover the hard way.

5.12 PageHeader and Page

Source: `src/components/ui/PageHeader.tsx`

```
<PageHeader
  breadcrumbs={[{ label: "Reservations", to: "/reservations" }, { label: r.reference }]}
  title={r.reference}
  description={` ${r.customerName} · ${r.hotelName}, ${r.hotelCity}`}
  badge={<StatusPill tone="success">Confirmed</StatusPill>}
  actions={<><Button ...>Voucher</Button><Button variant="primary" ...>Approve</Button></>}
>
  { /* optional filter row */ }
</PageHeader>
```

`Page` wraps content in `px-5 py-6 sm:px-7 sm:py-8 max-w-[1600px] mx-auto`. Every screen uses both, which is what makes 38 routes feel like one product rather than 38 pages.

The `actions` slot is right-aligned and wraps on narrow screens rather than overflowing.

5.13 Accessibility contract

What every component in this library guarantees:

Guarantee	Mechanism
Keyboard reachable	Native elements throughout; Radix for anything custom
Visible focus	Global <code>:focus-visible</code> in <code>theme.css</code> — cannot be forgotten
Labelled	<code>Field</code> generates label/ <code>id</code> ; icon-only buttons carry <code>aria-label</code>
Described	<code>aria-describedby</code> wired to hint and error text
State announced	<code>aria-invalid</code> , <code>aria-selected</code> , <code>aria-current</code> , <code>aria-expanded</code>
Focus trapped in overlays	Radix <code>Dialog</code> / <code>DropdownMenu</code>
Escape closes	Radix default
Reduced motion respected	Global media query

The two fixes made during the accessibility pass

Icon-only-at-narrow-widths buttons. The global search and the role switcher both hide their text label below the `sm` breakpoint, leaving an unnamed button:

```
<button
  aria-label={`Viewing as ${ROLE_LABELS[role]} (${user.name}). Change role`}
  ...
```

Combobox listbox semantics — §5.7.

Known limitations, stated honestly

Limitation	Impact	Status
Clickable <code><tr></code> has no interactive role	Screen readers do not announce rows as clickable	Accepted; every such row has an alternative path
Charts are not described	Recharts SVG has no text alternative	Open — the underlying table is present on every report screen, which mitigates it
No skip-to-content link	Keyboard users tab through the sidebar on every navigation	Open — should be added in Phase 2

VOLUME VI

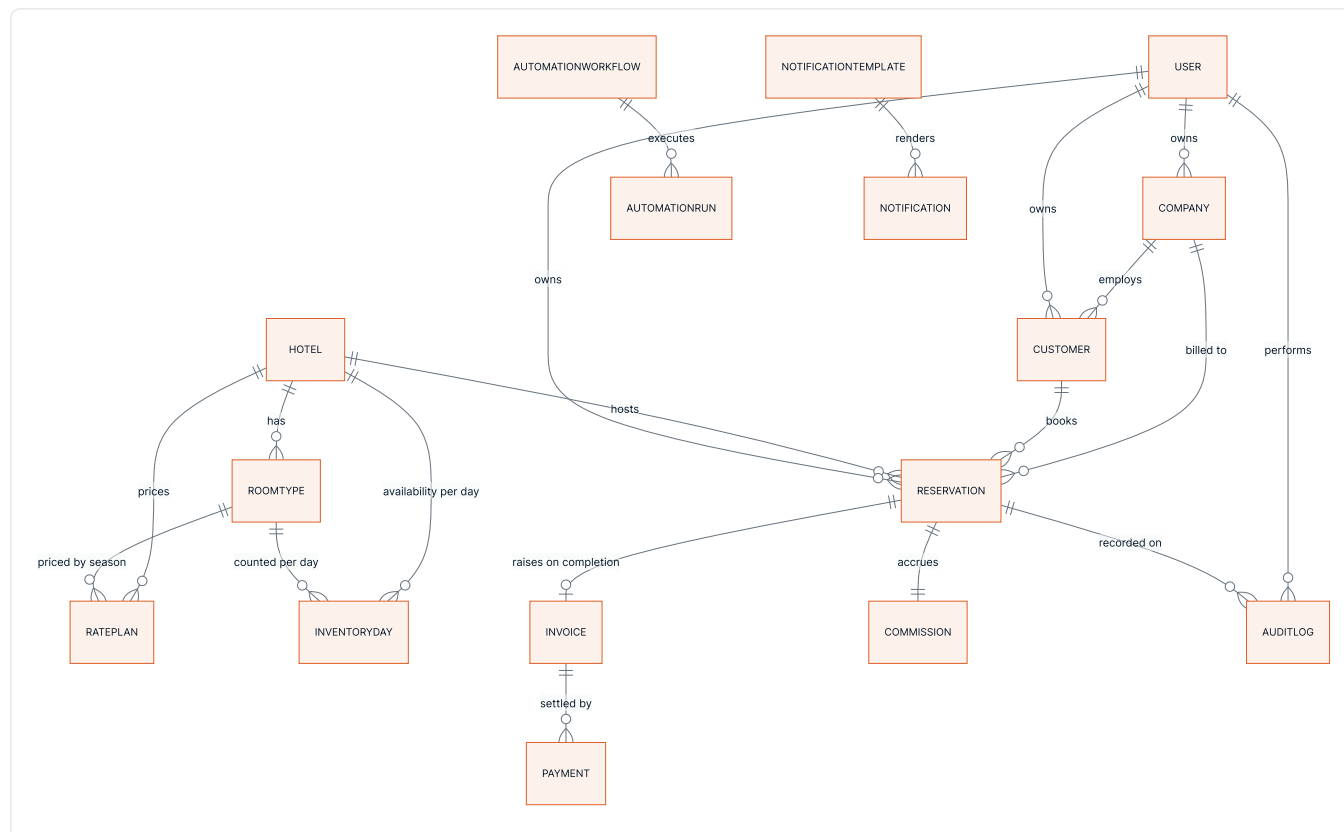
Data model

Source: `src/data/types.ts` — 46 exported types, 18 collections.

Every shape here is what Phase 2 will store in Firestore. The header of the file states the governing constraint:

```
/* Shaped to match the Firestore collections they will become in Phase 2.
Denormalised display fields (e.g. hotelName on a reservation) are
intentional - Firestore has no joins, so the read model carries what
the list screens need. */
```

6.1 Entity relationship overview



The two foreign keys that drive everything

FIELD	PRESENT ON	PURPOSE
ownerId	Customer , Company , Reservation	Row-level scoping for salespeople. Also drives commission attribution
hotelId	Reservation , RoomType , RatePlan , InventoryDay , Invoice , Commission , User	Row-level scoping for hotel managers

`scopeRecords()` in `src/lib/permissions.ts` filters on exactly these two fields and nothing else — which is why the generic signature works across every collection:

```
export function scopeRecords<T extends { ownerId?: string; hotelId?: string }>(
  ctx: ScopeContext, records: T[],
): T[]
```

6.2 Shared conventions

```
export type IsoDate      = string;    // "2026-07-28"      - date only
export type IsoDateTime = string;    // ISO-8601 timestamp - becomes Firestore Timestamp

export interface Auditable {
  createdAt: IsoDateTime;
  createdBy: string;      // user id
  updatedAt: IsoDateTime;
  updatedBy: string;      // user id
}
```

Fifteen of the eighteen collections extend `Auditable`. The three that do not — `InventoryDay`, `AuditLog`, `AppNotification` — are either derived or immutable by nature.

Why string dates rather than `Date` objects. Three reasons:

1. They are what Firestore date-only fields will hold, and what JSON carries.
2. String comparison on `yyyy-MM-dd` is lexicographically correct, so range filters are plain `>=` / `<` with no parsing (`r.checkIn >= monthStart && r.checkIn < nextMonthStart`).
3. `Date` objects in React state are a common source of re-render bugs, because two `Date`s representing the same instant are not `===`.

⚠ **The trap this creates:** timezone. `new Date("2026-07-28")` parses as UTC midnight; in IST that is 05:30 on the 28th, which is fine, but in a negative-offset timezone it would be the 27th. All formatting goes through `src/lib/format.ts`, and the app is single-timezone (Asia/Kolkata) by declaration in `orgSettings`. 🐛 Phase 2 must revisit this if the platform ever serves multiple timezones.

6.3 hotels

The anchor collection. All 32 documents are **real data** extracted from the fact-sheet PDFs (see [ADR-09](#) and Volume VII).

FIELD	TYPE	NOTES
<code>id</code>	<code>string</code>	<code>htl-{city}-{slug}</code> , e.g. <code>htl-goa-turtle-beach-resort</code>
<code>name</code>	<code>string</code>	Full property name
<code>shortName</code>	<code>string</code>	For tables and chips, where the full name would wrap
<code>city</code> / <code>state</code> / <code>address</code>	<code>string</code>	From the fact sheet
<code>category</code>	<code>HotelCategory</code>	<code>business</code> · <code>resort</code> · <code>heritage</code> · <code>beach</code> · <code>hill_station</code> · <code>banquet</code>
<code>status</code>	<code>HotelStatus</code>	<code>active</code> · <code>onboarding</code> · <code>paused</code>
<code>starRating</code>	<code>number</code>	3–5
<code>totalRooms</code>	<code>number</code>	Real. Range across the portfolio: 17 → 236
<code>description</code>	<code>string</code>	Marketing copy from the fact sheet
<code>roomMix</code>	<code>string[]</code>	Free text, e.g. <code>"Deluxe Room - 115 Rooms"</code>
<code>features</code> / <code>facilities</code> / <code>amenities</code> / <code>thingsToDo</code>	<code>string[]</code>	From the fact sheet
<code>distances</code>	<code>HotelDistance[]</code>	<code>{ label, km }</code> — real landmarks and distances
<code>contacts</code>	<code>HotelContact[]</code>	<code>{ name, designation, email, phone }</code>
<code>managerId</code>	<code>string?</code>	Links to the pinned hotel-manager user
<code>commissionPercent</code>	<code>number</code>	8–18. What Fidato earns
<code>onboardedAt</code>	<code>IsoDate</code>	

Why `roomMix` is free text and not structured. The fact sheets express it inconsistently — some as `"Suite Room - 5 Rooms"`, some as `"5 x Suite"`. Normalising it would have meant inventing structure the source does not have. The *structured* room data lives in `roomTypes`; `roomMix` is preserved verbatim as the property's own description of itself, and is displayed as-is on the property detail page.

Why `shortName` exists. *"Bngv THE Grandeur Hotel & Banquets"* does not fit a table column. Deriving a short name at render time would produce inconsistent truncation across screens; storing it makes the choice once.

6.4 roomTypes and ratePlans

```
export interface RoomType extends Auditable {
  id: string;
  hotelId: string;  hotelName: string;      // denormalised
  name: string;     code: string;          // e.g. "Deluxe Room", "DR"
  description: string;
  totalRooms: number;
  maxOccupancy: number;
  baseRate: number;
  extraAdultRate: number;
  amenities: string[];
  sizeSqft: number;
}

export type MealPlan = "EP" | "CP" | "MAP" | "AP";

export interface RatePlan extends Auditable {
  id: string;
  hotelId: string;      hotelName: string;
  roomId: string;      roomTypeName: string;
  name: string;         code: string;       // "DR-AP"
  mealPlan: MealPlan;
  rate: number;
  validFrom: IsoDate;  validTo: IsoDate;    // the season window
  minNights: number;
  cancellationPolicy: string;
  isActive: boolean;
}
```

Meal plans

CODE	MEANING	INCLUDED
EP	European Plan	Room only
CP	Continental Plan	Room + breakfast
MAP	Modified American Plan	Room + breakfast + one meal
AP	American Plan	Room + all meals

These are the standard Indian hospitality codes, used unchanged because that is what the industry — and the partner properties — already use. Inventing friendlier labels would make the system harder to reconcile against a property's own paperwork. The UI shows both: **Full Board (AP)**.

The rate plan cardinality

```
hotel → room types (2-5) → rate plans (meal plan × season)
```

A property with 3 room types × 2 meal plans × 2 seasons = 12 rate plans. Across 32 properties this yields ~370 rate plans.

Why a rate plan is per-room-type rather than per-hotel. Because a Deluxe and a Suite do not move in lockstep — a monsoon discount might be 20% on Deluxe and 10% on Suite. Modelling rates at hotel level would make the common case impossible to express.

6.5 inventory — a derived collection

```
export interface InventoryDay {
  id: string;           // inv-{roomId}-{date}
  hotelId: string;
  roomId: string;
  date: IsoDate;
  totalRooms: number;
  booked: number;
  blocked: number;
  available: number;    // totalRooms - booked - blocked
  rate: number;
}
```

⚠ **This is the one collection not held in the store.** It is generated on demand by `buildInventory(hotelId, days)` because materialising it would mean 32 hotels × ~4 room types × 60 days ≈ **7,700 documents** for data that is entirely derived.

```
// src/data/seed/index.ts
export function buildInventory(hotelId: string, days = 60) {
  const local = createRandom(
    hotelId.split("").reduce((acc, ch) => acc + ch.charCodeAt(0), 0),
  );
  ...
  const pressure = dow === 5 || dow === 6 ? 0.82 : 0.55; // weekends run hotter
  const booked = Math.min(rt.totalRooms,
    Math.round(rt.totalRooms * pressure * (0.5 + local.next())));
  const blocked = local.bool(0.08) ? local.int(1, 2) : 0;
  ...
}
```

The **per-hotel seed derived from the hotel id** is what makes this work. The same property always produces the same inventory, so navigating away and back shows identical numbers — but different properties differ. Without it, either every property would look identical or every visit would show different data.

🔗 **Phase 2 replaces this entirely** with a real `inventory` collection fed by the channel manager. The interface — `hotelsRepo.inventory(hotelId, days)` — does not change.

6.6 companies

FIELD	TYPE	NOTES
id	string	cmp-NNN
name / legalName	string	Trading name and registered name
tier	CompanyTier	key_account · corporate · sme · travel_agent
status	CompanyStatus	active · prospect · dormant
industry	string	
gstIn	string	Indian tax registration — appears on invoices
city / state / address / website / phone / email	string	
ownerId / ownerName	string	Assigned salesperson. Drives scoping
creditLimit / creditUsed	number	Utilisation bar on the list and detail screens
paymentTermDays	number	15 / 30 / 45 / 60 — sets invoice due dates
negotiatedDiscountPercent	number	Applied automatically by the quote engine
contractStart / contractEnd	ISODate?	
totalReservations / totalRevenue	number	Rolled up onto the document, Firestore-style
lastActivityAt	ISODateTime	
notes	string	

Rolled-up counters — the Firestore pattern

`totalReservations` and `totalRevenue` are stored on the company document rather than counted from reservations.

Why. Firestore charges per document read. Counting a key account's bookings by reading every reservation would be hundreds of reads to render one table cell — on a list screen showing 25 companies, that is thousands of reads per page view.

The cost: these counters can drift if a write path forgets to update them. In Phase 2 they should be maintained by a Cloud Function on reservation write, not by client code. 🔗 Volume XIV §14.5.

6.7 customers

FIELD	TYPE	NOTES
id	string	cus-NNNN
firstName / lastName / fullName	string	fullName stored, not derived — it is the search and sort field
email	string	Unique across the platform (BR-06)

FIELD	TYPE	NOTES
phone	string	Unique across the platform (BR-06)
status	CustomerStatus	active · lead · inactive
source	CustomerSource	direct · referral · website · ota · corporate · walk_in · campaign
companyId / companyName	string?	Absent for individual guests
designation	string?	Their role at the company
city / state	string	
ownerId / ownerName	string	Scoping
preferences	string[]	Surfaced in the reservation wizard and sent to the property
vip	boolean	Property is notified before arrival
totalReservations / totalRevenue	number	Rolled up
lastStayAt	IsoDateTime?	Absent until a stay completes
lastActivityAt	IsoDateTime	
notes	string	Internal only, never shown to the guest

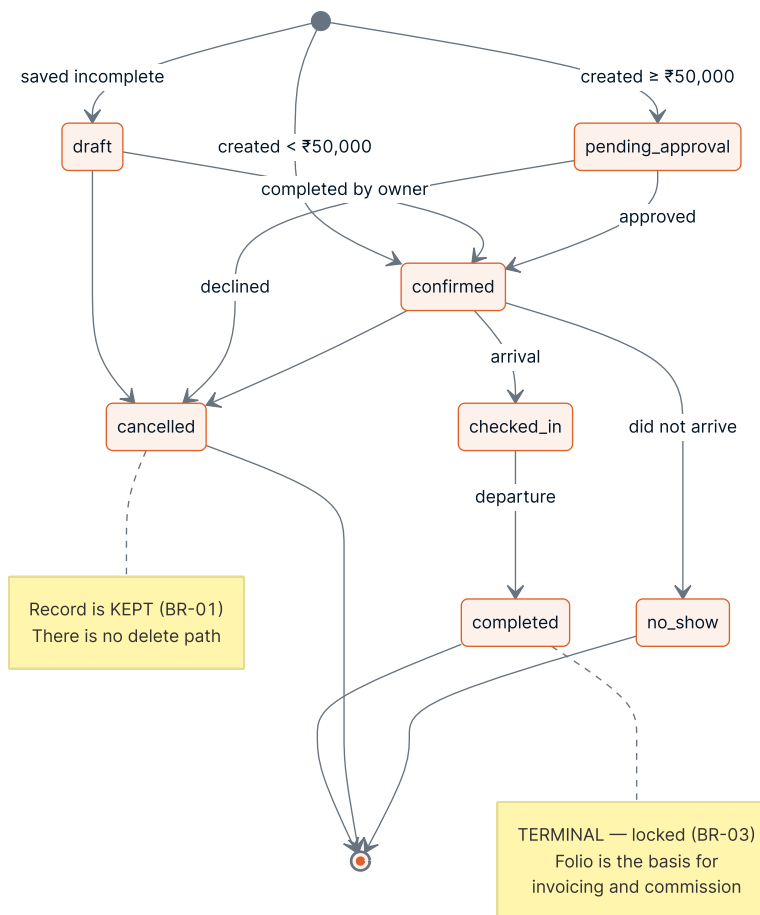
Why `fullName` is stored rather than computed. It is what gets searched, sorted and displayed. Computing it at query time would mean `matchesSearch` could not scan it as a plain field — and in Firestore, you cannot index a computed value.

6.8 reservations — the central collection

```
export type ReservationStatus =
  | "draft" | "pending_approval" | "confirmed"
  | "checked_in" | "completed" | "cancelled" | "no_show";

export type BookingChannel =
  | "direct_sales" | "corporate" | "travel_agent"
  | "website" | "phone" | "walk_in";
```

The status lifecycle



Encoded in `src/lib/rules.ts`:

```
const TERMINAL_STATUSES: ReservationStatus[] = ["completed", "cancelled", "no_show"];

export function nextStatuses(current: ReservationStatus): ReservationStatus[] { ... }
```

Field groups

Identity

FIELD	NOTES
<code>id</code>	<code>res-NNNN</code>
<code>reference</code>	<code>FH-2026-04821</code> — human-readable, quoted to guests
<code>status</code> / <code>channel</code>	

Denormalised parties — these five fields are why the reservations list renders with zero joins:

`customerId` + `customerName` · `companyId?` + `companyName?` · `hotelId` + `hotelName` + `hotelCity` · `ownerId` + `ownerName`

Stay

`checkIn` · `checkOut` · `nights` · `rooms: ReservationRoom[]` · `guests: ReservationGuest[]` · `totalRooms` · `totalAdults` · `totalChildren`

Money — every component of the quote is stored, not just the total:

`roomCharges` · `extrasCharges` · `discountAmount` · `taxAmount` · `totalAmount`

Why store every component? Because the folio must be reconstructible. If only `totalAmount` were stored, then when a company's negotiated discount changes next quarter, the historical folio would become unexplainable — you could no longer show *why* that booking cost what it cost. Storing the breakdown makes every past reservation self-describing.

Approval and cancellation

`requiresApproval` · `approvedBy?` · `approvedAt?` · `approvalNote?` `cancelledAt?` · `cancelledBy?` · `cancellationReason?`

Notes — two, with a real distinction:

FIELD	VISIBILITY
<code>specialRequests</code>	Sent to the property with the booking
<code>internalNotes</code>	Never leaves the platform

This distinction is labelled on-screen at every point of entry and display. Conflating them is how a note like *"client is difficult, watch the discount"* ends up in a hotel's inbox.

ReservationRoom — the line item

```
export interface ReservationRoom {
  roomTypeId: string;  roomTypeName: string;
  ratePlanId: string;  ratePlanName: string;
  mealPlan: MealPlan;
  quantity: number;
  ratePerNight: number; // frozen at time of booking
  adults: number;
  children: number;
}
```

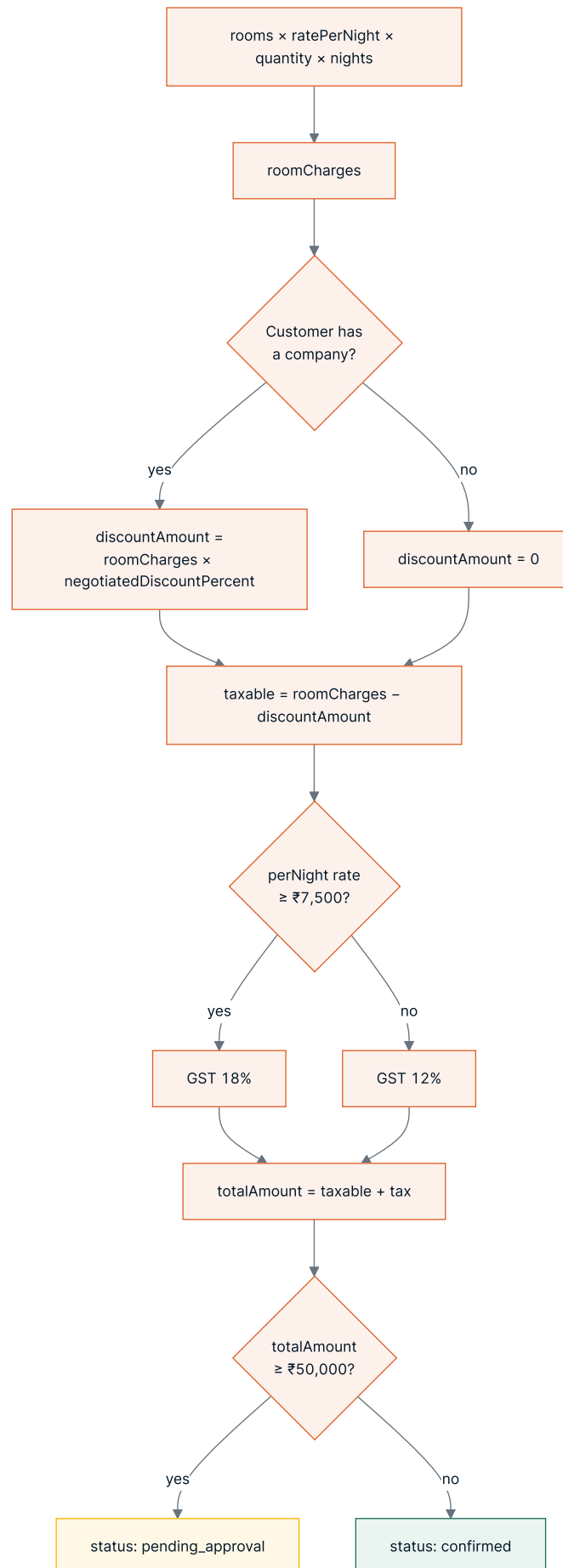
⚠ **`ratePerNight` is a snapshot, not a reference.** When the revenue team changes a rate plan, existing reservations keep the rate they were quoted. This is stated in the rate-edit dialog:

Changing a rate affects new bookings only. Existing reservations keep the rate they were quoted, and the change is written to the audit trail.

6.9 The quote engine

`reservationsRepo.quote()` is the single pricing function. The wizard calls it live on every change; `create()` calls the same function so the confirmed price can never differ from the quoted one.

```
quote: (rooms: Reservation["rooms"], nights: number, companyId?: string) => {
  const roomCharges = rooms.reduce((s, r) => s + r.ratePerNight * r.quantity * nights, 0);
  const company = companyId ? db.companies.find((c) => c.id === companyId) : undefined;
  const discountPercent = company?.negotiatedDiscountPercent ?? 0;
  const discountAmount = Math.round((roomCharges * discountPercent) / 100);
  const totalRooms = rooms.reduce((s, r) => s + r.quantity, 0);
  const perNight = totalRooms > 0 && nights > 0 ? roomCharges / totalRooms / nights : 0;
  const taxable = roomCharges - discountAmount;
  const taxRate = perNight >= 7500 ? 0.18 : 0.12;
  const taxAmount = Math.round(taxable * taxRate);
  const totalAmount = taxable + taxAmount;
  return { roomCharges, discountPercent, discountAmount, taxRate, taxAmount, totalAmount,
    requiresApproval: totalAmount >= APPROVAL_THRESHOLD, companyName: company?.name };
}
```



Three details that matter

The GST band is decided by the per-night rate, not the total. Indian GST on hotel accommodation is banded on the *tariff per room per night*: 12% below ₹7,500, 18% at or above. A ten-night booking of a ₹4,000 room totals ₹40,000 but is still taxed at 12%, because the band follows the nightly rate. Using the total would over-tax long stays — a real commercial error.

Discount is applied before tax. GST is charged on the amount actually payable, which is the discounted figure. Taxing before discount would overstate the liability.

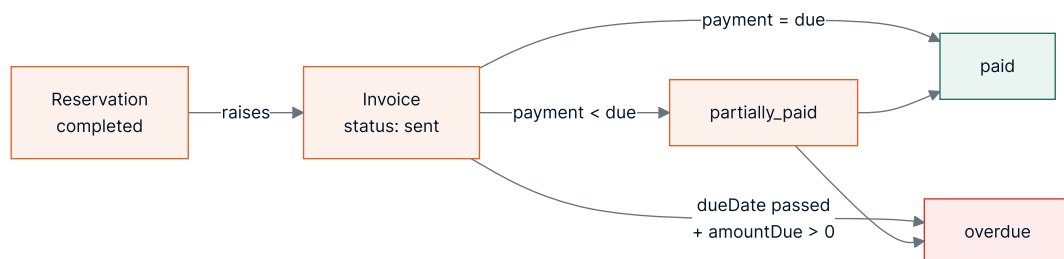
The approval threshold tests the total *including tax*, which is the figure the customer is committed to and the one shown on screen. Testing the pre-tax figure would mean a booking displaying ₹52,000 slipping through a ₹50,000 gate — indefensible when explaining the rule to a sales manager.

6.10 invoices, payments, commissions

```
export type InvoiceStatus = "draft" | "sent" | "partially_paid" | "paid" | "overdue" | "void";
```

An invoice carries frozen `lines: InvoiceLine[]` plus `subtotal`, `taxAmount`, `totalAmount`, `amountPaid`, `amountDue`.

Why `amountDue` is stored rather than computed as `total - paid`. It is the field lists filter and sort on. In Firestore you cannot query on a computed value, so the stored field is the only way to ask "show me everything with money outstanding" without reading every document.



Status is recalculated on payment:

```
// financeRepo.recordPayment (abridged)
invoice.amountPaid += amount;
invoice.amountDue = invoice.totalAmount - invoice.amountPaid;
invoice.status = invoice.amountDue <= 0 ? "paid" : "partially_paid";
```

Commission

```
export interface Commission extends Auditable {
  id: string;
  reservationId: string; reservationReference: string;
  hotelId: string;      hotelName: string;
  ownerId: string;      ownerName: string;
  bookingValue: number;
  percent: number;
  amount: number;
  status: CommissionStatus; // accrued → approved → paid
  periodMonth: string;      // "2026-07"
}
```

percent is copied from the hotel at accrual time, not referenced. If a property renegotiates its commission, historical accruals are unaffected — the same snapshot principle as `ratePerNight`.

6.11 users

```
export interface User extends Auditable {
  id: string;
  name: string; email: string; phone: string;
  role: Role;
  hotelId?: string; hotelName?: string; // hotel managers are pinned to one property
  department: string;
  isActive: boolean;
  lastSeenAt: IsoDateTime;
  avatarColor: string;
}
```

`hotelId` on a user is what makes hotel-manager scoping work: `useScope()` reads it and passes it into `ScopeContext`, and `scopeRecords()` filters every collection by it.

6.12 auditLogs — append-only

```
export interface AuditLog {
  id: string;
  entityType: "reservation" | "customer" | "company" | "hotel" | "invoice" | "user" | "rate";
  entityId: string;
  entityLabel: string; // denormalised, e.g. "FH-2026-04498"
  action: AuditAction;
  summary: string;
  detail?: string;
  actorId: string; actorName: string; actorRole: Role;
  at: IsoDateTime;
}
```

Ten actions: `created` · `updated` · `status_changed` · `cancelled` · `approved` · `merged` · `exported` · `viewed` · `note_added` · `email_sent`.

Written by one helper, so no write path can forget it:

```
function recordAudit(entry: { ... }) {
  db.auditLogs.unshift({ id: nextId("aud", 5), ...entry, at: nowIso() });
}
```

Not Auditable. An audit entry that could be updated would defeat its own purpose. It has `at` and an actor, and nothing else. `unshift` keeps newest-first, which is the order every screen wants.

`entityLabel` is **denormalised** so the audit screen can render "FH-2026-04498" without reading the reservation — which matters when the log holds 2,600 entries spanning seven entity types.

6.13 Notifications and automation

```
export interface AppNotification {
  id: string; title: string; body: string;
  category: NotificationCategory; // reservation | approval | payment | system | customer | automation
  channel: NotificationChannel; // in_app | email | whatsapp | sms | push
  isRead: boolean;
  actorName?: string;
  link?: string; // deep link to the record
  at: IsoDateTime;
}
```

```
export interface AutomationWorkflow extends Auditable {
  id: string; name: string; description: string;
  trigger: string; triggerDetail: string;
  conditions: string[];
  steps: AutomationStep[];
  status: AutomationStatus; // active | paused | draft
  runsLast30Days: number;
  successRate: number;
  averageDurationMs: number;
  lastRunAt?: IsoDateTime;
}

export interface AutomationStep {
  id: string; order: number;
  kind: AutomationStepKind; // generate_pdf | send_email | send_whatsapp | update_record
  // notify_user | webhook | wait | condition
  label: string; detail: string;
  n8nNode?: string; // ← the Phase 3 contract
}
```

 **n8nNode** is the seam for Phase 3. Each step declares which n8n node will implement it. The workflow detail screen displays it, so the definition in this system *is* the specification n8n will be built against.

6.14 Query types

```
export interface ListQuery {
  search?: string;
  filters?: Record<string, string>;
  sortBy?: string;
  sortDir?: "asc" | "desc";
  page?: number;
  pageSize?: number;
}

export interface ListResult<T> {
  items: T[];
  total: number;
  page: number;
  pageSize: number;
}
```

Deliberately minimal, and deliberately Firestore-shaped: `filters` is equality-only, because that is what Firestore `where()` does cheaply. There is no `OR`, no `LIKE`, no computed predicate. See [ADR-06](#).

 `page` is offset-based. Firestore paginates by cursor. This is the **one honest deviation** in the data layer, and Volume XIV §14.3 covers the migration.

6.15 Collection sizes

COLLECTION	DOCUMENTS	GENERATED HOW
hotels	32	Extracted from fact-sheet PDFs
roomTypes	~120	Derived from each hotel's real room mix
ratePlans	~370	room type × meal plan × season
inventory	derived	<code>buildInventory()</code> on demand
companies	40	Seeded
customers	180	Seeded
reservations	1,100	Seeded across −330...+150 days
invoices	~450	For completed and in-house stays
payments	~380	Against invoices
commissions	~800	One per non-cancelled reservation
users	24	Across all 8 roles
auditLogs	~2,600	Generated from reservation lifecycle events

COLLECTION	DOCUMENTS	GENERATED HOW
notifications	40	Across 5 channels
notificationTemplates	12	Per event
automationWorkflows	12	Hand-written definitions
automationRuns	~340	Simulated history
integrations	14	Hand-written
orgSettings	1	Singleton

Volume VII covers how each is generated.

VOLUME VII

The seed engine

Source: `src/data/seed/` — four files, ~1,600 lines.

This volume documents throwaway code. `src/data/seed/` is deleted in Phase 2. It is documented in full anyway, because for the whole of Phase 1 it *is* the database, and every number anyone reviews comes out of it. If you cannot explain where a figure came from, you cannot defend it in a review.

```
src/data/seed/
├ random.ts      73 lines  - deterministic PRNG
├ names.ts       ~180 lines - name pools, industries, preferences, copy
├ hotels.data.ts ~900 lines - 32 real properties, extracted from PDFs
└ index.ts       ~1,180 lines - the generator
```

7.1 Determinism

```
// src/data/seed/index.ts
const SEED = 20260728;
const rng = createRandom(SEED);
export const TODAY = new Date(2026, 6, 28);
```

One fixed integer produces the entire world. Reload the page and every one of the 1,100 reservations has the same reference, the same guest, the same price.

Why not `Math.random()`

WITH <code>MATH.RANDOM()</code>	WITH A FIXED SEED
"The Peerless Inn booking looks wrong" — unreproducible	Anyone can open the same record
Screenshots in a document go stale immediately	This manual describes the screens you will actually see
A bug that appears in 1 run of 20 is untraceable	Deterministic, therefore debuggable
Two reviewers describe different products	Everyone reviews the same product

The generator

`mulberry32` — 8 lines, no dependency, good enough distribution for fixtures:

```
export function createRandom(seed: number) {
  let state = seed >>> 0;

  function next(): number {
    state |= 0;
    state = (state + 0x6d2b79f5) | 0;
    let t = Math.imul(state ^ (state >>> 15), 1 | state);
    t = (t + Math.imul(t ^ (t >>> 7), 61 | t)) ^ t;
    return ((t ^ (t >>> 14)) >>> 0) / 4294967296;
  }

  ...
}
```

Chosen over `seedrandom` (a dependency for 8 lines), a Linear Congruential Generator (visible patterns in low bits — clustering in weighted picks) and Mersenne Twister (600 lines of state for fixture data).

The helper API

HELPER	SIGNATURE	PURPOSE
<code>next()</code>	→ <code>number</code>	Float in [0, 1)
<code>int(min, max)</code>	→ <code>number</code>	Inclusive integer
<code>pick(items)</code>	→ <code>T</code>	One element
<code>sample(items, n)</code>	→ <code>T[]</code>	<i>n</i> distinct elements
<code>bool(p = 0.5)</code>	→ <code>boolean</code>	Weighted coin

HELPER	SIGNATURE	PURPOSE
<code>weighted(entries)</code>	<code>→ T</code>	Pick by relative weight
<code>money(min, max, step = 100)</code>	<code>→ number</code>	Rounded to a step

Two of these deserve explanation.

`weighted` is what makes the data look real rather than uniform:

```
const CHANNELS: readonly (readonly [BookingChannel, number])[] = [
  ["direct_sales", 5], ["corporate", 5], ["travel_agent", 3],
  ["website", 3], ["phone", 2], ["walk_in", 1],
];
```

A uniform pick would give each channel 16.7%, which no real hospitality business has. The weights reflect a plausible mix — direct sales and corporate dominate, walk-ins are rare. Every distribution in the seed is weighted: room counts, night counts, statuses, tiers, sources.

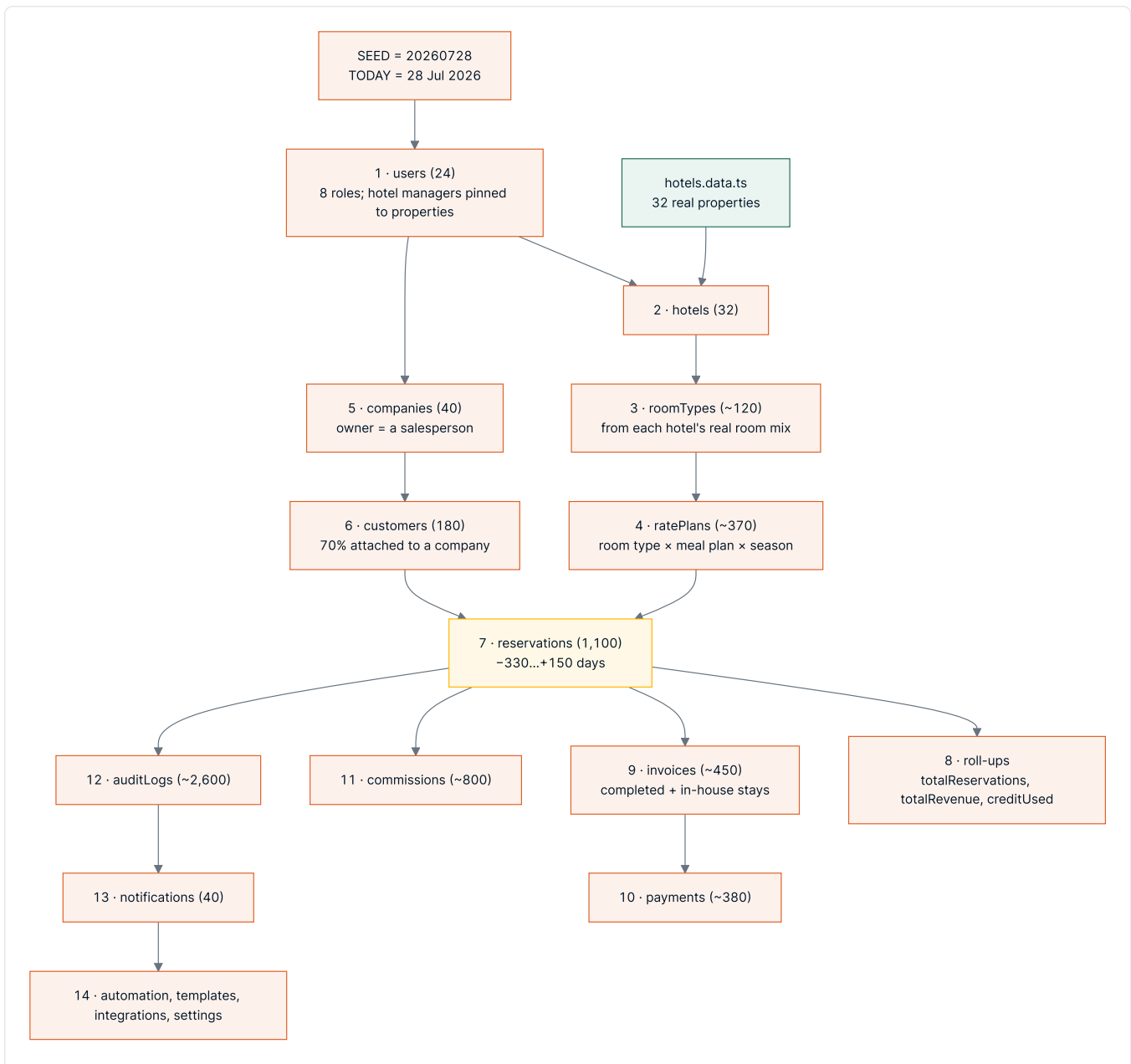
`money(min, max, step = 100)` rounds to the nearest ₹100:

```
function money(min: number, max: number, step = 100): number {
  return Math.round(int(min, max) / step) * step;
}
```

Real tariffs are ₹4,600, not ₹4,637. Unrounded money is the single fastest way to make seeded data look fake — the eye catches it immediately in a rate table.

7.2 The generation pipeline

Order matters absolutely: every stage depends on the ones before it.



⚠ **Because there is a single shared `rng` instance, order is load-bearing.** Inserting a generation step in the middle shifts every subsequent random draw and changes the entire dataset. This is a feature — it keeps generation cheap and deterministic — but it means the seed file cannot be casually reordered.

7.3 The 32 real properties

`hotels.data.ts` opens with its own provenance note:

```

/* REAL PROPERTY DATA - Fidato partner portfolio

Extracted from the 32 official fact sheets in
D:\Fidato Assets\Hotel Fact Sheets. Names, cities, room counts,
room mixes, features, amenities and landmark distances are the
property's actual published details. Commercial figures
(commission, rates) are simulated for Phase 1.

Generated - do not hand-edit. Re-run the extractor to refresh. */

```

What is real and what is not

REAL (FROM THE PDFS)	SIMULATED
Property name, short name	Commission percentage
City, state, full address	Base rates and rate plans
Star rating, category	Contact names, emails, phones
Total rooms	Onboarding date
Room mix and per-type counts	Status (active / onboarding / paused)
Description copy	
Features, facilities, amenities	
Things to do nearby	
Landmark distances in km	

This split is deliberate and stated: **operational data is real, commercial data is simulated**. Nobody should read a commission figure off this build and act on it.

The shape

```
export interface SeedHotel {
  id: string;           // htl-goaturtle-beach-resort
  name: string;         // "Turtle Beach Resort"
  shortName: string;
  city: string; state: string; address: string;
  category: string;     // resort | business | heritage | beach | hill_station | banquet
  status: string;
  starRating: number;
  totalRooms: number;
  description: string;
  roomMix: string[];    // verbatim: "Deluxe Room - 115 Rooms"
  roomTypes: SeedRoomType[]; // parsed: { name: "Deluxe Room", count: 115 }
  features: string[];
  facilities: string[];
  amenities: string[];
  thingsToDo: string[];
  distances: SeedDistance[]; // { label: "Shaniwar Wada", km: 11.4 }
}
```

Note `roomMix` and `roomTypes` . The first is the property's own words, preserved for display; the second is the parsed structure the room-type generator consumes. Keeping both means the display never has to reconstruct a sentence from parsed data, and the parser's occasional imperfection never corrupts what is shown.

What the real distribution exposed

PROPERTY	ROOMS
The Crescent, Udaipur	17
Ayati Resort & Spa, Mahabaleshwar	30
Turtle Beach Resort, Goa	111
Marigold, Pune	150
Agra Hotel Taj Pearl	236

A **14x spread**. This is exactly the kind of thing invented data never has, and it changed a product decision: ranking properties by total revenue systematically flatters the large ones, so the Property Performance report offers *revenue per available room* as an equal-status sort. That insight came from the data, not from a requirement.

7.4 Users

```
const spec: { role: Role; count: number; department: string }[] = [
  { role: "super_admin", count: 1, department: "Technology" },
  { role: "admin", count: 2, department: "Operations" },
  { role: "sales_manager", count: 2, department: "Sales" },
  { role: "salesperson", count: 8, department: "Sales" },
  { role: "hotel_manager", count: 5, department: "Property Operations" },
  { role: "finance", count: 3, department: "Finance" },
  { role: "support", count: 2, department: "Guest Support" },
  { role: "viewer", count: 1, department: "Leadership" },
];
```

24 users, and the shape is deliberate: **8 salespeople** so that row-level scoping is observable — a salesperson sees roughly an eighth of the customer base, which is visibly different from the full list. With two salespeople the scoping would be much harder to perceive.

5 hotel managers, each pinned to a different property via `hotelCursor++`, so switching to the Hotel Manager role lands on a property that actually has bookings.

Names are drawn by index arithmetic rather than random pick:

```
const first = FIRST_NAMES[n % FIRST_NAMES.length]!;
const last = LAST_NAMES[(n * 7 + 3) % LAST_NAMES.length]!;
```

The `* 7 + 3` stride avoids first and last names marching in lockstep, which would produce "Aarav Agarwal, Aditi Bose, Akash Chopra..." — visibly generated.

7.5 Reservations — the heart of the seed

1,100 documents, and the most carefully tuned part of the generator.

Date distribution

```
// Spread across 11 months back and 5 months forward. At ~1,100
// reservations that is roughly three arrivals a day across the
// portfolio, so any given day has something happening on it -
// which is what a 32-property book actually looks like.
const offset = rng.int(-330, 150);
const checkIn = addDays(TODAY, offset);
const nights = rng.weighted([[1, 4], [2, 5], [3, 4], [4, 2], [5, 1], [7, 1]]);
```

Why 1,100 and not 320. The first version generated 320. It typechecked, it built, and the dashboard looked *dead*: 0 arrivals today, 0 in house, 1 departure. 320 bookings over 480 days is 0.67 arrivals per day across 32 properties — statistically an empty day is normal, but it misrepresents the business. At 1,100 the portfolio shows ~3 arrivals and ~4 in-house on a typical day, which is what a real book of this size looks like. See [D-05](#).

Why nights are weighted toward 2–3. Business travel dominates the channel mix, and business stays are short. A uniform 1–7 would produce an average stay of 4 nights, which would be a resort's profile, not this portfolio's.

Status follows the dates

```
function statusForDates(checkIn: Date, checkOut: Date, total: number): ReservationStatus {
  const past = checkOut < TODAY;
  const inHouse = checkIn <= TODAY && checkOut >= TODAY;

  if (past) {
    return rng.weighted<ReservationStatus>([
      ["completed", 12], ["cancelled", 2], ["no_show", 1],
    ]);
  }
  if (inHouse) return "checked_in";
  if (total >= 50_000 && rng.bool(0.45)) return "pending_approval";
  return rng.weighted<ReservationStatus>([
    ["confirmed", 8], ["draft", 1], ["cancelled", 1],
  ]);
}
```

This is the function that makes the data *coherent*. A reservation whose checkout was three months ago cannot be "confirmed" — it must have resolved. One whose dates straddle today must be "checked in". Random status assignment would produce future bookings marked "completed", and every date-sensitive screen would look broken.

The past-stay mix — 12 : 2 : 1 — yields roughly 80% completed, 13% cancelled, 7% no-show, which lands the portfolio cancellation rate at a believable ~12%.

🚨 **The 45% coin on `pending_approval`** . Not every large booking is still waiting; some were approved. Without the coin, *every* future booking over ₹50,000 would sit in the approval queue and the queue would be absurd. With it, 66 sit pending — enough to review, few enough to be plausible.

Lead time, and the bug it hid

```
// Booked some weeks before arrival - but never in the future. A
// forward booking was necessarily raised on or before today.
const leadTime = subDays(checkIn, rng.int(3, 60));
const created = leadTime > TODAY ? subDays(TODAY, rng.int(0, 21)) : leadTime;
```

The first version was just `subDays(checkIn, rng.int(3, 60))` . For a check-in in October 2026, that produced a `createdAt` in August or September — **in the future**. The approvals queue then displayed *"raised by Meera Bose · in about 1 month"*, which is nonsense: a booking cannot have been created after today. See [D-06](#).

The clamp is a small piece of code carrying a real domain invariant: **creation precedes existence**.

Money

Room selection, then the shared quote engine:

```
const roomCount = rng.weighted([[1, 6], [2, 4], [3, 2], [5, 1], [8, 1]]);
const chosen = rng.sample(types, Math.min(rng.int(1, 2), types.length));
```

Most bookings are 1–2 rooms; a few are group bookings of 5 or 8. Those large ones are what push totals past ₹50,000 and populate the approval queue — the distribution is what makes the business rule observable.

Pricing then uses the *same* `quote()` logic the wizard uses, so seeded reservations and user-created ones are priced identically. Duplicating the pricing maths in the seed would let the two drift, and a folio created by the seed would eventually disagree with one created by the app.

7.6 Roll-ups

After reservations exist, aggregates are written back onto the parent documents:

```
for (const r of reservations) {
  if (r.status === "cancelled" || r.status === "draft") continue;
  const customer = customerById.get(r.customerId);
  if (customer) {
    customer.totalReservations += 1;
    customer.totalRevenue += r.totalAmount;
    if (r.status === "completed") customer.lastStayAt = ts(new Date(r.checkOut));
  }
  const company = r.companyId ? companyById.get(r.companyId) : undefined;
  if (company) {
    company.totalReservations += 1;
    company.totalRevenue += r.totalAmount;
  }
}
```

Cancelled and draft bookings are excluded from revenue roll-ups. A cancelled booking that still counted toward a customer's lifetime value would corrupt every account-value figure in the product.

This mirrors what a Firestore Cloud Function will do on reservation write in Phase 2 — the same denormalisation, computed at a different time. 🐞
Volume XIV §14.5.

7.7 Deliberately planted data

Some data exists specifically so that a feature has something to show. This is legitimate — a merge screen with no duplicates demonstrates nothing — but it should be *known*.

PLANTED	WHERE	PURPOSE
4 duplicate customer groups (8 records)	<code>/crm/merge</code>	Phone, email and name collisions so all three detection strategies are visible
~66 reservations ≥ ₹50,000 pending	<code>/reservations/approvals</code>	A queue worth reviewing
Overdue invoices	<code>/finance/invoices</code>	Exercises the overdue state and the finance dashboard

PLANTED	WHERE	PURPOSE
Unreconciled payments	<code>/finance/payments</code>	Exercises the reconciliation column
1 paused property	<code>/reservations/new</code>	The wizard disables paused properties — otherwise unprovable
3 onboarding properties	<code>/hotels</code>	The status filter has something to filter
1 automation workflow with elevated failures	<code>/automation</code>	The failure-rate styling has a subject
2 rows in the sample CSV colliding with existing customers	<code>/crm/import</code>	Import warnings are demonstrable without the reviewer preparing a file

That last one is called out in the UI itself:

Two rows in the sample collide with existing records, so you can see how warnings behave.

7.8 names.ts

Pools sized so repetition is not conspicuous:

POOL	SIZE
<code>FIRST_NAMES</code>	60 Indian given names
<code>LAST_NAMES</code>	40 surnames
<code>COMPANY_PREFIXES</code> / <code>SUFFIXES</code>	40 / 12
<code>LEGAL_SUFFIXES</code>	5 (Pvt Ltd, LLP, ...)
<code>INDUSTRIES</code>	18
<code>DESIGNATIONS</code>	14
<code>GUEST_PREFERENCES</code>	12
<code>SPECIAL_REQUESTS</code>	14
<code>CANCELLATION_REASONS</code>	10
<code>INTERNAL_NOTES</code>	10
<code>AVATAR_COLORS</code>	8

60 × 40 = 2,400 name combinations for 180 customers plus 24 users, so collisions are rare without being impossible — which is itself realistic.

The prose pools matter more than they look. `SPECIAL_REQUESTS` contains real-sounding requests — *"Anniversary stay — cake and room decoration requested"*, *"Guest arriving on a late flight, please hold the room"* — because a reservation detail page whose special-requests field says *lorem ipsum* cannot be evaluated. You cannot tell whether the field is the right length, the right prominence, or the right tone.

7.9 Cost of the seed

Lines of throwaway code	~1,600
Time to write	~1 day
Generation time on load	~40 ms
Memory	~6 MB
Deleted in Phase 2	All of it, except <code>hotels.data.ts</code>

`hotels.data.ts` survives — it becomes the seeding script that populates the real Firestore `hotels` collection. The 32 properties are genuine reference data, not fixtures.

7.10 If you need to change the seed

GOAL	CHANGE	WATCH OUT FOR
More/fewer reservations	<code>Array.from({ length: 1100 }, ...)</code>	Below ~800, "today" starts looking empty again
Shift the demo date	<code>TODAY</code>	Rate plan season windows in <code>hotels.data.ts</code> are absolute dates and will need moving
Different data entirely	<code>SEED</code>	Every planted case in §7.7 must be re-verified — a new seed may produce zero duplicates
Add a collection	Append to the end of <code>index.ts</code>	Inserting mid-file shifts every subsequent random draw and changes all existing data
Change a distribution	The <code>weighted</code> table	Check the downstream KPI it feeds; the status weights set the cancellation rate

⚠️ **The most common mistake** is changing `SEED` to "get different data" and not re-checking §7.7. The merge screen can silently become an empty state, and a reviewer then concludes the feature does not work.

VOLUME VIII

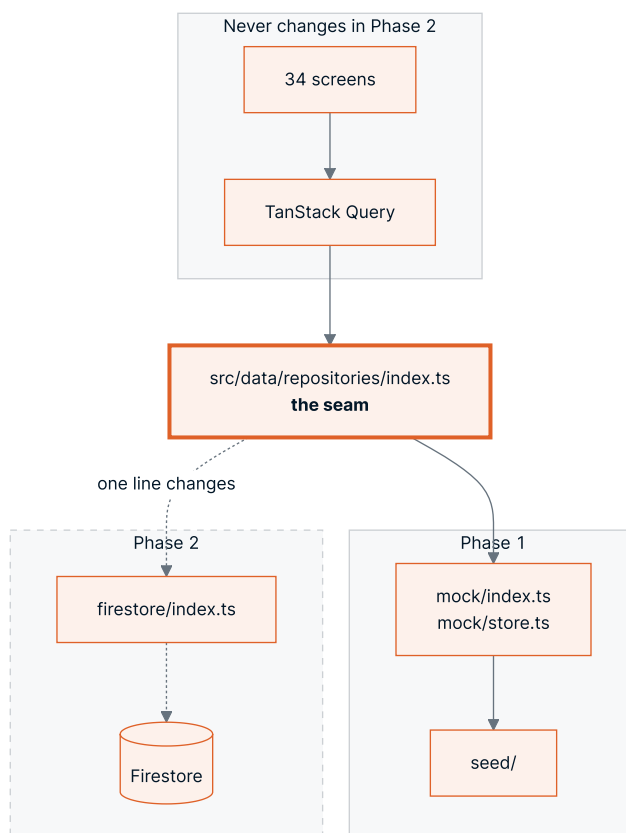
The repository layer

Source: `src/data/repositories/`

This is the most important layer in the codebase for Phase 2, because it is the **only** layer Phase 2 replaces. Everything above it — 34 screens, 28 components, all state management — continues unchanged.

```
src/data/repositories/
├─ index.ts      ← the ONLY import path any screen uses
└─ mock/
   ├─ store.ts   db · latency · the query pipeline · id generation
   └─ index.ts   11 repositories, ~1,060 lines
```

8.1 The contract



`index.ts` is a re-export today:

```
export * from "./mock";
```

In Phase 2 it becomes:

```
export * from "./firestore";
```

One line. That is the entire point of the layer, and it only holds if the discipline in §8.2 is maintained.

8.2 The Firestore constraint

Every method obeys what Firestore can actually do ([ADR-06](#)).

FIRESTORE REALITY	CONSEQUENCE HERE
No joins	Documents are denormalised. A reservation carries <code>customerName</code> , <code>companyName</code> , <code>hotelName</code> , <code>hotelCity</code> , <code>ownerName</code>
No <code>LIKE</code> / full-text	<code>matchesSearch()</code> is a client-side substring scan over named fields
Equality and range filters only, on indexed fields	<code>applyFilters()</code> does equality only
No <code>GROUP BY</code>	Reports aggregate in the client
No server-side computed predicates	<code>amountDue</code> is stored, not derived, so it can be filtered
Cursor pagination	⚠ <code>paginate()</code> is offset-based — the one honest deviation. Volume XIV §14.3
Per-document writes	Each write touches one document plus explicit denormalisation updates

⚠ **This constraint is a discipline, not a mechanism.** Nothing in the type system prevents someone adding `customersRepo.searchFullText()`. If that happens, the Phase 2 estimate in Volume XIV becomes fiction. It should be a review checkpoint.

8.3 Simulated latency

```
// src/data/repositories/mock/store.ts
const MIN_LATENCY = 120;
const MAX_LATENCY = 400;

export function latency(): Promise<void> {
  const ms = MIN_LATENCY + Math.random() * (MAX_LATENCY - MIN_LATENCY);
  return new Promise((resolve) => setTimeout(resolve, ms));
}

export async function read<T>(produce: () => T): Promise<T> {
  await latency();
  return produce();
}

export async function write<T>(produce: () => T): Promise<T> {
  await latency();
  await new Promise((resolve) => setTimeout(resolve, 80)); // writes cost more
  return produce();
}
```

Every repository method is wrapped in `read()` or `write()`. There is no synchronous path — by design, so no screen can accidentally depend on data being available on first render.

⚠ **latency() uses `Math.random()`, not the seeded PRNG.** This is intentional: the *data* must be deterministic, the *timing* must not be. Deterministic timing would mean queries always resolve in the same order, which would hide exactly the race conditions this is meant to expose.

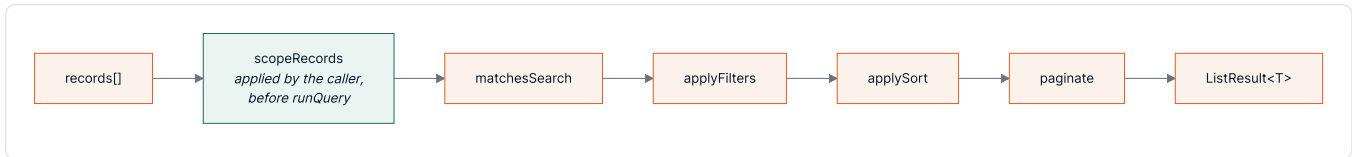
What the jitter caught

The dashboard fires four independent queries — KPIs, revenue series, day sheet, approvals. With zero latency they appeared to resolve together, and a single shared loading flag looked correct. With jitter, they visibly landed out of order, which made it obvious that each panel needed its own skeleton. That is now the pattern across the app.

8.4 The query pipeline

Every list repository funnels through one function:

```
export function runQuery<T>({
  records: T[],
  query: ListQuery | undefined,
  searchFields: (keyof T | string)[],
}): ListResult<T> {
  const q = query ?? {};
  let result = records;
  if (q.search) result = result.filter((r) => matchesSearch(r, q.search!, searchFields));
  result = applyFilters(result, q.filters);
  result = applySort(result, q.sortBy, q.sortDir);
  return paginate(result, q.page ?? 1, q.pageSize ?? 25);
}
```



Scope is applied by the caller, before `runQuery` — not inside it. This ordering is a security property, not an implementation detail:

```

list: (query?: ListQuery, ctx?: ScopeContext): Promise<ListResult<Reservation>> =>
  read() => {
    const scoped = ctx ? scopeRecords(ctx, db.reservations) : db.reservations;
    return runQuery(scoped, query, ["reference", "customerName", "hotelName", ...]);
  },

```

If scoping ran *after* search, **total** would count records the actor cannot see, leaking their existence through the pagination readout. Running it first means a salesperson searching for another rep's account gets a clean "no results" — not an error, and no evidence the record exists.

matchesSearch

```

export function matchesSearch<T>(record: T, term: string, fields: (keyof T | string)[]): boolean {
  if (!term.trim()) return true;
  const needle = term.trim().toLowerCase();
  return fields.some((field) => {
    const value = getField(record, String(field));
    return typeof value === "string" && value.toLowerCase().includes(needle);
  });
}

```

Case-insensitive substring across explicitly named fields — never across the whole document. Searching every field would match on ids, internal notes and audit text, producing results the user cannot explain.

Each repository declares its own search surface:

REPOSITORY	SEARCHABLE FIELDS
reservations	reference, customerName, companyName, hotelName, hotelCity, ownerName
customers	fullName, email, phone, city, companyName
companies	name, legalName, city, industry, email, gstin
hotels	name, shortName, city, state, address
invoices	number, reservationReference, customerName, companyName, hotelName
users	name, email, department, hotelName
auditLogs	entityLabel, summary, actorName, detail

🔗 In Phase 2 this stays client-side until the collection outgrows it, then moves to Algolia or Typesense. The interface does not change.

applyFilters

```

export function applyFilters<T>(records: T[], filters: ListQuery["filters"]): T[] {
  if (!filters) return records;
  const active = Object.entries(filters).filter(
    ([, value]) => value !== undefined && value !== "" && value !== "all",
  );
  if (!active.length) return records;
  return records.filter((record) =>
    active.every(([key, value]) => {
      const field = getField(record, key);
      if (Array.isArray(field)) return field.includes(value as never);
      return field === value;
    })
  );
}

```

Two details:

- **"all"** is treated as *no filter*, so the UI can use a single `<select>` with an "All" option rather than special-casing an empty value.
- Array fields use **includes**, matching Firestore's **array-contains** operator exactly.

applySort

Null handling is explicit, and nulls always sort last regardless of direction:

```
if (left == null && right == null) return 0;
if (left == null) return 1;
if (right == null) return -1;
```

A customer with no `lastStayAt` should not lead a descending sort by last stay. Missing data is not "smallest"; it is *absent*, and absent belongs at the end.

Numbers compare numerically, everything else via `localeCompare`.

paginate

```
export function paginate<T>(records: T[], page = 1, pageSize = 25): ListResult<T> {
  const start = (page - 1) * pageSize;
  return { items: records.slice(start, start + pageSize), total: records.length, page, pageSize };
}
```

⚠ Offset-based. Firestore is cursor-based. Volume XIV §14.3.

8.5 The eleven repositories

REPOSITORY	READS	WRITES
hotelsRepo	list all get roomTypes ratePlans inventory	update
ratePlansRepo	list	update
companiesRepo	list all get	create update
customersRepo	list all get reservations duplicates	create update merge importMany
reservationsRepo	list get pendingApprovals daySheet inRange audit quote	create setStatus
financeRepo	invoices invoice payments paymentsForInvoice	recordPayment
reportsRepo	kpis revenueSeries hotelPerformance salesPerformance occupancyByCity channelMix forecast	—
automationRepo	workflows workflow runs runsForWorkflow	setStatus
notificationsRepo	list unreadCount templates template	markRead markAllRead
adminRepo	users user allUsers auditLog integrations settings	updateUser updateSettings
searchRepo	query	—

`quote()` is the one synchronous method, because the wizard calls it on every keystroke and awaiting a promise per change would make the price flicker.

8.6 Writes and the audit trail

Every write goes through one helper, so no path can forget the audit entry:

```
function recordAudit(entry: {
  entityType: AuditLog["entityType"];
  entityId: string; entityIdLabel: string;
  action: AuditAction; summary: string; detail?: string;
  actor: { id: string; name: string; role: Role };
}) {
  db.auditLogs.unshift({
    id: nextId("aud", 5),
    entityType: entry.entityType, entityId: entry.entityId, entityIdLabel: entry.entityLabel,
    action: entry.action, summary: entry.summary, detail: entry.detail,
    actorId: entry.actor.id, actorName: entry.actor.name, actorRole: entry.actor.role,
    at: nowIso(),
  });
}
```

Every write method takes an `Actor`:

```
export interface Actor { id: string; name: string; role: Role; }
```

supplied by `useActor()` from the session store. There is no ambient "current user" inside the repository layer — identity is always passed in explicitly.

Why. A repository that reads global state cannot be tested, cannot be called from a Cloud Function, and hides who did what. Passing the actor makes every write self-documenting and keeps the layer pure.

Worked example: `setStatus`

```
setStatus: (id, status, actor, extra?) =>
  write() => {
    const index = db.reservations.findIndex((r) => r.id === id);
    if (index < 0) throw new Error("Reservation not found");
    const current = db.reservations[index]!;

    const updated: Reservation = {
      ...current, status,
      ...(status === "cancelled" && {
        cancelledAt: nowIso(),
        cancelledBy: actor.name,
        cancellationReason: extra?.reason ?? "No reason recorded",
      }),
      ...(status === "confirmed" && current.status === "pending_approval" && {
        approvedBy: actor.name, approvedAt: nowIso(), approvalNote: extra?.note,
      }),
      updatedAt: nowIso(), updatedBy: actor.id,
    };

    db.reservations[index] = updated;
    recordAudit({ entityType: "reservation", entityId: id, entityLabel: updated.reference, ... });
    return updated;
  },
```

Note the conditional spreads: cancellation metadata is written only on cancellation, approval metadata only on the `pending_approval → confirmed` transition. A blanket assignment would stamp an approver onto a cancelled booking.

`merge` — the most complex write

Implements BR-07. Three phases, in order:

```
merge: (survivorId, absorbedIds, patch, actor) =>
  write() => {
    // 1 · Re-point every child record
    for (const r of db.reservations) {
      if (absorbedIds.includes(r.customerId)) {
        r.customerId = survivorId;
        r.customerName = survivor.fullName; // denormalised field must follow
      }
    }
    for (const inv of db.invoices) { /* same */ }

    // 2 · Fill gaps on the survivor from the absorbed records
    Object.assign(survivor, patch);

    // 3 · Remove the absorbed records, roll up their history
    survivor.totalReservations = ...;
    survivor.totalRevenue = ...;
    db.customers = db.customers.filter((c) => !absorbedIds.includes(c.id));

    recordAudit({ action: "merged", ... });
    return survivor;
  },
```

⚠ **Order is not optional.** Re-pointing must happen before removal, or the reservations become orphaned. And `customerName` must be updated alongside `customerId`, because the denormalised copy is what the lists render — updating only the id would leave the old name on screen with no way to notice.

🔧 This is the single hardest operation to port to Firestore. It must become a batched write or a transaction across three collections, and it is the one place where Firestore's 500-write batch limit could realistically be hit. Volume XIV §14.7.

8.7 Aggregation in `reportsRepo`

Firestore cannot `GROUP BY`, so reports aggregate in the client. This is not a limitation of the mock — it is an accurate preview of Phase 2.

```
kpis: (ctx?: ScopeContext) =>
  read() => {
    const scoped = ctx ? scopeRecords(ctx, db.reservations) : db.reservations;
    const monthStart = isoDate(startOfMonth(TODAY));
    const lastMonthStart = isoDate(startOfMonth(subMonths(TODAY, 1)));
    // Upper bound matters: without it "this month" silently absorbs the
    // entire forward book and every growth figure becomes nonsense.
    const nextMonthStart = isoDate(startOfMonth(addMonths(TODAY, 1)));

    const thisMonth = scoped.filter((r) => r.checkIn >= monthStart && r.checkIn < nextMonthStart);
    const lastMonth = scoped.filter((r) => r.checkIn >= lastMonthStart && r.checkIn < monthStart);
    ...
  },
```

That comment marks **D-03**, which is worth understanding because it is the kind of bug that looks like working software. The original had no upper bound:

```
const thisMonth = scoped.filter((r) => r.checkIn >= monthStart); // X
```

Since the seed extends 150 days forward, "this month" silently meant *this month and the entire forward book*. The dashboard reported **+757.4% growth** and 105 reservations in a month that actually had 19. Nothing threw. The number was simply, confidently wrong.

The lesson: every date-range filter needs both bounds. An open-ended range over data that extends into the future is a bug that reports success.

The occupancy helpers

```
const occupancyCache = new Map<string, number>();

function occupancyForHotel(hotelId: string, days = 30): number {
  const key = `${hotelId}:${days}`;
  const cached = occupancyCache.get(key);
  if (cached !== undefined) return cached;

  const rows = buildInventory(hotelId, days);
  const capacity = rows.reduce((s, r) => s + r.totalRooms, 0);
  const booked = rows.reduce((s, r) => s + r.booked, 0);
  const value = capacity > 0 ? (booked / capacity) * 100 : 0;

  occupancyCache.set(key, value);
  return value;
}

function portfolioOccupancy(hotelIds?: string[]): number {
  const hotels = hotelIds ? db.hotels.filter((h) => hotelIds.includes(h.id)) : db.hotels;
  const totalRooms = hotels.reduce((s, h) => s + h.totalRooms, 0);
  if (totalRooms === 0) return 0;
  return hotels.reduce((s, h) => s + occupancyForHotel(h.id) * h.totalRooms, 0) / totalRooms;
}
```

Two things to note.

The cache is necessary. `buildInventory` generates ~120 rows per property. The Occupancy report calls `portfolioOccupancy` per city; without memoisation, rendering it would regenerate inventory for all 32 properties several times over.

Portfolio occupancy is weighted by room count. A flat average across properties would let a 17-room retreat at 90% and a 236-room hotel at 40% average to 65% — a figure describing nothing. Weighting by rooms gives the number an operational meaning: the proportion of the portfolio's *beds* that are sold.

The same reasoning appears in the Occupancy report screen:

```
// Weighted by room count - a 236-key city hotel should not be averaged
// flat against a 17-key retreat.
const averageOccupancy = totalRooms
  ? rows.reduce((s, r) => s + r.occupancyPercent * r.rooms, 0) / totalRooms
  : 0;
```

8.8 Missing documents return `null`

```
get: (id: string): Promise<Hotel | null> =>
  read() => db.hotels.find((h) => h.id === id) ?? null,
```

The `?? null` matters. Returning `undefined` makes TanStack Query throw:

Query data cannot be undefined. Please make sure to return a value other than undefined from your query function.

Every detail page then logged an error and sat in an error state for what is a perfectly normal condition — a deleted or mistyped id. With `null`, the page resolves cleanly and the standard guard renders Not-found:

```
if (hotel.isLoading) return <DetailSkeleton />;
if (!hotel.data) return <NotFound />;
```

All eight `get` -style methods were converted. See [D-08](#).

8.9 Cache keys and invalidation

Query keys always include the scope, so switching role invalidates automatically:

```
useQuery({
  queryKey: ["reservations", list.query, scope.role, scope.userId],
  queryFn: () => reservationsRepo.list(list.query, scope),
});
```

There is **no manual cache clearing anywhere in the codebase**. Changing role changes the key; TanStack Query does the rest.

Mutations invalidate explicitly, and the set is deliberately broad:

```
onSuccess: (updated) => {
  queryClient.invalidateQueries({ queryKey: ["reservation", id] });
  queryClient.invalidateQueries({ queryKey: ["reservation-audit", id] });
  queryClient.invalidateQueries({ queryKey: ["reservations"] });
  queryClient.invalidateQueries({ queryKey: ["pending-approvals"] });
  queryClient.invalidateQueries({ queryKey: ["kpis"] });
  toast.success(`Reservation ${labelFor(updated.status).toLowerCase()}`, updated.reference);
},
```

Approving a reservation changes: the reservation, its audit trail, every list containing it, the approval queue, and the dashboard KPIs. Missing one leaves a stale count somewhere — and a stale approval count on the dashboard is exactly the kind of thing that erodes trust in a system.

Convention: invalidate by prefix (`["reservations"]`), not by exact key. Exact keys miss the variants produced by different filters and pages.

8.10 What Phase 2 actually changes

FILE	CHANGE
<code>repositories/index.ts</code>	One line: <code>./mock</code> → <code>./firestore</code>
<code>repositories/firestore/*</code>	New. Same method signatures, Firestore calls inside
<code>repositories/mock/*</code>	Deleted
<code>data/seed/*</code>	Deleted, except <code>hotels.data.ts</code> which becomes a seeding script
<code>data/types.ts</code>	Unchanged, except <code>IsoDateTime</code> → <code>Timestamp</code> at the boundary
Everything above	Unchanged

Signature-compatible example:

```
// Phase 1
list: (query?: ListQuery, ctx?: ScopeContext): Promise<ListResult<Reservation>> =>
  read(() => {
    const scoped = ctx ? scopeRecords(ctx, db.reservations) : db.reservations;
    return runQuery(scoped, query, [...]);
  }),

// Phase 2 - same signature, same return type
list: async (query?: ListQuery, ctx?: ScopeContext): Promise<ListResult<Reservation>> => {
  let q = collection(firestore, "reservations");
  if (ctx?.role === "salesperson") q = query(q, where("ownerId", "=", ctx.userId));
  if (ctx?.role === "hotel_manager") q = query(q, where("hotelId", "=", ctx.hotelId));
  ...
},
```

Volume XIV covers the migration in full, including the three places where the mapping is not one-to-one: pagination, merge, and full-text search.

VOLUME IX

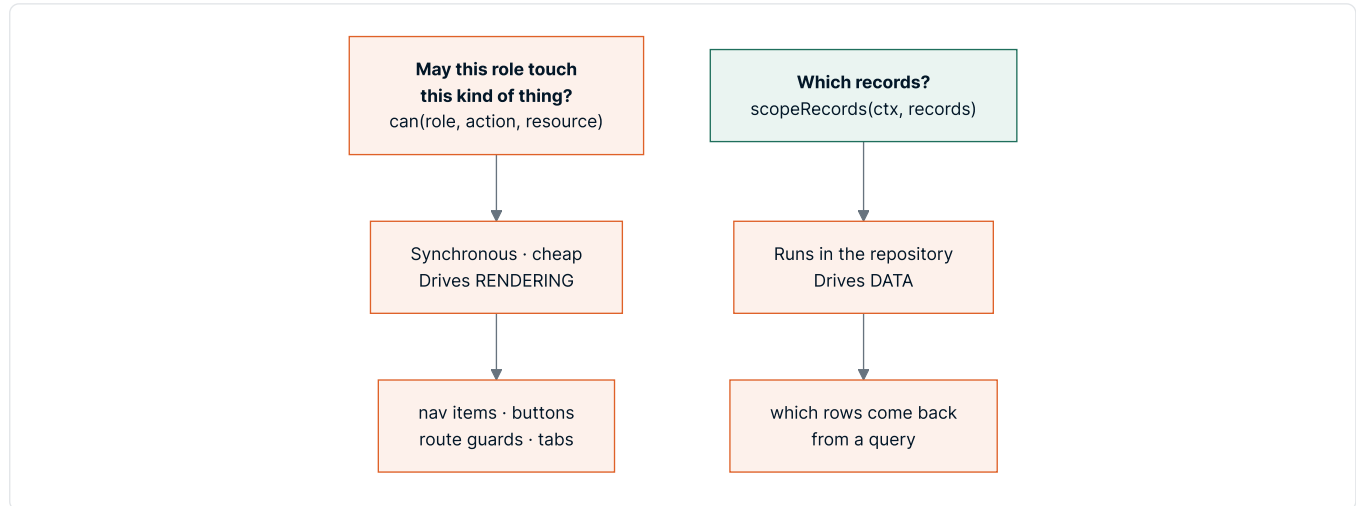
Permissions and business rules

Sources: `src/lib/permissions.ts` (296 lines) · `src/lib/rules.ts` (~210 lines)

These two files are the smallest part of the codebase and the most consequential. Every screen consults them; nothing duplicates them.

The live view is `/admin/roles`, which is **generated from these modules** — it is not a diagram of the permission model, it is the model rendered.

9.1 The two questions



They are kept separate (ADR-13) because they run at different times and different granularity. `can()` must be synchronous to drive rendering; scoping must run against a record set inside the data layer. Fusing them would mean either fetching everything and filtering in the UI — leaky and slow — or making every permission check async, which is unusable for deciding whether to render a button.

9.2 The three axes

```

export const ROLES = [
  "super_admin", "admin", "sales_manager", "salesperson",
  "hotel_manager", "finance", "support", "viewer",
] as const;

export const RESOURCES = [
  "dashboard", "customer", "company", "reservation", "reservation_approval",
  "hotel", "inventory", "rate", "invoice", "payment", "commission",
  "report", "automation", "notification", "ai",
  "user", "role", "integration", "audit_log", "setting",
] as const;

export const ACTIONS = [
  "view", "create", "edit", "cancel", "approve", "export", "merge", "import",
] as const;
  
```

8 roles × 20 resources = **160 cells**, each holding a set drawn from 8 actions.

Why these particular actions

ACTION	WHY IT IS NOT COVERED BY <code>EDIT</code>
<code>view</code>	The baseline. Absence of <code>view</code> removes the nav item entirely
<code>create</code>	Reading and editing an existing customer is a different privilege from creating new ones
<code>edit</code>	
<code>cancel</code>	Reservations are never deleted (BR-01). <code>cancel</code> replaces <code>delete</code> in this domain. Support may edit a booking without being able to cancel it
<code>approve</code>	Sign-off authority is orthogonal to editing. A salesperson may edit a booking they cannot approve
<code>export</code>	Data egress is its own concern. Viewer may read but not extract

ACTION	WHY IT IS NOT COVERED BY <code>EDIT</code>
<code>merge</code>	Destructive and irreversible. Deliberately not implied by <code>edit</code>
<code>import</code>	Bulk write. Also not implied by <code>create</code>

There is **no `delete`**. Its absence from the action list is the strongest possible statement of BR-01: the vocabulary of the system does not contain the concept.

Why `reservation_approval` is a resource, not just an action

`approve` on `reservation` would conflate two things: *may you approve* and *may you see the queue*. Finance can see the approval queue (it holds money) but cannot approve. A separate resource expresses that cleanly.

9.3 The matrix

```
const ALL: readonly Action[] = ACTIONS;
const READ: readonly Action[] = ["view"];
const READ_EXPORT: readonly Action[] = ["view", "export"];

const MATRIX: Record<Role, ResourceGrants> = {
  super_admin: Object.fromEntries(RESOURCES.map((r) => [r, ALL])),
  ...
};
```

Super Admin is generated rather than enumerated, so a newly added resource is automatically granted. The alternative — a hand-written list — would silently lock Super Admin out of every new resource until someone remembered to update it.

Full matrix

Legend: ● full control · R read-only · no access

RESOURCE	SUPER ADMIN	ADMIN	SALES MGR	SALES	HOTEL MGR	FINANCE	SUPPORT	VIEWER
Dashboard	●	R	R	R	R	R	R	R
Customers	●	●	●	●	·	R	●	R
Companies	●	●	●	●	·	R	R	R
Reservations	●	●	●	●	R	R	●	R
Approvals	●	●	●	·	·	·	·	·
Properties	●	●	R	R	●	R	R	R
Inventory	●	●	R	R	●	·	R	R
Rate plans	●	●	R	R	R	·	·	R
Invoices	●	●	R	R	·	●	R	R
Payments	●	●	·	·	·	●	·	·
Commissions	●	R	R	R	·	●	·	·
Reports	●	R	R	R	R	R	R	R
Automation	●	●	R	·	·	·	·	·
Notifications	●	●	R	R	R	R	●	R
Assistant	●	R	R	R	R	R	R	·
Users	●	●	R	·	·	·	·	·
Roles	●	·	·	·	·	·	·	·
Integrations	●	·	·	·	·	·	·	·
Audit log	●	R	R	·	·	R	·	·
Settings	●	●	·	·	·	·	·	·

The bolded cell is the most instructive in the table: **hotel managers have read-only access to rate plans**. It is the clearest demonstration of the permission model in the running product, and BR-04 exists to enforce it.

Observable consequence

ROLE	NAVIGATION ITEMS
Super Admin	16
Admin	14
Sales Manager	13
Salesperson	13
Finance	11
Support	9
Viewer	8
Hotel Manager	7

Verified in the browser: switching Super Admin → Hotel Manager takes the sidebar from 16 items to 7, changes the dashboard to a property day-sheet, and turns 12 Edit buttons on the rate screen into 12 "Locked" markers.

9.4 The API

```
/** Can this role perform `action` on `resource`? */
export function can(role: Role, action: Action, resource: Resource): boolean {
  return MATRIX[role]?.[resource]?.includes(action) ?? false;
}

/** Does this role have any access at all? Drives nav visibility. */
export function canAccess(role: Role, resource: Resource): boolean {
  return (MATRIX[role]?.[resource]?.length ?? 0) > 0;
}

/** Every action a role holds on a resource – used by the admin matrix screen. */
export function grantsFor(role: Role, resource: Resource): readonly Action[] {
  return MATRIX[role]?.[resource] ?? [];
}
```

Both `can` and `canAccess` **default to denying**. The `?? false` and `?? 0` mean a resource absent from a role's grants is closed, not open. Adding a resource to `RESOURCES` without adding it to a role's map denies it — the safe direction.

Where each is used

FUNCTION	USED BY	EXAMPLE
<code>canAccess</code>	Route guards, nav visibility	<code>if (!canAccess(role, "invoice")) return <Forbidden/></code>
<code>can</code>	Buttons, actions, sections	<code>{can(role, "create", "reservation") && <Button...>}</code>
<code>grantsFor</code>	<code>/admin/roles</code> matrix cells	Renders the tooltip listing granted actions

9.5 Row-level scoping

```
export interface ScopeContext {
  role: Role;
  userId: string;    // in Phase 1, from the role switcher
  hotelId?: string;  // hotel managers are pinned to one property
}

export function scopeRecords<T extends { ownerId?: string; hotelId?: string }>(
  ctx: ScopeContext, records: T[],
): T[] {
  if (ctx.role === "salesperson") {
    return records.filter((r) => !r.ownerId || r.ownerId === ctx.userId);
  }
  if (ctx.role === "hotel_manager" && ctx.hotelId) {
    return records.filter((r) => !r.hotelId || r.hotelId === ctx.hotelId);
  }
  return records;
}
```

Twelve lines. Two rules.

⚠️ **The `!r.ownerId ||` clause is deliberate and subtle.** A record with no owner is visible to everyone. This is correct — an unassigned lead should not be invisible to the entire sales team — but it means *ownership absence is permissive*. If a future collection uses `ownerId` with different semantics, this default must be revisited.

The scope flows from the session

```
// src/lib/session.ts
export function useScope(): ScopeContext {
  const user = useCurrentUser();
  return { role: user.role, userId: user.id, hotelId: user.hotelId };
}
```

`useCurrentUser()` resolves the seeded user for the current role, so `hotelId` is present exactly when the role is `hotel_manager`.

Applied before search — a security property

Covered in Volume VIII §8.4, restated because it matters: scoping runs *before* search and pagination. `total` therefore counts only visible records, and a salesperson searching for another rep's account gets a clean empty result rather than evidence the record exists.

9.6 The eight business rules

`src/lib/rules.ts`. Each is a function; each is also declared in `BUSINESS_RULES` with its rationale and the name of the function enforcing it, so `/admin/roles` can render them.

```
export interface BusinessRule {
  id: string;
  rule: string;
  rationale: string;
  enforcedIn: string;
}
```

#	RULE	ENFORCED IN
BR-01	A reservation is never deleted, only cancelled	<code>canCancelReservation()</code> + absence of any delete path
BR-02	Bookings ≥ ₹50,000 require approval	<code>requiresApproval()</code>
BR-03	Completed / cancelled / no-show reservations are locked	<code>canEditReservation()</code>
BR-04	Hotel managers cannot edit rate plans	<code>canEditRates()</code>
BR-05	A salesperson sees only their assigned accounts	<code>scopeRecords()</code>
BR-06	Customer email and phone must be unique	<code>isDuplicateEmail()</code> / <code>isDuplicatePhone()</code>
BR-07	Merging moves all reservations and invoices to the survivor	<code>customersRepo.merge()</code>
BR-08	Every change is written to the append-only audit log	<code>recordAudit()</code>

9.7 The { allowed, reason } pattern

Three rule functions return a shape rather than a boolean:

```
export function canCancelReservation(
  role: Role, reservation: Reservation,
): { allowed: boolean; reason?: string } {
  if (!can(role, "cancel", "reservation")) {
    return { allowed: false, reason: "Your role cannot cancel reservations." };
  }
  if (reservation.status === "cancelled") {
    return { allowed: false, reason: "This reservation is already cancelled." };
  }
  if (reservation.status === "completed") {
    return { allowed: false, reason: "Completed reservations are locked." };
  }
  if (reservation.status === "checked_in") {
    return { allowed: false, reason: "Guest is in-house. Check out before cancelling." };
  }
  return { allowed: true };
}
```

This shape is what makes **ADR-12** implementable. A boolean would let the UI hide the button; a reason obliges it to explain:

```
{cancelCheck.allowed ? (
  <CancelDialog ... />
) : (
  <Tooltip content={cancelCheck.reason}>
    <span>
      <Button variant="secondary" disabled leadingIcon={<Ban className="size-4" />}>
        Cancel
      </Button>
    </span>
  </Tooltip>
)}
```

Four distinct reasons are produced, and each is genuinely different information for the user. "You can't do that" would have collapsed all four into one unhelpful message.

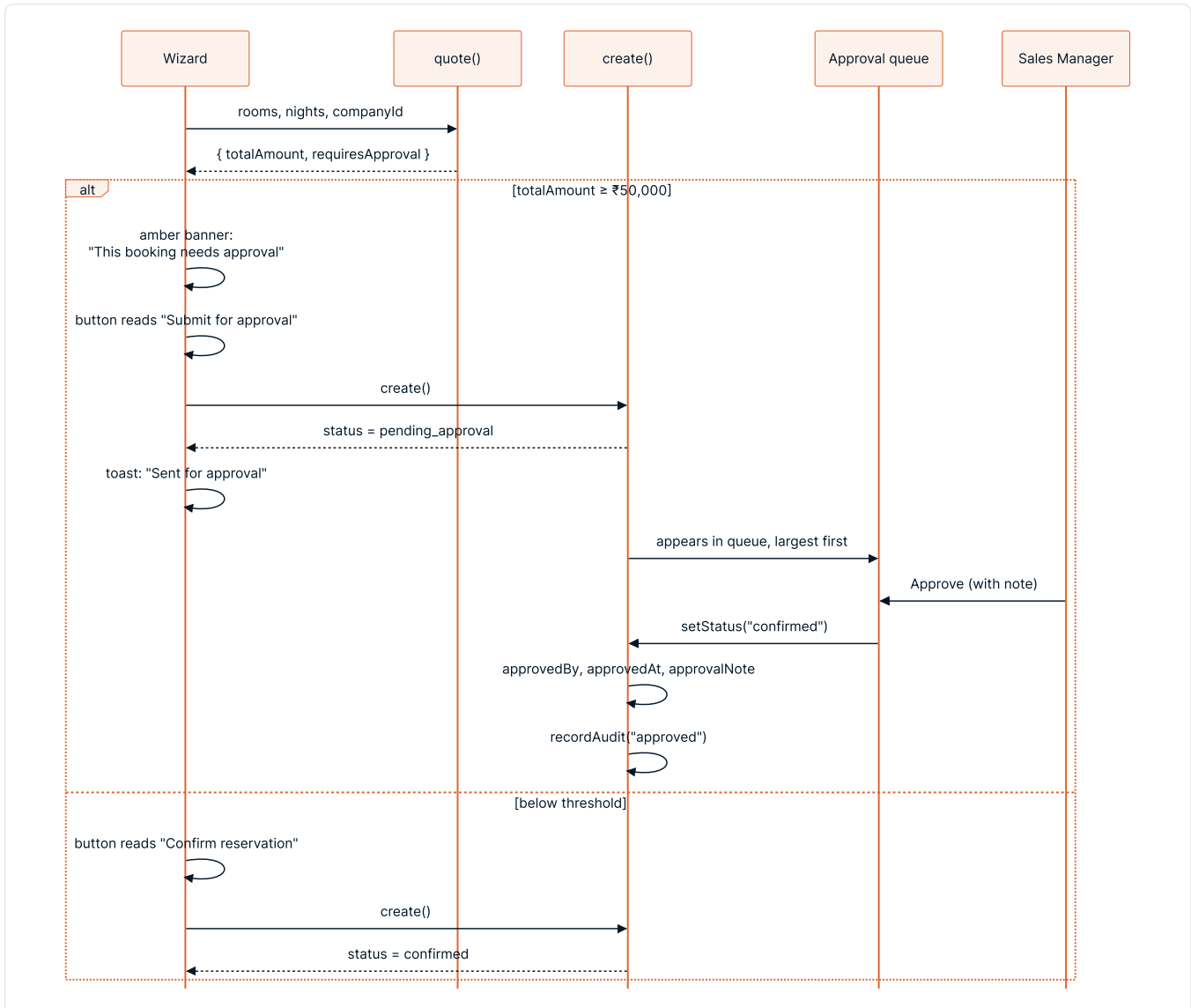
Note the wrapping `<span>` — a disabled button emits no pointer events, so the tooltip would never fire without an enabled element to receive them.

9.8 BR-02 — the approval threshold

```
export const APPROVAL_THRESHOLD = 50_000;

export function requiresApproval(totalAmount: number): boolean {
  return totalAmount >= APPROVAL_THRESHOLD;
}
```

One constant, imported by the quote engine, the wizard, the approvals screen, the settings screen and the rules table. Changing the threshold is a one-line change with no search-and- replace.



The threshold is surfaced **three times** before it can surprise anyone: in the live quote panel, in a banner on the review step, and in the button's own label — which changes from "Confirm reservation" to "Submit for approval". A rule discovered only after submission is a rule that generates support tickets.

9.9 BR-03 — the terminal-status lock

```

const TERMINAL_STATUSES: ReservationStatus[] = ["completed", "cancelled", "no_show"];

export function isTerminal(status: ReservationStatus): boolean {
  return TERMINAL_STATUSES.includes(status);
}

export function canEditReservation(role: Role, reservation: Reservation) {
  if (!can(role, "edit", "reservation")) {
    return { allowed: false, reason: "Your role has read-only access here." };
  }
  if (isTerminal(reservation.status)) {
    return { allowed: false, reason: `${labelFor(reservation.status)} reservations are locked.` };
  }
  return { allowed: true };
}

```

Surfaced on the detail page as a lock chip beside the status, a tooltip on disabled actions, and a side-rail card carrying the reason.

Why it exists. Once a stay has resolved, its folio is the basis for the invoice and the commission accrual. Editing it after the fact would silently change money already accounted for — the invoice would no longer reconcile to the booking that produced it.

The state machine

```
export function nextStatuses(current: ReservationStatus): ReservationStatus[] {
  const transitions: Record<ReservationStatus, ReservationStatus[]> = {
    draft: ["pending_approval", "confirmed", "cancelled"],
    pending_approval: ["confirmed", "cancelled"],
    confirmed: ["checked_in", "cancelled", "no_show"],
    checked_in: ["completed"],
    completed: [],
    cancelled: [],
    no_show: [],
  };
  return transitions[current];
}
```

The three terminal states return an empty array, which is what drives the *absence* of action buttons on the detail page. The UI does not decide which transitions exist; it asks.

Note `checked_in → ["completed"]` only. A guest who is physically in the hotel cannot have their booking cancelled — they must be checked out first. `canCancelReservation` states this explicitly: *"Guest is in-house. Check out before cancelling."*

9.10 BR-04 — pricing is owned centrally

```
export function canEditRates(role: Role): { allowed: boolean; reason?: string } {
  if (role === "hotel_manager") {
    return { allowed: false, reason: "Rate plans are managed centrally by the revenue team." };
  }
  if (!can(role, "edit", "rate")) {
    return { allowed: false, reason: "Your role cannot edit pricing." };
  }
  return { allowed: true };
}
```

⚠️ **The explicit `hotel_manager` check appears redundant** — the matrix already grants them only `["view"]` on `rate`. It is deliberate, for two reasons:

1. It produces a *specific* reason. The generic path says "Your role cannot edit pricing"; this says who *does* own it, which is what the user actually needs to know.
2. It is defence in depth. If someone widened the hotel-manager grant while reasoning about something else, this check would still hold the rule.

How it presents

The rate screen renders a banner, and every row's action cell becomes a lock:

```
{!access.allowed && (
  <Card className="mb-6 bg-grey-50">
    <CardBody className="flex items-start gap-3">
      <Lock className="size-4 text-grey-400 shrink-0 mt-0.5" />
      <div>
        <p className="text-base font-medium text-ink-900">Read-only for your role</p>
        <p className="text-sm text-grey-600 mt-1 leading-relaxed">
          {access.reason} You can see every rate here, but changes must come from
          the revenue team. Switch role in the top bar to see the editable view.
        </p>
      </div>
    </CardBody>
  </Card>
)}
```

The last sentence is the detail worth copying elsewhere: it tells the reviewer how to *see the other side*, which turns a restriction into a demonstration.

9.11 BR-06 — uniqueness

```
export function isDuplicateEmail(
  email: string, existing: { email: string; id: string }[], ignoreId?: string,
): boolean {
  const normalised = email.trim().toLowerCase();
  return existing.some((c) => c.id !== ignoreId && c.email.trim().toLowerCase() === normalised);
}

export function isDuplicatePhone(
  phoneValue: string, existing: { phone: string; id: string }[], ignoreId?: string,
): boolean {
  const digits = phoneValue.replace(/\D/g, "").slice(-10);
  if (digits.length < 10) return false;
  return existing.some(
    (c) => c.id !== ignoreId && c.phone.replace(/\D/g, "").slice(-10) === digits,
  );
}
```

Normalisation is the whole design

Email: trimmed and lower-cased. `Ananya@Bose.com` and `ananya@bose.com` are the same address, and treating them as distinct is how duplicate customer records are born.

Phone: reduced to the **last 10 digits**. In India these are equivalent:

```
+91 98765 43210
09876543210
9876543210
+919876543210
```

Comparing raw strings would treat all four as different people. Taking the last 10 digits after stripping non-digits collapses them correctly, and sidesteps the country-code and trunk-prefix variations entirely.

`ignoreId` excludes the record being edited, so saving a customer without changing their email does not report them as their own duplicate.

It warns; it never blocks

Per [ADR-21](#). The form shows:

▲ Another customer already has this email. You can still save, then resolve it on the [duplicates](#) screen.

9.12 BR-07 — merge semantics

Detection runs three strategies, in confidence order:

ORDER	MATCH ON	CONFIDENCE	WHY THIS ORDER
1	Normalised phone	High	The strongest identity signal in this market — people change email, keep numbers
2	Lower-cased email	High	Strong, but shared family and role addresses exist
3	Exact full name	Worth checking	Two real people can share a name

The merge itself re-points children, fills gaps on the survivor, rolls up history, then removes the absorbed records. Order is load-bearing — Volume VIII §8.6.

The UI defaults the survivor to the record with the most history:

```
const [survivorId, setSurvivorId] = useState(
  // Default to the record with the most history - it has the most to lose.
  [...group.records].sort(
    (a, b) => b.totalReservations - a.totalReservations || (a.createdAt < b.createdAt ? -1 : 1),
  )[0]!.id,
);
```

Ties break toward the older record.

9.13 BR-08 — the audit trail

One helper, called by every write. Covered in Volume VIII §8.6.

What makes it trustworthy:

PROPERTY	MECHANISM
Append-only	Only <code>unshift</code> . No update or delete path exists
Attributed	Every write takes an <code>Actor</code> ; id, name and role are all recorded
Complete	The helper is the only way to write; forgetting it means the write does not compile into the established pattern
Navigable	<code>entityType</code> + <code>entityId</code> let <code>/admin/audit-log</code> link every row to its record

9.14 Where the rules surface

RULE	SURFACES AT
BR-01	No delete control anywhere; cancel dialog wording; cancelled records stay in every list
BR-02	Wizard quote panel · review banner · button label · approvals queue · settings · rules table
BR-03	Lock chip · disabled actions with tooltips · side-rail explanation
BR-04	Read-only banner · Locked markers · tooltip on the property page's rate button
BR-05	Scoped lists · role-aware page descriptions · role-aware empty states
BR-06	Inline warnings on the customer form · import validation
BR-07	Merge screen: survivor choice, preview dialog, consequence text
BR-08	Reservation timeline tab · <code>/admin/audit-log</code>

9.15 Phase 2 — the rules become security rules

🔑 Everything in this volume currently runs **only in the browser**. That is acceptable in Phase 1 because there is no server to protect. In Phase 2 it is not.

The matrix must be restated as Firestore security rules:

```
// Sketch - Phase 2
match /reservations/{id} {
  allow read: if request.auth != null && (
    hasRole(['super_admin','admin','sales_manager','finance','support','viewer'])
    || (hasRole(['salesperson']) && resource.data.ownerId == request.auth.uid)
    || (hasRole(['hotel_manager']) && resource.data.hotelId == userHotelId())
  );

  allow update: if hasPermission('reservation','edit')
    && !(resource.data.status in ['completed','cancelled','no_show']); // BR-03

  allow delete: if false; // BR-01
}
```

`allow delete: if false` is BR-01 expressed at the only level that truly enforces it.

The risk to manage: two copies of the rules — TypeScript and Firestore — that can drift. Volume XIV §14.8 proposes generating the security rules from **MATRIX** rather than hand-writing them, which is the only reliable way to keep them in step.

VOLUME X

Screen teardown

All 38 routes, one at a time. Each entry gives the route, the guard, the data it pulls, the states it handles, the interactions it offers, and — where there was a real choice — why it is shaped the way it is.

Contents

\$	SCREEN	ROUTE
10.1	App shell	<i>(frame)</i>
10.2	Dashboard	/dashboard
10.3	Reservations list	/reservations
10.4	Calendar	/reservations/calendar
10.5	New reservation	/reservations/new
10.6	Reservation detail	/reservations/:id
10.7	Approvals	/reservations/approvals
10.8	Customers	/crm/customers
10.9	Customer detail	/crm/customers/:id
10.10	Customer form	/crm/customers/new , /:id/edit
10.11	Companies	/crm/companies
10.12	Company detail	/crm/companies/:id
10.13	Company form	/crm/companies/new , /:id/edit
10.14	Duplicates	/crm/merge
10.15	Import	/crm/import
10.16	Properties	/hotels
10.17	Property detail	/hotels/:id
10.18	Inventory	/hotels/:id/inventory
10.19	Rate plans	/hotels/:id/rates
10.20	Invoices	/finance/invoices
10.21	Invoice detail	/finance/invoices/:id
10.22	Payments	/finance/payments
10.23	Commissions	/finance/commissions
10.24	Reports ×6	/reports/*
10.25	Automation ×3	/automation/*
10.26	Notifications ×2	/notifications/*
10.27	Assistant	/ai
10.28	Admin ×5	/admin/*
10.29	Design system	/design-system

10.1 App shell

Source: `src/components/app/` — `AppShell` · `Sidebar` · `TopBar` · `RoleSwitcher` · `CommandPalette` · `AIPanel` · `navigation.ts`



Navigation is generated from permissions

`navigation.ts` declares each item with the resource it needs; the sidebar filters:

```
{ label: "Invoices", to: "/finance/invoices", icon: Receipt, resource: "invoice" }
```

Nothing is hardcoded per role. Adding a resource to a role's grants makes the nav item appear.

Layout responsibilities

ELEMENT	BEHAVIOUR
Sidebar	Fixed. Collapsible to a 68 px icon rail; preference persisted
Sidebar (< lg)	Becomes a Radix Dialog drawer
Top bar	Search, AI panel, notifications, role switcher
Main	The only scrolling region — <code>overflow-y-auto</code>

⚠ **The top bar carries no create action.** It did originally, which put a second orange primary button roughly 100 px above the page header's own — visible on the dashboard and the reservations list, and a direct breach of the one-primary-action-per-view rule in Volume V §5.2. Create actions now live only in the page header, where they sit beside that screen's other actions. %K is the global path. See [D-10](#).

Making `<main>` the scroll container rather than the document keeps the sidebar and top bar fixed without `position: fixed`, which avoids the whole class of iOS viewport bugs.

The scroll-lock guard

```
useEffect(() => {
  if (document.querySelector("[data-radix-focus-guard]")) return;
  if (document.body.style.pointerEvents === "none") {
    document.body.style.removeProperty("pointer-events");
  }
  document.body.removeAttribute("data-scroll-locked");
}, [pathname]);
```

Defensive. Radix sets `pointer-events: none` on `<body>` while a modal is open; a modal that unmounts while open can strand it, making the **entire app unclickable with no visible cause**. The guard checks for a live focus guard first, so it never interferes with a legitimately open modal. See [D-02](#).

Command palette (%K)

Groups: navigation · actions · records. Record search calls `searchRepo.query()` across reservations, customers, companies, hotels and invoices, scoped to the actor.

```
function choose(entry: Entry) {
  // Close before navigating. Unmounting the route while the modal is
  // still open can strand Radix's scroll lock on <body>.
  setOpen(false);
  navigate(entry.to);
}
```

10.2 Dashboard

Route `/dashboard` · **Guard** `dashboard` · **Source** `features/dashboard/DashboardPage.tsx`

The only screen that changes shape by role rather than merely filtering.

ROLE	TITLE	KPI ROW
Hotel manager	<i>"{Property} today"</i>	Arrivals · Departures · In house · Occupancy
Salesperson	<i>"Your pipeline"</i>	Your revenue · Reservations · Awaiting approval · Cancellation rate
Finance	<i>"Financial position"</i>	Revenue · Overdue invoices · Average booking · Awaiting approval
Others	<i>"Portfolio overview"</i>	Revenue · Reservations · Awaiting approval · Cancellation rate

Data

```
const kpis      = useQuery({ queryKey: ["kpis", scope.role, scope.userId, ... ]});
const series    = useQuery({ queryKey: ["revenue-series", ...], ... });
const daySheet  = useQuery({ queryKey: ["day-sheet", today, ...], ... });
const approvals = useQuery({ queryKey: ["pending-approvals", ...],
                                enabled: can(role, "view", "reservation_approval") });
const recent    = useQuery({ queryKey: ["recent-reservations", ...], ... });
```

Five independent queries, each with its own skeleton. `enabled` on the approvals query means roles without the permission never issue it — no wasted call, no flash of a panel that then disappears.

Why four KPI variants rather than one set

A hotel manager does not care about portfolio revenue; they care who is arriving. Finance does not care about arrivals; they care what is overdue. Showing everyone the same four numbers would mean at least two of them are noise for every viewer.

Sections

- 1. **KPI row** — 4 tiles; the first carries a 2 px `brand-gradient` top edge, the only place the gradient appears outside the logo.
- 2. **Revenue chart** (2 cols) — 12-month area chart.
- 3. **Today** (1 col) — arrivals / departures / in-house, plus the first four arriving guests.
- 4. **Waiting on you** — approval queue preview, only for roles that can approve.
- 5. **Recent reservations** (2 cols) + **Daily briefing** (1 col) — the AI card; for roles without `ai` but with `invoice` , a Receivables card takes its place.
- 6. **Room nights by month** — bar chart.

Empty "today"

The day panel showing zeros is legitimate on a quiet day. It was also the symptom that exposed the seed being too small — see [D-05](#).

10.3 Reservations list

Route `/reservations` · Guard `reservation`

COLUMN	NOTES
Reference	Sortable; reference over guest name as a two-line cell
Property	Two-line: name + city
Stay	<code>hideBelow="md"</code> — check-in and night count
Rooms	<code>hideBelow="lg"</code> , numeric
Channel	<code>hideBelow="xl"</code>
Status	Pill + a <code>≥50k</code> marker where approval applies
Owner	<code>hideBelow="xl"</code>
Value	Sortable, numeric

Filters: status (7) · channel (6). Search covers reference, guest, property.

The page description changes by role — *"Bookings on your accounts"* for a salesperson, *"Bookings at your property"* for a hotel manager, *"Every booking across the 32-property portfolio"* otherwise. The scoping is real; the sentence explains it rather than leaving the user to infer it.

🚧 **Known gap.** The screen mentions bulk actions in its description but row selection is not implemented. Volume XIII §13.7.

10.4 Reservation calendar

Route `/reservations/calendar` · Source `features/reservations/CalendarPage.tsx`

Two views over the same data.

Month grid

A stay occupies **every night from check-in up to, but not including, check-out** — the hospitality convention, not an inclusive date range:

```
for (const d of eachDayOfInterval({ start, end: addDays(end, -1) })) {
  map.set(key, [...(map.get(key) ?? []), r]);
}
```

Getting this wrong by one day is the classic hotel-software bug: a guest checking out on the 5th would appear to occupy the night of the 5th, and every occupancy figure would be inflated.

Three bookings per day are shown, then `+N more`.

Property timeline

Rows are properties, columns are days, bookings are bars. Overlapping bars are packed into sub-rows by a greedy interval algorithm:

```
function packRows(items: Reservation[], firstDay: Date, dayCount: number): PackedBar[][] {
  const bars = items.map((r) => { ... }).sort((a, b) => a.offset - b.offset);

  const rows: PackedBar[][] = [];
  for (const bar of bars) {
    const row = rows.find((r) => {
      const last = r[r.length - 1];
      return last.offset + last.span <= bar.offset; // fits after the last bar
    });
    if (row) row.push(bar);
    else if (rows.length < 6) rows.push([bar]); // cap at 6 sub-rows
  }
  return rows;
}
```

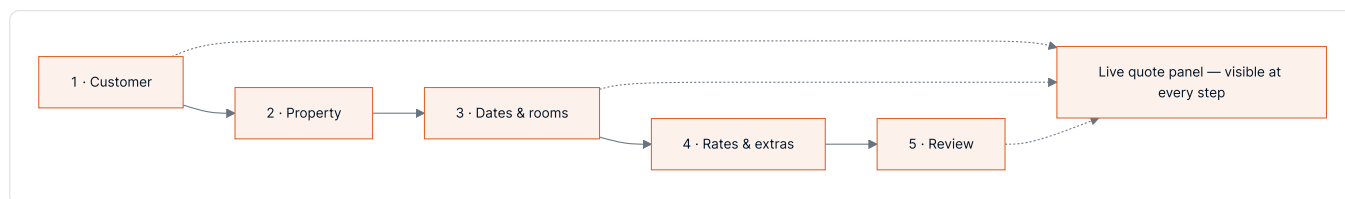
Sorting by start, then placing each bar in the first row whose last bar has ended. $O(n \cdot \text{rows})$, which at this scale is nothing. The 6-row cap stops a very busy property from producing a 40-row-tall band.

Bars are positioned as percentages so the grid stays responsive:

```
style={{ left: `${(offset / days.length) * 100}%`, width: `${(span / days.length) * 100}%` }}
```

10.5 New reservation wizard

Route `/reservations/new` · Guard `reservation` · 700 lines, the most complex screen.



Step gating

```
const canAdvance =
  (step === "customer" && Boolean(customerId)) ||
  (step === "property" && Boolean(hotelId)) ||
  (step === "dates" && nights > 0 && rooms.length > 0) ||
  step === "rates" || step === "review";
```

Continue is `disabled` until the step's decision is made. The stepper allows jumping *back* but never forward — you cannot skip a decision the later steps depend on.

The live quote

Recomputed with `useMemo` on every change:

```
const quote = useMemo(
  () => reservationsRepo.quote(rooms, nights, customer?.companyId),
  [rooms, nights, customer?.companyId],
);
```

The panel is `lg:sticky lg:top-6`, so on desktop the price stays visible while scrolling through room selection. Watching the total move as rooms are added is the single most useful feedback in the flow.

The approval threshold, surfaced three times

- 1. In the quote panel, as soon as the total crosses ₹50,000.
- 2. As an amber banner on the review step, quoting the amount and the threshold.
- 3. In the submit button, which reads **"Submit for approval"** rather than "Confirm reservation".

Room selection

Per room type: a stepper for quantity, then rate plan, adults and children once quantity > 0. The increment button is capped at `roomType.totalRooms`.

Progressive disclosure is deliberate — showing rate plan, adults and children for every room type at once would produce a wall of controls for a property with five room types, most of them irrelevant.

Paused properties

```
<button disabled={h.status === "paused"} ...>
  {disabled && <StatusPill tone="warning">Paused</StatusPill>}
```

A paused property cannot take new bookings, and the reason is on the card rather than left to be discovered.

10.6 Reservation detail

Route `/reservations/:id` · Source `ReservationDetailPage.tsx` (~750 lines)

Header: breadcrumbs, reference, guest + property, status pill, lock chip when terminal, and the action set.

Actions are computed, never assumed

```
const cancelCheck = canCancelReservation(role, r);
const editCheck   = canEditReservation(role, r);
const locked      = isTerminal(r.status);
const transitions = nextStatuses(r.status);
const canApprove  = r.status === "pending_approval"
  && can(role, "approve", "reservation_approval");
```

Every button asks the rules layer. Check-in appears only if `transitions` includes `checked_in`; Cancel is disabled with a tooltip carrying `cancelCheck.reason`.

Contextual banners

STATUS	BANNER
<code>pending_approval</code>	Amber — the amount, the threshold, who raised it and when
<code>cancelled</code>	Red — reason, who cancelled, when, and <i>"The record is kept for audit — reservations are never deleted"</i>

Four tabs

Folio — room lines, then a right-aligned totals block: room charges → extras → corporate discount → GST → total. Every component of the quote is shown because the folio must be self-explaining (Volume VI §6.8).

Guests — primary guest flagged.

Timeline — the audit trail as a vertical timeline, colour-coded by action.

Documents — voucher, confirmation email, invoice link. Buttons fire toasts explaining that delivery arrives in Phase 3, rather than pretending to work.

Side rail

Booking facts with links to customer, company and property; the AI summary card; and — when editing is blocked — a card carrying the reason.

10.7 Approval queue

Route `/reservations/approvals` · **Guard** `reservation_approval`

Sorted by value **descending** — the bookings holding the most money get seen first, which is the order a manager would choose anyway.

Each card carries reference, status, channel, guest, company, property, dates, nights, rooms, who raised it and when, the total, the per-room-night rate, and any special request.

The per-room-night figure is the reviewer's actual tool. ₹6,44,280 means little in isolation; ₹11,505 per room-night is immediately assessable against what that property should fetch.

Roles that can see but not act

```
{!canDecide && (
  <Card className="mb-6 bg-grey-50">
    <CardBody className="flex items-start gap-3">
      <ShieldCheck className="size-4 text-grey-400 shrink-0 mt-0.5" />
      <p className="text-base text-grey-600 leading-relaxed">
        You can see this queue but not act on it. Approving is limited to sales
        managers and admins – switch role in the top bar to try it.
      </p>
    </CardBody>
  </Card>
)}
```

Approve and decline

Both open dialogs. Approve takes an optional note recorded on the audit trail; Decline requires a reason from the standard list and cancels the reservation — declining is not a separate status, because a declined booking *is* a cancelled booking, and inventing a status for it would fragment the cancellation report.

10.8 Customers list

Route `/crm/customers` · **Guard** `customer`

Columns: Customer (avatar, name, VIP star, email) · Company · Phone · City · Status · Stays · Revenue · Owner · Last activity.

Actions in the header adapt: **Duplicates** if `can(merge)`, **Import** if `can(import)`, **New customer** if `can(create)`.

The VIP star carries a tooltip — “VIP — notify the property before arrival” — because a star with no explanation is decoration.

10.9 Customer detail

Route `/crm/customers/:id`

Summary strip: reservations · lifetime value · last stay · source and owner.

Tabs: **Reservations** (table, click-through) · **Preferences** (chips, with a note that they are passed to the property) · **Notes** (internal, with created/updated stamps).

Side rail: contact block with `mailto:` and formatted phone, company link, and the AI summary.

The empty state is written for the specific customer: “*{name} has not booked with Fidato so far.*” — not a generic “No records”.

10.10 Customer form

Routes `/crm/customers/new` · `/crm/customers/:id/edit`

One component for both. `react-hook-form` + Zod:

```
const schema = z.object({
  firstName: z.string().min(1, "First name is required"),
  email: z.string().min(1, "Email is required").email("Enter a valid email address"),
  phone: z.string().min(1, "Phone is required")
    .refine((v) => v.replace(/\D/g, "").length >= 10, "Enter a valid 10-digit number"),
  ...
});
```

Live duplicate warning

```
const duplicateEmail = email && isDuplicateEmail(email, db.customers, id);
const duplicatePhone = phone && isDuplicatePhone(phone, db.customers, id);
```

Warns as you type, using the same rule functions the import wizard uses. `id` is passed so a record is not flagged as its own duplicate.

Error messages are written as instructions — *"Enter a valid email address"* — not as diagnoses — *"Invalid email"*.

10.11 Companies list

Route `/crm/companies` · Guard `company`

The distinguishing column is **credit used**, rendered as a percentage with a `ProgressBar` whose tone shifts at 60% and 80%, and a tooltip giving the absolute figures.

The empty state is role-aware:

No accounts assigned to you A sales manager assigns accounts. Switch role in the top bar to see the full list.

Offering "Add a company" to someone whose problem is *assignment* would be unhelpful.

10.12 Company detail

Route `/crm/companies/:id`

Tabs: Reservations · Contacts · Contract.

Side rail leads with a **Credit** card — utilisation as a large percentage, absolute figures, a progress bar, and above 70% a line of advice:

Utilisation is high. Worth a conversation with finance before the next large booking.

That sentence is the difference between a dashboard and a tool. The number alone requires the reader to know the threshold; the sentence tells them what to do.

10.13 Company form

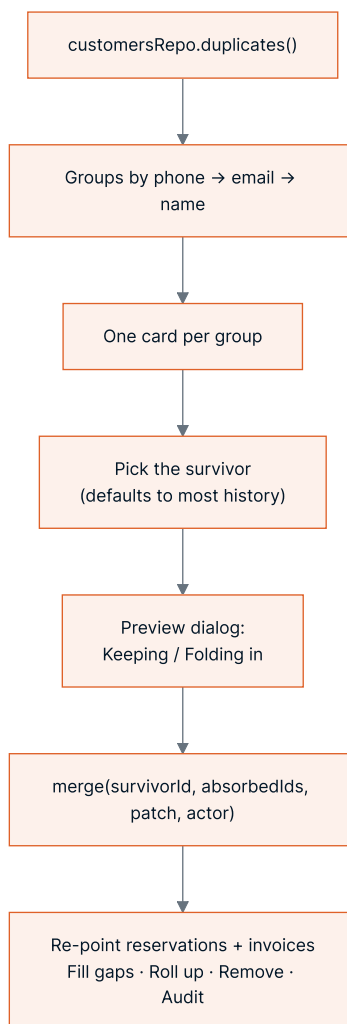
Routes `/crm/companies/new` · `/crm/companies/:id/edit`

Grouped: identity · classification (tier, status, industry) · contact · address · commercial (credit limit, payment terms, negotiated discount) · notes.

The commercial group is where the fields that *change how the system behaves* live — the negotiated discount feeds the quote engine, payment terms set invoice due dates.

10.14 Duplicate merge

Route `/crm/merge` · Guard `customer` (+ `merge` for the action)



Each record renders as a selectable card showing company, stays, revenue, created date and source — the facts you need to decide which record is the real one.

The confirmation dialog splits **Keeping** (green) from **Folding in** (grey) and states the consequence in plain terms:

3 reservations and any invoices will move onto Ananya Bose. Missing fields are filled from the records being folded in. The merge is written to the audit trail.

Empty state is a success, not a void: *"No duplicates found — every customer record has a distinct phone number, email address and name."*

10.15 Import wizard

Route `/crm/import` · Guard `customer (+ import)`

Three steps: **Upload** → **Map columns** → **Review & import**.

Column auto-mapping

```
function guessMapping(headers: string[]): Record<string, string> {
  const normalise = (s: string) => s.toLowerCase().replace(/[^a-z]/g, "");
  for (const field of TARGET_FIELDS) {
    const target = normalise(field.label);
    const alt = normalise(field.key);
    const match = headers.find((h) => {
      const n = normalise(h);
      return n === target || n === alt || n.includes(alt) || alt.includes(n);
    });
    if (match) map[field.key] = match;
  }
  return map;
}
```

`first_name`, `First Name`, `firstname` and `FIRSTNAME` all match. The common case then needs no work at all, and the mapping step becomes a confirmation rather than a chore.

Two classes of problem

	TREATMENT	WHY
Duplicated inside the file	Error — row skipped	A file containing the same person twice is a mistake in the file
Collides with an existing record	Warning — row imports	May be legitimate; resolve on the merge screen (ADR-21)

Also errors: missing required field, malformed email, phone under 10 digits.

The review table shows every row with ✓ / ⚠ / ✕ and the specific issue. Counts summarise: *will import* / *with warnings* / *skipped*.

The sample file

```
Use a sample file (8 rows)
Two rows in the sample collide with existing records, so you can see how warnings behave.
```

A reviewer can exercise the whole flow, including the warning path, without preparing a CSV.

10.16 Properties list

Route `/hotels` · Guard `hotel`

Grid and table views, switchable.

Grid cards carry a 1 px `brand-gradient` top edge standing in for property photography, then name, city, star rating, category, description, room count and commission, with room-mix chips.

🔧 The gradient band is a placeholder. Phase 2 adds real photography, and the card is laid out to accept a 16:9 image in that position without reflowing.

10.17 Property detail

Route `/hotels/:id`

Tabs: **Overview** (description, features, facilities, amenities as chips) · **Rooms** (room types with rates) · **Bookings** · **Location**.

The Location tab is the clearest payoff from using real fact-sheet data — real landmarks with real distances:

Airport – Pune	24 km
Railway Station	17 km
Shaniwar Wada	11.4 km

The rate-plan button changes by role: primary **"Rate plans"** when editable, secondary **"View rates"** wrapped in a tooltip carrying the restriction when not.

10.18 Inventory grid

Route `/hotels/:id/inventory` · Guard `inventory`

Room types down, days across, 30 / 45 / 60-day horizons.

Cell shading by pressure

```
function pressureClass(soldPercent: number): string {
  if (soldPercent >= 100) return "bg-brand-red";
  if (soldPercent >= 88)  return "bg-brand-orange";
  if (soldPercent >= 70)  return "bg-brand-orange-100";
  if (soldPercent >= 45)  return "bg-brand-yellow-50";
  return "bg-white";
}
```

The cell shows rooms **still available**; the shade shows how sold-out the day is. A wall of dark cells reads as "we are full" before a single number is read — which is the entire point of a grid rather than a table.

Weekends carry a tinted background, today's column is highlighted, and every cell has a tooltip with the full breakdown.

A footnote states plainly that the data is simulated and Phase 2 reads live PMS inventory.

10.19 Rate plans

Route `/hotels/:id/rates` · Guard `rate`

The clearest demonstration of the permission model in the product.

AS SUPER ADMIN	AS HOTEL MANAGER
No banner	"Read-only for your role" banner
12 Edit buttons	12 Locked markers

Both verified in the browser (Volume XIII).

The edit dialog states the consequence:

Changing a rate affects new bookings only. Existing reservations keep the rate they were quoted, and the change is written to the audit trail.

Footnote explains the GST banding — 12% below ₹7,500, 18% at or above — because a rate near the boundary has a tax consequence the person setting it should know about.

10.20 Invoices

Route `/finance/invoices` · Guard `invoice`

Totals strip: invoices · collected · outstanding · overdue.

Overdue rows render the due date and amount in `brand-red` — the row is legible as a problem without reading the status column.

10.21 Invoice detail

Route `/finance/invoices/:id`

A real document, laid out to print:

Fidato Hotels	(Georgia)	Tax Invoice
Fidato Hospitality Pvt Ltd		INV-2607-0193
Registered address		Issued 2 Jul
GSTIN 27AABCF1234M1Z5		Due 17 Jul

Billed to	Stay
Meridian Logistics	Ayati Resort & Spa
...address, GSTIN...	Reservation FH...

Description	Qty	Rate	Amount
...			
		Subtotal	₹54,000.00
		GST	₹ 9,720.00
		Total	₹63,720.00
		Paid	-₹45,320.00
		Due	₹18,400.00

`.print-serif` on the header is the surviving Georgia usage. The side rail carries payments and account links and is `.no-print`.

Record payment updates `amountPaid`, `amountDue` and recalculates status to `paid` or `partially_paid`.

10.22 Payments

Route `/finance/payments` · **Guard** `payment`

Receipts against invoices, with a **reconciled** column. Unreconciled means received but not yet matched to a bank statement — a real finance distinction, and the reason the totals strip separates "received" from "unreconciled".

Rows navigate to the invoice, not to a payment detail page: a payment in isolation is rarely what you want.

10.23 Commissions

Route `/finance/commissions` · **Guard** `commission`

What Fidato actually earns. Revenue figures elsewhere are gross booking value; this screen is the one that answers "what did we make?".

Ranked by commission earned, **not** by booking value — a high-volume property on 8% can be worth less than a smaller one on 18%, and ranking by revenue would bury that.

Footnote records the basis: commission is calculated on gross booking value including tax, which is how the current property agreements are written, and cancelled bookings are excluded.

10.24 Reports

Six screens sharing `ReportShell` (breadcrumbs, export, print, and a Georgia print cover).

`/reports` — the gallery

Each card states **the question the report answers**, not just its name:

REPORT	QUESTION
Revenue	Where is the money coming from, and is it growing?
Sales performance	Who is selling, and how well?
Occupancy	How full are we, and where are the gaps?
Property performance	Which properties are earning their place?
Forecast	What does the next six months look like?

A gallery of six nouns makes you open all six to find the one you want.

`/reports/revenue`

Area chart, bookings bar chart, channel-mix donut, and a month table with a totals footer.

/reports/sales-performance

Leaderboard with accounts, bookings, average value, cancellations, conversion and revenue. Revenue cells carry a `ProgressBar` scaled to the top performer, so relative standing is visible without arithmetic.

/reports/occupancy

Where `ADR-19` is most visible. Two adjacent, deliberately distinct metrics:

METRIC	SOURCE	MEANING
Fidato room nights	Reservations	What this platform booked
Property occupancy	Inventory model	How full the properties are, all channels

The footnote explains the distinction rather than leaving it to be misread.

/reports/hotel-performance

Sortable three ways: by revenue, **by revenue per room**, by occupancy.

The per-room sort exists because the real portfolio spans 17 to 236 rooms. Ranking by total revenue systematically flatters scale, and a 17-key property outperforming per room is exactly the insight the report should surface.

/reports/forecast

Six-month straight-line projection from the last six months. Solid line for booked, dashed for projected, with a "Today" reference line.

An info banner sits above the numbers:

This is a straight-line projection from recent trend, not a demand model. It takes no account of seasonality, the events calendar or properties still onboarding. Treat it as a direction, not a number to plan against.

A forecast presented without its limits will be quoted as if it had none.

10.25 Automation

/automation

Workflow cards showing the trigger → steps chain as a readable sequence:

 Reservation Confirmed →  generate_pdf →  send_email →  notify_user +2

A Switch pauses/resumes for roles with `edit`; others see a status pill.

Footer states plainly that these do not execute in Phase 1 and that the run history is simulated.

/automation/:id

Trigger (with detail), conditions, then the ordered steps as a vertical timeline. Each step names the **n8n node** it will map to — the definition here *is* the Phase 3 specification.

The side rail shows the webhook path Phase 3 will POST to:

POST /webhook/reservation-confirmed

/automation/runs

Full run log with result, workflow, record, duration, steps completed and failure reason.

10.26 Notifications

/notifications

Inbox, all/unread. Unread rows carry a tinted background; approval and payment categories get a red icon treatment because those are the ones people act on.

`/notifications/templates`

Templates per channel, in two modes:

- **Preview** — merge fields substituted from a sample booking, so you read the real wording.
- **Merge fields** — the raw template with `{{tokens}}` visible.

WhatsApp templates render in a green-tinted frame; SMS templates show a character count and segment count, because a template that silently costs three segments is a template nobody budgeted for.

Unknown tokens are left intact rather than blanked, so a typo is visible instead of producing an empty sentence.

10.27 Assistant

Route `/ai` · Guard `ai`

Answers are **computed from live seed data**, not generated (**ADR-22**). Verified output:

Across the portfolio you are holding 932 live reservations worth ₹5,94,79,298. • Room nights sold: 6,655 • Average booking value: ₹63,819 • Cancellation rate: 12.3% • Properties live: 29 of 32

Every figure reconciles with `/reports/revenue`, because both read the same store.

Each response carries a footnote:

Generated from live platform data. Phase 1 uses scripted analysis, not a language model.

Contract for Phase 2: `answerFor(question): ChatTurn` becomes async against a real completion. `AiPage.tsx` does not change shape.

10.28 Administration

`/admin/users`

24 users, filterable by role and status. A footnote reminds the reviewer that there is no login in Phase 1 and points at the role switcher.

`/admin/roles`

Generated from `permissions.ts` and `rules.ts`. Role cards with user counts and permission counts, the 20 × 8 matrix, and the eight business rules with rationale and enforcing function.

Matrix cells encode three levels — ● can change, R read-only, · no access — with a tooltip listing the exact actions granted.

`/admin/integrations`

14 integrations grouped by category, with connected / available / error status and an `n8n` badge on the ones Phase 3 will route through.

`/admin/audit-log`

Append-only, newest first. Rows link to their record where one exists.

Footnote states the property that makes it worth keeping:

Because reservations are cancelled rather than deleted, the history of any booking can always be reconstructed from these entries.

`/admin/settings`

Organisation details used on invoices, vouchers and outbound messages. Read-only below Super Admin, with inputs visibly disabled and a note explaining why.

A side card lists the platform rules that are **fixed in Phase 1** — approval threshold, GST bands, no reservation deletion, central rate ownership — and states that they become configurable in Phase 2 once they live in Firestore rather than in the bundle.

10.29 Design system

Route `/design-system`

The living style guide. Every token and component rendered from `src/components/ui` — the same modules the product uses, so it cannot drift.

Sections: brand mark · colour (including both documented additions, with their rationale on-screen) · typography and the scale · buttons · status pills · form controls · data display · overlays and feedback · tabs · empty/loading/error states · motion and spacing.

Why ship it in the product rather than as a separate Storybook. It uses the real modules in the real bundle. A separate tool would need its own build, its own deployment, and would drift the first time someone changed a component without opening it.

VOLUME XI

Diagnostics

Organised by **symptom**, because when something is wrong you do not yet know which part it is.

Each entry gives what you observe, the likely causes in order of probability, how to confirm each, and the fix.

11.0 First moves

Before anything else, three checks that resolve a surprising share of problems:

#	CHECK	COMMAND / ACTION
1	Is it a stale build?	Hard reload — <code>Ctrl+Shift+R</code>
2	Does it typecheck?	<code>npx tsc --noEmit -p tsconfig.app.json</code>
3	Is the console clean?	DevTools → Console, filter to Errors

```
npx tsc --noEmit -p tsconfig.app.json
```

⚠️ **Remember the reset.** Data resets on every refresh ([ADR-08](#)). If you created a booking and it "disappeared", that is the design, not a defect.

11.1 The whole application is unclickable

Symptom. Nothing responds. Buttons do nothing, menus do not open, links do not navigate. No console error. The page looks completely normal.

This is the highest-priority entry in this volume because it presents as total failure with no diagnostic trace.

Confirm

```
// DevTools console
getComputedStyle(document.body).pointerEvents
// → "none" ← confirmed
document.body.getAttribute("data-scroll-locked")
// → "1"
```

Cause

Radix applies `pointer-events: none` to `<body>` while a modal is open, and removes it on close. If a modal **unmounts while still open** — which happens when a route change is triggered from inside one — the cleanup never runs and the style is stranded.

Immediate recovery

```
document.body.style.removeProperty("pointer-events");
document.body.removeAttribute("data-scroll-locked");
```

Or simply reload.

Permanent fixes, both already in place

1 — Close before navigating. Any modal that navigates must close first:

```
function choose(entry: Entry) {
  setOpen(false); // ← first
  navigate(entry.to);
}
```

2 — The route-change guard in `AppShell`:

```
useEffect(() => {
  if (document.querySelector("[data-radix-focus-guard]")) return; // a modal IS open - leave it
  if (document.body.style.pointerEvents === "none") {
    document.body.style.removeProperty("pointer-events");
  }
  document.body.removeAttribute("data-scroll-locked");
}, [pathname]);
```

If you add a new modal

Any component that both renders a Radix overlay **and** can trigger navigation must close before navigating. Grep for the pattern when adding one:

```
grep -rn "navigate(" src/components/app src/features --include=*.tsx | grep -i "dialog\|modal\|palette\|drawer"
```

See [D-02](#).

11.2 A screen renders blank, with a `<Slot.Slot>` warning

Symptom. White page. Console shows:

An error occurred in the `<Slot.Slot>` component.

Cause

Radix `Slot` accepts **exactly one child**. Something passed more than one — almost always a `Button` with `asChild` plus icons.

Confirm

```
grep -rn "asChild" src --include=*.tsx | head -40
```

Look for a `Button asChild` whose implementation places `leadingIcon`, `children` and `trailingIcon` as siblings.

Fix

Fold the icons *inside* the single child — the pattern already in `Button.tsx`:

```
const child = Children.only(children) as ReactElement<{ children?: ReactNode }>;
const merged = cloneElement(child, undefined, leading, child.props.children, trailingIcon);
return <Slot ref={ref} className={styles} {...props}>{merged}</Slot>;
```

⚠️ Also do not forward `disabled` through `Slot` onto an anchor — it is not a valid attribute there.

See [D-01](#).

11.3 A KPI shows an absurd percentage

Symptom. `+757.4% vs last month`, or a count far larger than the period should contain.

Cause, in order of likelihood

1 — A date range missing its upper bound. The most common. An open-ended range over data that extends into the future silently includes the entire forward book.

```
const thisMonth = scoped.filter(r => r.checkIn >= monthStart);           // X
const thisMonth = scoped.filter(r => r.checkIn >= monthStart
                                && r.checkIn < nextMonthStart);         // ✓
```

2 — A near-zero denominator. `(this - last) / last` explodes when `last` is small. Guard:

```
revenueChangePercent: revenueLast > 0 ? ((revenueThis - revenueLast) / revenueLast) * 100 : 0,
```

3 — Cancelled or draft records included. Check whether the aggregate should exclude them.

Confirm

```
// In the console, on the dashboard
const rows = window.__db?.reservations ?? [];
rows.filter(r => r.checkIn >= "2026-07-01" && r.checkIn < "2026-08-01").length
```

If that count and the KPI disagree, the filter is the problem.

Rule to apply

Every date-range filter needs both bounds. Search for one-sided comparisons:

```
grep -rn "checkIn >=\\|issueDate >=\\|at >=" src/data/repositories --include=*.ts
```

See [D-03](#).

11.4 Occupancy looks wrong ($\approx 1\%$, or $\approx 100\%$)

Symptom. Occupancy reads as a fraction of a percent, or pins at 100%.

If it reads $\approx 1\%$

The metric is being computed from **reservations** rather than from the inventory model.

```
// X Wrong metric. Fidato books a slice of each partner hotel.
occupancyPercent: (roomNights / (totalRooms * days)) * 100

// J
occupancyPercent: occupancyForHotel(hotelId)
```

This is not an arithmetic bug — it is the wrong denominator. See [ADR-19](#) and [D-04](#).

If it reads $\approx 100\%$

`buildInventory`'s pressure constants have been raised, or `blocked` is being double-counted. Expected range is **55–82%**:

```
const pressure = dow === 5 || dow === 6 ? 0.82 : 0.55;
const booked = Math.min(rt.totalRooms, Math.round(rt.totalRooms * pressure * (0.5 + local.next())));
```

If a stale value persists after changing the generator

The memo cache is holding it:

```
const occupancyCache = new Map<string, number>();
```

It is module-level and never invalidated — correct for Phase 1 because the inventory model is deterministic and immutable. 🛠️ If Phase 2 makes inventory mutable, this cache **must** be invalidated on write or it will serve stale occupancy indefinitely.

11.5 A detail page errors instead of showing Not-found

Symptom. Console:

Query data cannot be undefined. Please make sure to return a value other than undefined from your query function. Affected query key: ["hotel", "..."]

Cause

A `get`-style repository method returned `undefined` for a missing document. TanStack Query treats `undefined` as a contract violation.

Fix

Coalesce to `null`:

```
get: (id: string): Promise<Hotel | null> =>
  read() => db.hotels.find((h) => h.id === id) ?? null,
```

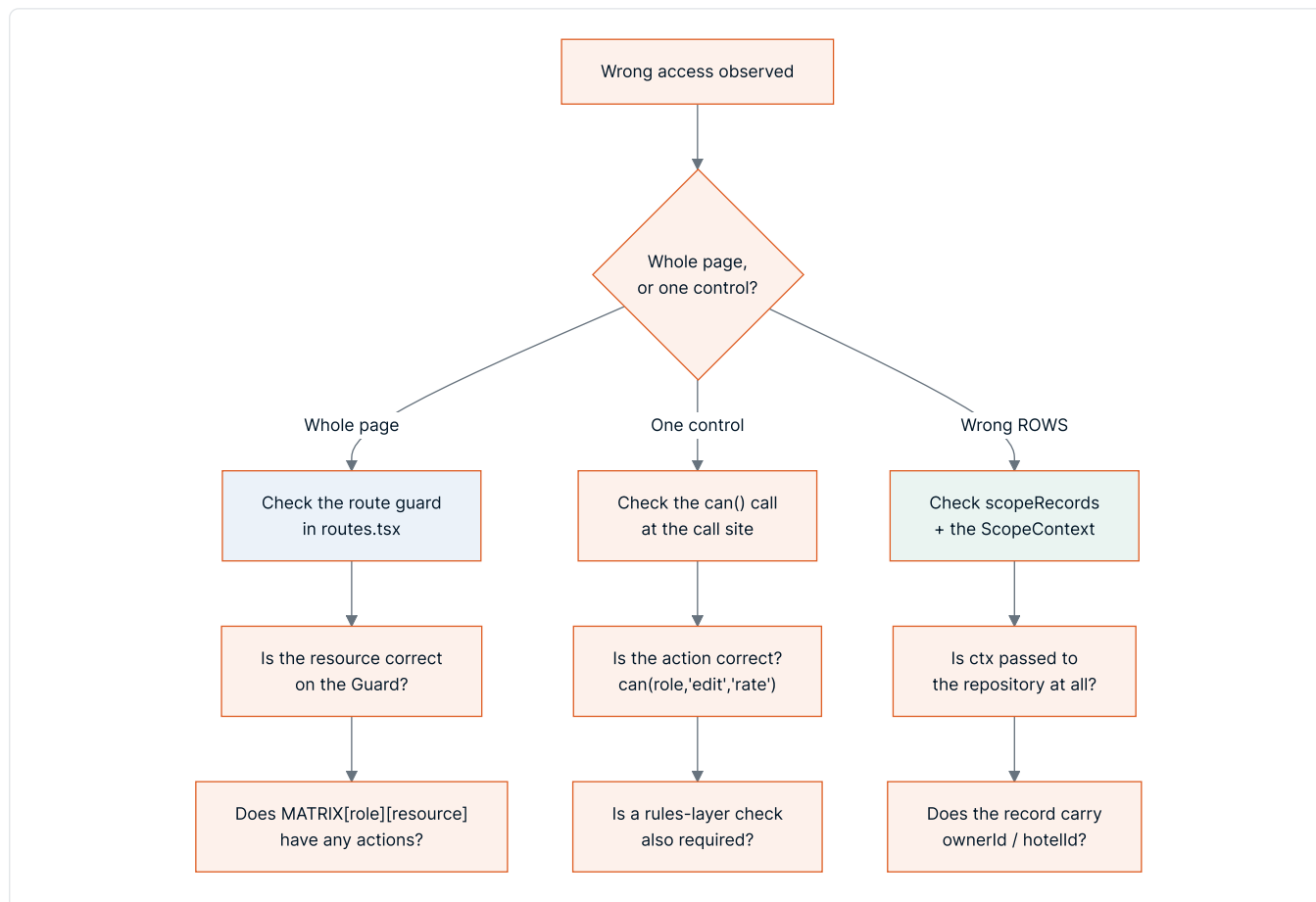
All eight `get`-style methods were converted. Check any new one:

```
grep -n "read() => db\...\find(" src/data/repositories/mock/index.ts
```

Every match must end `?? null`. See [D-08](#).

11.6 A role sees something it should not (or cannot see something it should)

Decision tree



Confirm quickly

Open `/admin/roles` — it is generated from the live matrix. If the cell there is wrong, the matrix is wrong. If the cell is right but the screen misbehaves, the call site is wrong.

Common causes

SYMPTOM	LIKELY CAUSE
Nav item missing	<code>canAccess</code> <code>false</code> — the role has no actions on that resource
Page loads but is empty	Guard passed, but <code>scopeRecords</code> filtered everything out
Button missing	<code>can(role, action, resource)</code> <code>false</code>
Button present but should not be	Wrong action passed at the call site — e.g. <code>"view"</code> where <code>"edit"</code> was meant
Rows from another owner visible	<code>ctx</code> not passed to the repository call
Hotel manager sees all properties	<code>hotelId</code> missing on the seeded user

⚠ **The permissive default in scoping.** A record with `no ownerId` is visible to everyone:

```
records.filter((r) => !r.ownerId || r.ownerId === ctx.userId);
```

Intentional — an unassigned lead should not be invisible to the whole sales team. But if a new collection uses `ownerId` with different semantics, this default will surprise you.

11.7 Filters or sort do not survive a refresh

Symptom. Filtering, then reloading, returns to an unfiltered list.

Cause

The screen is holding list state in `useState` instead of `useListState`.

Fix

```
const list = useListState({
  filterKeys: ["status", "channel"],
  defaultSortBy: "checkIn",
  defaultSortDir: "desc",
});
```

and pass `list.query` into the query key **and** the repository call:

```
useQuery({
  queryKey: ["reservations", list.query, scope.role, scope.userId],
  queryFn: () => reservationsRepo.list(list.query, scope),
});
```

⚠️ **filterKeys must be declared.** A filter key not in the array is never read back from the URL, so it appears to work until you reload.

11.8 A list shows "no records exist" when records do exist

Symptom. A filtered table shows the empty state offering "Create the first...", when the dataset is simply filtered.

Cause

`hasFilters` is not being passed to `DataTable`, so it cannot distinguish empty from no-results.

Fix

```
<DataTable
  hasFilters={list.hasFilters}
  onClearFilters={list.clear}
  empty={EmptyState ... } />
```

`DataTable` then chooses:

```
if (!rows.length) {
  return hasFilters ? <NoResultsState onClear={onClearFilters} /> : empty;
}
```

This distinction matters — see Volume II §2.8.

11.9 Numbers jitter between rows

Symptom. A column of money or dates does not line up; digits appear to shift.

Cause

Missing tabular figures.

Fix — three ways, in order of preference

1. Mark the column `numeric: true` in the `Column` definition — this applies `data-numeric`, which the global CSS rule picks up.
2. Add `className="tabular"` to the cell.
3. For a `<Stat>` or custom markup, add `tabular` directly.

```
# Find money rendered without tabular treatment
grep -rn "money(" src/features --include=*.tsx | grep -v "tabular\|numeric" | head -20
```

11.10 A chart is blank or mis-scaled

SYMPTOM	CAUSE	FIX
Blank, no error	Container has no height. Recharts <code>ResponsiveContainer</code> needs a fixed parent height	Wrap in <code><div className="h-[260px]"></code>
Axis labels overlap	Too many categories	Add <code>angle={-25} textAnchor="end" height={60} interval={0}</code>
Y-axis unreadable	Raw rupee values	<code>tickFormatter={v => moneyCompact(v)}</code>
Tooltip unstyled	Default Recharts tooltip	Use the shared <code><ChartTooltip /></code> from <code>DashboardPage</code>
Line breaks at the join	<code>connectNulls</code> not set on the projected series	<code>connectNulls</code> on the forecast line only

11.11 The dev server will not start

ERROR	CAUSE	FIX
<code>EADDRINUSE :5173</code>	Port taken	<code>npm run dev -- --port 5174</code>
<code>UNABLE_TO_VERIFY_LEAF_SIGNATURE</code>	This machine's TLS-inspecting proxy	<code>win-ca-bundle.pem</code> + <code>.npmrc</code> are present; run <code>npm install</code> from PowerShell , not Git Bash
<code>ENOENT spawn cmd.exe</code>	npm lifecycle scripts run from Git Bash	Use PowerShell
<code>'C:\Program' is not recognized</code>	A launcher not quoting the Node path	Run <code>npm run dev</code> directly rather than through the launcher
Alias <code>@/...</code> unresolved	<code>vite.config.ts</code> or <code>tsconfig</code> alias missing	Both must declare <code>@</code> → <code>./src</code>

11.12 The build fails but the dev server works

Vite's dev server is more forgiving than the production build.

CAUSE	DETECTION
Unused import	<code>noUnusedLocals</code> — build only
Type error in a path dev never rendered	<code>npx tsc --noEmit</code>
Case-mismatched import	Windows is case-insensitive; the build is not
Circular import	Manifests as <code>undefined</code> at module init

Always run the typecheck before the build — it gives better messages:

```
npx tsc --noEmit -p tsconfig.app.json
```

11.13 A mutation succeeds but the screen does not update

Symptom. Approve a reservation; the toast fires, but the queue still shows it.

Cause

Missing or too-narrow query invalidation.

Fix

Invalidate **every** key the write touches, by prefix:

```
onSuccess: () => {
  queryClient.invalidateQueries({ queryKey: ["reservation", id] });
  queryClient.invalidateQueries({ queryKey: ["reservation-audit", id] });
  queryClient.invalidateQueries({ queryKey: ["reservations"] }); // prefix - all filters/pages
  queryClient.invalidateQueries({ queryKey: ["pending-approvals"] });
  queryClient.invalidateQueries({ queryKey: ["kpis"] });
},
```

⚠ Invalidate by **prefix**, not exact key. `["reservations", query, role, userId]` produces a different key per filter combination and page; only the prefix catches them all.

Checklist for a new mutation

QUESTION
Which detail queries show this record?
Which lists contain it?
Does it change a dashboard KPI?
Does it change a count in the nav or top bar?
Does it write an audit entry a timeline shows?

11.14 Everything feels slow

CAUSE	CONFIRM	FIX
Sequential queries	Network tab shows a staircase	Independent queries should mount together, not chain
Missing <code>useMemo</code> on a heavy derive	React DevTools Profiler	Memoise the grouping/packing
Occupancy recomputed per row	Long report render	The <code>occupancyCache</code> should be doing this
Too many rows	<code>pageSize</code> raised above 25	Return to 25, or virtualise

Remember the deliberate 120–400 ms per repository call. A screen firing six sequential calls takes over a second **by design** — that is the simulation telling you the call graph is wrong, which is exactly what [ADR-07](#) is for.

11.15 A date is off by one day

Symptom. A stay shows one night too many, or a check-out appears occupied.

Cause A — inclusive interval

A stay occupies nights from check-in **up to but not including** check-out:

```
eachDayOfInterval({ start, end: addDays(end, -1) }) // ✓
eachDayOfInterval({ start, end })                  // ✗ counts the check-out night
```

Cause B — timezone

`new Date("2026-07-28")` parses as **UTC midnight**. In a negative-offset timezone that renders as the 27th.

All formatting goes through `src/lib/format.ts`, and the platform declares Asia/Kolkata in `orgSettings`. 🛠 Revisit if the platform ever serves multiple timezones.

11.16 A merge loses data or orphans records

Symptom. After a merge, reservations show a stale customer name, or disappear.

Cause

The merge sequence is order-dependent:

1. Re-point children (`reservations`, `invoices`)
2. Fill gaps on the survivor

- 3. Roll up totals
- 4. Remove absorbed records

Removing before re-pointing orphans the children.

⚠ **The denormalised name must be updated alongside the id.** Updating `customerId` but not `customerName` leaves the old name rendering in every list, with nothing to indicate it is stale.

```
if (absorbedIds.includes(r.customerId)) {
  r.customerId = survivorId;
  r.customerName = survivor.fullName; // ← must not be forgotten
}
```

11.17 Automated browser testing does not work

Symptom. Clicks land on the right element but nothing happens. Escape does not close dialogs. Menus do not open.

This is an environment limitation, not an application defect

Confirmed during verification (Volume XIII §13.4). The evidence:

```
// React's handler IS attached
const b = [...document.querySelectorAll("button")].find(x => x.textContent.trim() === "Open dialog");
Object.keys(b).find(k => k.startsWith("__reactProps"));
// → "__reactProps$..." - onClick present

// Invoking it directly works
b[propsKey].onClick({ type: "click", currentTarget: b, target: b,
  preventDefault(){}, stopPropagation(){}} });
// → dialog opens, aria-expanded = "true"
```

The component is correct; the harness is not delivering trusted input events to the React root.

Diagnostic distinction

TEST	IF IT WORKS	IF IT FAILS
Direct handler invocation	Component is fine — harness issue	Genuine component bug
<code>element.click()</code> on a Radix menu	—	Expected: menus open on <code>pointerdown</code> , not <code>click</code>
<code>element.click()</code> on a plain React button	React delegation is working	React root not receiving events

What to do

Verify interactive flows **in a real browser by hand**. Use the automated harness for structure, routing, data and rendering — which it does reliably.

11.18 Quick reference

SYMPTOM	SECTION	FIRST THING TO CHECK
Nothing clickable	11.1	<code>document.body.style.pointerEvents</code>
Blank screen, Slot warning	11.2	<code>Button asChild</code> usage
Absurd percentage	11.3	Date range upper bound
Occupancy ≈1%	11.4	Wrong denominator
Detail page errors	11.5	<code>?? null on get</code>
Wrong access	11.6	<code>/admin/roles</code>
Filters lost on refresh	11.7	<code>useListState</code> + <code>filterKeys</code>
Wrong empty state	11.8	<code>hasFilters</code> prop
Numbers jitter	11.9	<code>numeric: true</code>
Blank chart	11.10	Parent height

SYMPTOM	SECTION	FIRST THING TO CHECK
Stale screen after write	11.13	Invalidation set
Date off by one	11.15	Inclusive interval

VOLUME XII

Defect log

Ten defects found and fixed during the Phase 1 build. Each is recorded with what was observed, what caused it, why it was not caught earlier, and what changed.

This volume exists because **the defects are the most instructive part of the build**. Four of them would have shipped looking like working software.

#	DEFECT	SEVERITY	CLASS
D-01	Radix <code>Slot</code> crash blanked the entire app	Critical	Integration
D-02	Stranded scroll lock made the app unclickable	Critical	Integration
D-03	Unbounded date range reported +757% growth	High	Silent
D-04	Occupancy computed from the wrong denominator	High	Silent
D-05	Seed too small — "today" looked dead	Medium	Silent
D-06	Reservations created in the future	Medium	Silent
D-07	Two icon-only buttons had no accessible name	Medium	Accessibility
D-08	<code>undefined</code> from missing documents broke React Query	Medium	Contract
D-09	Unsubstituted merge token in template preview	Low	Data
D-10	Two primary buttons on the same view	Low	Design system

Silent = produced no error, no warning, no crash. The application appeared to work and reported a wrong answer confidently. These are the dangerous ones.

D-01

Radix `Slot` crash blanked the entire application

Severity Critical · **Found** first browser load · **Class** Integration

Observed

Completely white page. `#root` had zero children. Console showed only a warning:

An error occurred in the `<Slot.Slot>` component. Consider adding an error boundary...

No stack trace pointing at application code.

Cause

`Button` with `asChild` passed three children to Radix `Slot`, which accepts exactly one:

```
// X
const Comp = asChild ? Slot : "button";
return (
  <Comp ...>
    {loading ? <Loader2 /> : leadingIcon}
    {children}
    {!!loading && trailingIcon}
  </Comp>
);
```

`Button` `asChild` is used in the sidebar, top bar and most page headers — so the crash occurred during the shell's own render, taking the whole tree with it.

Fix

Fold the icons *inside* the single child:

```

if (asChild) {
  const child = Children.only(children) as ReactElement<{ children?: ReactNode }>;
  const merged = isValidElement(child)
    ? cloneElement(child, undefined, leading, child.props.children, !loading && trailingIcon)
    : child;
  return <Slot ref={ref} className={styles} {...props}>{merged}</Slot>;
}

return (
  <button ref={ref} disabled={disabled || loading} className={styles} {...props}>
    {leading}{children}{!loading && trailingIcon}
  </button>
);

```

`disabled` is deliberately not forwarded in the `asChild` branch — it is not a valid anchor attribute.

Why it was not caught earlier

TypeScript cannot express "exactly one child" for `Slot`. The code typechecked and built cleanly. **Only rendering the page could reveal it** — which is the strongest argument in this whole log for browser verification not being optional.

Prevented in future by

Volume V §5.2 documents the pattern; Volume XI §11.2 gives the symptom-to-fix path.

D-02

Stranded scroll lock made the entire app unclickable

Severity Critical · **Found** during interactive verification · **Class** Integration

Observed

Every control stopped responding. No console error. The page looked entirely normal — this is what made it alarming.

Diagnosis

```

getComputedStyle(document.body).pointerEvents    // → "none"
document.body.getAttribute("data-scroll-locked")  // → "1"

```

Cause

Radix applies `pointer-events: none` to `<body>` while a modal is open and removes it on close. A modal that **unmounts while still open** never runs the cleanup, and the style is stranded.

The trigger in this codebase: the command palette navigated *before* closing, so the route change unmounted the palette while `open` was still true.

Fix — two layers

1 — Close before navigating:

```

function choose(entry: Entry) {
  // Close before navigating. Unmounting the route while the modal is
  // still open can strand Radix's scroll lock on <body>, which leaves
  // the whole app pointer-dead until a full reload.
  setOpen(false);
  navigate(entry.to);
}

```

2 — A route-change guard in `AppShell`, which checks for a live focus guard first so it never interferes with a legitimately open modal:

```

useEffect(() => {
  if (document.querySelector("[data-radix-focus-guard]")) return;
  if (document.body.style.pointerEvents === "none") {
    document.body.style.removeProperty("pointer-events");
  }
  document.body.removeAttribute("data-scroll-locked");
}, [pathname]);

```

Why it was not caught earlier

It requires a specific sequence — open a modal, navigate from inside it — that no typecheck or build can exercise. It also produces **no error of any kind**, so nothing in the console points at it.

The general lesson

A library that mutates global DOM state on mount must be assumed to leak that state on abnormal unmount. Where the leaked state is invisible and catastrophic, a defensive reset is justified.

D-03

Unbounded date range reported +757% growth

Severity High · Class Silent · The most instructive defect in this log

Observed

Dashboard, on first render with real data:

REVENUE THIS MONTH	₹59.3L	+757.4% vs last month
RESERVATIONS	105	+400.0% vs last month

105 reservations in one month, from a dataset of 1,100 spread over 480 days.

Cause

```
const thisMonth = scoped.filter((r) => r.checkIn >= monthStart); // X no upper bound
```

The seed extends **150 days into the future**. "This month" therefore meant *this month and the entire forward book* — five months of bookings compared against one month.

Fix

```
// Upper bound matters: without it "this month" silently absorbs the
// entire forward book and every growth figure becomes nonsense.
const nextMonthStart = isoDate(startOfMonth(addMonths(TODAY, 1)));

const thisMonth = scoped.filter(
  (r) => r.checkIn >= monthStart && r.checkIn < nextMonthStart,
);
```

After the fix: **₹5.6L, -19.1%, 19 reservations** — believable figures.

Why it was not caught earlier

Nothing threw. Nothing warned. The types were correct. The code read naturally — `checkIn >= monthStart` looks like "this month" to a reader.

It was caught **only because the number was implausible on sight**. A subtler version — say +40% — would have shipped.

The lesson

Every date-range filter needs **both** bounds. An open-ended range over data that extends into the future is a bug that reports success.

This class of defect is why the dashboard was reviewed with real data before being called done. A screen built against three fixture rows would never have exposed it.

D-04

Occupancy computed from the wrong denominator

Severity High · Class Silent · Produced **ADR-19**

Observed

OCCUPANCY 1%

Across a 1,969-room portfolio with 1,100 reservations.

First diagnosis — wrong

The initial reading was "the seed is too small". That would have led to inflating the dataset until the number looked acceptable, which is data falsification dressed as a bug fix.

Actual cause

The **metric** was wrong, not the data.

```
occupancyPercent: (roomNights / (totalRooms * 30)) * 100
```

Fidato books a *slice* of each partner hotel; other channels sell the rest. Dividing Fidato's bookings by the hotel's entire inventory answers a question nobody asked.

The arithmetic to make it "look right" is instructive: 1,969 rooms × 450 days × 60% ≈ 531,000 room-nights, which at ~~6 room-nights per reservation~~ ~~needs **88,000 reservations**~~. Not generatable client-side — and generating them would misrepresent the business.

Fix

Occupancy now reads from the inventory model, which represents each property's true position across all channels:

```
function occupancyForHotel(hotelId: string, days = 30): number {
  const rows = buildInventory(hotelId, days);
  const capacity = rows.reduce((s, r) => s + r.totalRooms, 0);
  const booked = rows.reduce((s, r) => s + r.booked, 0);
  return capacity > 0 ? (booked / capacity) * 100 : 0;
}
```

The Fidato-booked portion is surfaced separately and labelled "**Fidato room nights**", and the Occupancy report explains the distinction on the page.

Portfolio occupancy is weighted by room count — a flat average across a 17-key retreat and a 236-key hotel would describe nothing.

After the fix: **58–66% by city, 61% portfolio**.

Why it was not caught earlier

The formula is the textbook definition of occupancy. It is correct *for a hotel*. It is wrong *for a booking agent*, and nothing in the code says which one this system is.

The lesson

When a metric reads absurdly, check whether the **denominator** describes something you control before assuming the numerator is too small.

D-05

Seed too small — "today" looked dead

Severity Medium · **Class** Silent

Observed

```
Arrivals    0
Departures  1
In house    0
```

on a 32-property portfolio.

Cause

320 reservations spread across 480 days is **0.67 arrivals per day** across the whole portfolio. An empty day was statistically normal — and a misrepresentation of the business.

Fix

```
// Spread across 11 months back and 5 months forward. At ~1,100
// reservations that is roughly three arrivals a day across the
// portfolio, so any given day has something happening on it -
// which is what a 32-property book actually looks like.
const offset = rng.int(-330, 150);
...
export const reservations: Reservation[] = Array.from({ length: 1100 }, (_, i) => buildReservation(i));
```

After: **2 arrivals, 4 in house** on the demo date.

Was this falsifying data?

No — and the distinction matters. D-04 rejected inflating the seed because it would have made a *metric* read correctly by misrepresenting the business. Here the under-seeding was itself the misrepresentation: a real 32-property book does not see 0.67 arrivals a day. Raising the volume made the simulation **more** accurate, not less.

The lesson

Seed volume is a design parameter, not an arbitrary number. Choose it from the real-world rate the data represents, and check the screens that depend on daily density.

D-06

Reservations created in the future

Severity Medium · Class Silent

Observed

On the approvals queue:

Meera Bose · **in about 1 month**

A booking cannot have been created after today.

Cause

```
const created = subDays(checkIn, rng.int(3, 60)); // X
```

For a check-in in October 2026, this produced a `createdAt` in August or September — beyond the demo date of 28 July.

Fix

```
// Booked some weeks before arrival - but never in the future. A
// forward booking was necessarily raised on or before today.
const leadTime = subDays(checkIn, rng.int(3, 60));
const created = leadTime > TODAY ? subDays(TODAY, rng.int(0, 21)) : leadTime;
```

Verified after the fix — every entry now reads *"14 days ago"*, *"6 days ago"*, and no future timestamps remain.

Why it was not caught earlier

`relative()` rendered the value perfectly correctly. The formatting was right; the data violated a domain invariant nothing was checking.

The lesson

Derived dates need clamping against the invariants of the domain. **Creation precedes existence** is one that no type system will enforce for you.

D-07

Two icon-only buttons had no accessible name

Severity Medium · **Class** Accessibility

Observed

In the accessibility tree:

```
button [ref_15] type="button"    ← no name
button [ref_19] type="button"    ← no name
```

Cause

Both the global search and the role switcher hide their text label below the `sm` breakpoint:

```
<span className="hidden sm:block text-base truncate">Search or jump to...</span>
```

Below `sm` they became unnamed buttons. Above `sm` the accessible name came from visible text that a narrow viewport removes.

Fix

```
// Collapses to an icon below `sm`, so it needs its own name.
aria-label="Search or jump to a record"
```

```
// The name and role are hidden below `sm`, so the button would
// otherwise be an unlabelled avatar to a screen reader.
aria-label={`Viewing as ${ROLE_LABELS[role]} (${user.name})`. Change role`}
```

Verified: both now report full names in the accessibility tree.

The lesson

A control whose label is **responsive** needs an explicit `aria-label`. Checking accessibility at desktop width alone misses this entire class of defect.

D-08

undefined from missing documents broke React Query

Severity Medium · **Class** Contract

Observed

```
[error] Query data cannot be undefined. Please make sure to return a value other than
        undefined from your query function. Affected query key: ["hotel","htl-agra-taj-pearl"]
```

Cause

```
get: (id: string): Promise<Hotel | undefined> =>
  read(() => db.hotels.find((h) => h.id === id)), // X undefined when not found
```

TanStack Query treats `undefined` as a contract violation, so a perfectly normal condition — a mistyped or deleted id — put the query into an error state and logged an error.

The pages still rendered Not-found (because `!data` is true either way), so the user-visible behaviour was *nearly* correct — which is precisely why it could have been ignored.

Fix

All eight `get`-style methods converted to return `null`:

```
get: (id: string): Promise<Hotel | null> =>
  read(() => db.hotels.find((h) => h.id === id) ?? null),
```

Verified: `/hotels/does-not-exist` now renders Not-found cleanly with no console error.

The lesson

`null` means “looked, found nothing”. `undefined` means “did not look”. Data layers should return the first. The distinction is not pedantry — one library's contract depends on it.

D-09

Unsubstituted merge token in template preview

Severity Low · Class Data

Observed

On `/notifications/templates` in Preview mode, `{{number}}` rendered literally instead of being substituted.

Cause

The sample map covered `{{invoice_number}}` but not `{{number}}`, which the seeded invoice templates use.

Fix

```
"{{invoice_number}}": "INV-2607-0193",
"{{number}}": "INV-2607-0193",
```

Note

The design *intends* unknown tokens to render literally rather than blank, so a typo is visible rather than producing an empty sentence. This was a gap in the sample map, not a failure of that behaviour — but a token showing raw in the default Preview mode reads as a defect to a reviewer, which is reason enough to fix it.

D-10

Two primary buttons on the same view

Severity Low · Class Design system · Found on the first full-resolution screenshot

Observed

On `/dashboard` and `/reservations`, two identical orange **New reservation** buttons appeared roughly 100 px apart vertically — one in the top bar, one in the page header.

Cause

`TopBar` rendered a global create action, and every screen where creating a reservation is the natural next step *also* offered it in its page header, which is the conventional place for page actions.

Each was individually reasonable. Together they broke the rule the design system states about itself:

`primary` — **One per view**. The single most likely next action. — Volume V §5.2

Fix

The top-bar button was removed entirely. The page header keeps it, sitting beside that screen's other actions (Calendar, Export), and ⌘K covers the global path.

An intermediate fix — hiding the top-bar button above the `sm` breakpoint so it only appeared on mobile — was tried and rejected: the page header's action row stays visible at mobile width, so the duplication simply moved rather than disappearing. Verified in the accessibility tree at 375 px before and after.

Why it was not caught earlier

Every prior verification pass read the **accessibility tree and page text**, where the two buttons appear as two ordinary list entries. Nothing about that output looks wrong. The defect is purely visual — it needs a rendered screenshot at real resolution, which is what finally exposed it.

The lesson

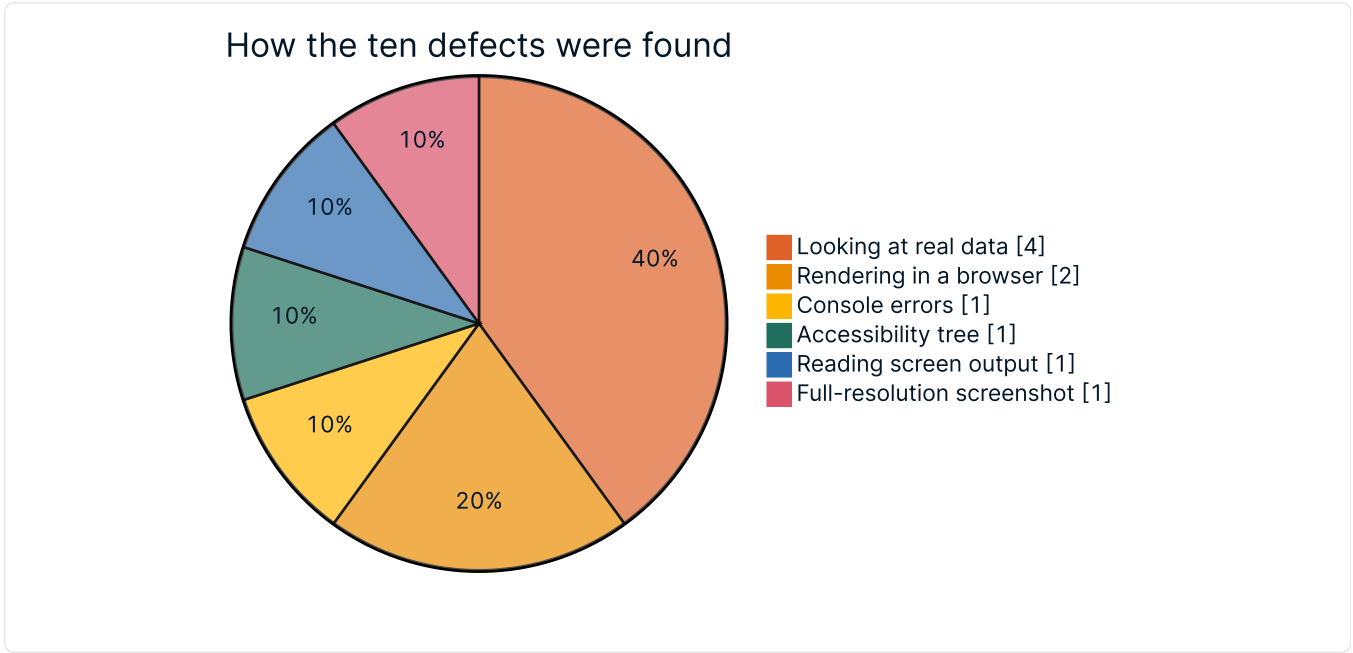
Structural verification cannot see visual duplication. A tree containing two "New reservation" links reads as completely normal; a screenshot of two identical primary buttons does not.

Analysis

By class

CLASS	COUNT	CAUGHT BY
Silent (wrong, but no error)	4	Looking at real data and finding it implausible
Integration (library contract)	2	Rendering the page
Accessibility	1	Reading the accessibility tree
Contract (API expectation)	1	Console error
Data	1	Reading the screen
Design system	1	A full-resolution screenshot

What caught what



Zero were caught by TypeScript or by the production build. Both ran clean throughout.

The uncomfortable conclusion

The four silent defects — D-03, D-04, D-05, D-06 — all produced software that *worked*. It rendered, it did not throw, it typechecked, it built. It simply reported wrong answers with complete confidence.

None would have been found by unit tests written against the same wrong assumptions. All four were found by **generating realistic data and looking at the result critically**:

- +757% is not a growth rate.
- 1% is not an occupancy.
- 0 arrivals is not a portfolio.
- "in about 1 month" is not a creation date.

This is the strongest available argument for [ADR-08](#) (realistic seeded data) and [ADR-09](#) (real property data). A build reviewed against three fixture rows would have shipped all four.

What would have caught them earlier

DEFECT	WOULD HAVE BEEN CAUGHT BY
D-01, D-02	Any browser render — i.e. rendering earlier and more often
D-03	A unit test asserting <code>thisMonth.length ≤ 31 days of bookings</code>
D-04	Stating the metric's definition in writing before implementing it
D-05	Deriving seed volume from a stated arrivals-per-day target
D-06	An invariant check: <code>createdAt ≤ TODAY</code> for all seeded records
D-07	An accessibility audit at mobile width
D-08	Reading the TanStack Query contract
D-10	Looking at a rendered screenshot of any screen

 **Recommendation for Phase 2.** Six of the nine would have been caught by a small set of invariant assertions over the seed and the aggregates. That is the highest-value testing available in this codebase and should precede any Firebase work — see [ADR-24](#).

VOLUME XIII

Verification record

What was tested, how, what passed — and, stated plainly, **what was not tested**.

A verification record that only lists passes is a marketing document. The gaps in §13.4 and §13.7 are the part of this volume that matters.

13.1 Summary

CATEGORY	RESULT
TypeScript strict typecheck	✔ Clean
Production build	✔ Clean — 2,959 modules
Route reachability (38)	✔ All resolve
Screen rendering (sampled)	✔ Verified
Role model	✔ Verified across 3 roles
Business rules	⚠ Partially — see §13.3
Interactive flows	✖ Not verified — see §13.4
Accessibility	⚠ Partially — see §13.6

13.2 Automated checks

Typecheck

```
npx tsc --noEmit -p tsconfig.app.json
```

Result: clean, no output.

Configuration: `strict: true`, `noUnusedLocals`, `noUnusedParameters`, `noUncheckedIndexedAccess`, `noFallthroughCasesInSwitch`.

`noUncheckedIndexedAccess` caught a genuine bug during the build — the calendar's interval packing assumed a non-empty array, which would have thrown on a month with no bookings.

Build

```
npm run build
```

```
✓ 2959 modules transformed.
✓ built in 8.95s
```

CHUNK	RAW	GZIPPED
<code>index</code>	390.6 kB	120.4 kB
<code>BarChart</code> (Recharts)	359.1 kB	103.9 kB
<code>ui</code> (Radix + library)	257.6 kB	81.4 kB
<code>schemas</code> (Zod + RHF)	100.4 kB	29.4 kB
Largest screen chunk	37.0 kB	11.3 kB

All 34 feature screens code-split as intended.

Route reachability

All 38 routes fetched — every one returned 200:

```
/dashboard /reservations /reservations/calendar /reservations/approvals
/crm/customers /crm/companies /crm/merge /crm/import /hotels
/finance/invoices /finance/payments /finance/commissions
/reports /reports/revenue /reports/sales-performance
/reports/hotel-performance /reports/forecast
/automation /automation/runs /notifications /notifications/templates
/ai /admin/users /admin/roles /admin/integrations /admin/audit-log
/admin/settings /design-system
```

⚠️ This proves the SPA shell serves; it does not prove each screen renders. §13.3 covers that.

13.3 Screen rendering — verified individually

Each screen below was loaded and its rendered text inspected.

Dashboard — as Sales Manager

```
Portfolio overview · 28 Jul 2026 · Viewing as Sales Manager
REVENUE THIS MONTH ₹33.2L -11.2% vs last month
RESERVATIONS 65 -14.5% vs last month
AWAITING APPROVAL 66 ₹87.3L
CANCELLATION RATE 12.3%
Arrivals 2 · Departures 0 · In house 4
```

✅ Plausible figures, populated day panel, approval queue with real bookings.

Occupancy report

```
TOTAL ROOMS 1,969 (24 cities) · FIDATO ROOM NIGHTS 6,983
PROPERTY OCCUPANCY 61% (all channels, next 30 days) · REVENUE ₹6.26Cr

Goa 192 rooms 770 nights ₹9,234 60% ₹71,10,547
Pune 332 rooms 808 nights ₹5,939 62% ₹47,98,437
...
```

✅ Occupancy 58–66%, the two metrics clearly distinguished (ADR-19).

Rate plans — the permission test

ROLE	BANNER	EDIT BUTTONS	LOCKED MARKERS
Hotel Manager	"Read-only for your role"		12
Super Admin	none	12	0

✅ BR-04 verified end to end.

Permission matrix

```
{ hasMatrix: true, hasRules: true, rows: 20, cols: 9 }
```

✅ 20 resources × 8 roles + label column, with the business rules rendered.

Calendar

```
Calendar · July 2026 · All properties · 110 interactive elements
```

✅ Month grid renders with bookings positioned.

Inventory grid

```
Turtle Beach Resort · next 30 days
ROOM NIGHTS 3,330 (111 rooms × 30 days) · BOOKED 2,017
AVAILABLE 1,295 (18 blocked) · OCCUPANCY 61%
```

✅ Arithmetic consistent: 2,017 + 1,295 + 18 = 3,330.

Duplicate merge

4 groups to review · 8 records involved
Same phone number · High confidence · 2 records share +91 98975 04652

✔ Detection working across all three strategies.

Assistant

Across the portfolio you are holding 932 live reservations worth ₹5,94,79,298.

- Room nights sold: 6,655
- Average booking value: ₹63,819
- Cancellation rate: 12.3%
- Properties live: 29 of 32

✔ Figures reconcile with the dashboard and reports — they read the same store.

Not-found handling

/hotels/does-not-exist → "Page not found", no console error. ✔ (D-08)

13.4 ❌ What was NOT verified — interactive flows

This is the most important section in this volume.

The limitation

The automated browser used for verification **does not deliver trusted input events to the React root**. Real clicks landed on the correct elements and did nothing; Escape did not close an open dialog.

The evidence it is the harness, not the app

```
// 1 · React's handler IS attached to the DOM node
const b = [...document.querySelectorAll("button")].find(x => x.textContent.trim() === "Open dialog");
const key = Object.keys(b).find(k => k.startsWith("__reactProps"));
Object.keys(b[key]);
// → ["ref", "disabled", "className", "type", "aria-haspopup", "aria-expanded",
//    "aria-controls", "data-state", "onClick", "children"]

// 2 · Invoking it directly works perfectly
b[key].onClick({ type: "click", currentTarget: b, target: b,
  preventDefault(), stopPropagation() });
// → { dialogs: 1, expanded: "true", bodyStyle: "pointer-events: none;" }
```

The dialog opened, `aria-expanded` flipped, and Radix applied its scroll lock correctly. The component is sound; the harness is not dispatching trusted events.

A control test confirmed React delegation itself works for plain buttons — `.click()` on the sidebar toggle successfully flipped the persisted state. Radix menus additionally open on `pointerdown` rather than `click`, so `.click()` legitimately does not open them.

What this leaves unverified

FLOW	BUILT	TYPECHECKED	CLICK-TESTED
Reservation wizard, end to end	✔	✔	❌
Create a booking ≥ ₹50,000 → approval queue	✔	✔	❌
Approve / decline dialogs	✔	✔	❌
Cancel dialog with reason	✔	✔	❌
Check-in / check-out transitions	✔	✔	❌
Customer create / edit submission	✔	✔	❌
Merge execution	✔	✔	❌
CSV import commit	✔	✔	❌

FLOW	BUILT	TYPECHECKED	CLICK-TESTED
Record payment	✓	✓	✗
Rate plan edit	✓	✓	✗
Command palette navigation	✓	✓	✗
Role switching via the dropdown	✓	✓	✗

What was verified instead

- **Role switching via the persisted store**, which exercises the same code path the dropdown triggers. Confirmed across Sales Manager, Hotel Manager and Super Admin.
- **Step gating** — `Continue` is `disabled` until a customer is chosen:

```
{ step1ContinueDisabled: true }
```

This is the actual guard; a real user cannot bypass it. Synthetic handler invocation *can* bypass a `disabled` attribute, which is why driving the wizard that way proved nothing about gating.

- **The rules layer**, by inspection against the code and against seeded outcomes: 66 reservations $\geq$ ₹50,000 correctly sit in `pending_approval`.

Required before Phase 2 sign-off

A manual click-through of the twelve flows above, in a real browser. They are built and typechecked; they are not behaviourally verified, and this manual should not be read as claiming otherwise.

13.5 Role model verification

Performed by setting the persisted session and reloading.

OBSERVATION	SUPER ADMIN	HOTEL MANAGER
Navigation items	16	7
Dashboard title	"Portfolio overview"	"Agra Hotel Taj Pearl today"
KPI row	Revenue / Reservations / Approvals / Cancellations	Arrivals / Departures / In house / Occupancy
"New reservation" button	present	absent
Rate plan Edit buttons	12	0
Rate plan Locked markers	0	12
Read-only banner	absent	present

- ✓ The permission model demonstrably drives the interface.

One copy defect was found and fixed during this pass: the hotel-manager dashboard's recent- bookings panel still read *"Latest bookings across the portfolio"* while showing correctly scoped data. Now reads *"Latest bookings at {property}"*.

13.6 Accessibility

Verified

CHECK	RESULT
All interactive elements have accessible names	✓ after <code>D-07</code>
Focus visible on all focusable elements	✓ global <code>:focus-visible</code>
Form fields labelled and described	✓ via <code>Field</code>
Error state announced	✓ <code>aria-invalid</code> + <code>aria-describedby</code>
Combobox exposes listbox semantics	✓ after fix
Reduced motion respected	✓ global media query

CHECK	RESULT
Contrast ratios	✅ audited — Volume IV §4.3

Not verified

GAP	STATUS
Screen reader pass (NVDA / JAWS / VoiceOver)	❌ Not performed
Full keyboard-only traversal of every screen	❌ Not performed
Clickable <code><tr></code> announced as interactive	⚠️ Known limitation — no valid ARIA role exists; every such row has an alternative path
Charts have no text alternative	⚠️ Open — mitigated by the underlying table being present on every report
Skip-to-content link	❌ Missing — keyboard users tab the sidebar on every navigation

🔗 The skip link is a small, high-value addition for Phase 2.

13.7 Known gaps in the build

Stated so they are not discovered as surprises.

GAP	WHERE	IMPACT
Bulk actions not implemented	<code>/reservations</code> describes them; no row selection exists	The description overstates the build. Either implement selection or amend the copy
Charts lack text alternatives	All report screens	Accessibility
No skip-to-content link	App shell	Accessibility
Offset pagination	Repository layer	Must become cursor-based in Phase 2 — Volume XIV §14.3
11 direct <code>db</code> reads in screens	Listed in Volume XIV §14.4	Each becomes a real aggregate in Phase 2
Roll-up counters maintained client-side	<code>customers</code> , <code>companies</code>	Must move to a Cloud Function — Volume XIV §14.5
No test framework	Whole project	ADR-24 — the weakest decision in the log

13.8 Performance

Measured on the dev server, Chromium, desktop.

MEASURE	VALUE
Dev server cold start	~600 ms
Production build	8.9 s
Route chunk (median)	~2.2 kB gzipped
Seed generation on load	~40 ms
Repository call (simulated)	120–400 ms by design
Heaviest screen	<code>/reports/revenue</code> — 11.3 kB gzipped

Not measured: Lighthouse, real-device performance, memory over a long session. 🔗 Phase 4.

13.9 Reproducing this verification

```
cd "D:\fidato crs"
npm install          # from PowerShell — see Volume XI §11.11
npx tsc --noEmit -p tsconfig.app.json
npm run build
npm run dev
```

Then, in a **real browser** at `http://localhost:5173`:

1. **Role switching** — Sales Manager → Hotel Manager → Finance → Super Admin. Nav, dashboard and available actions should each change.
2. **Reservation flow** — `/reservations/new`, build a booking over ₹50,000, confirm the amber banner and the "Submit for approval" label, submit, find it in `/reservations/approvals`, approve it, confirm the audit timeline updates.
3. **CRM flow** — create a customer, attach a company, trigger a duplicate warning, merge two records on `/crm/merge`.
4. **Restrictions** — as Hotel Manager confirm rate editing is blocked; as Salesperson confirm only assigned companies appear; confirm a completed reservation is read-only; confirm no delete control exists anywhere.
5. **States** — filter a list to zero results and confirm the no-results state (not the empty state); throttle the network and confirm skeletons.
6. **Brand** — `/design-system` renders every token and component; the logo is unaltered; ⌘K opens the palette.

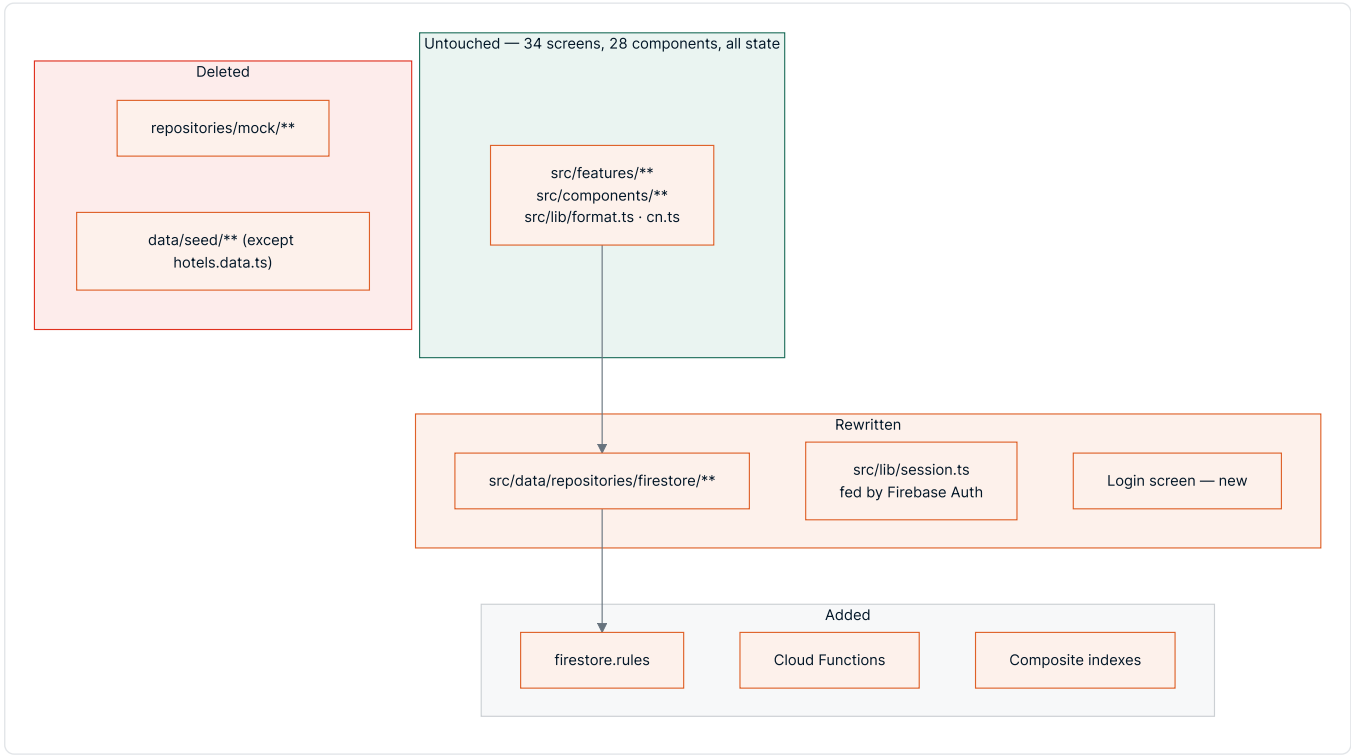
Steps 2–4 are the ones §13.4 could not cover.

VOLUME XIV

Phase 2 handover

Everything needed to take this build to Firebase. Written for whoever picks it up, including the parts that will not be pleasant.

14.1 The shape of the work



Estimate

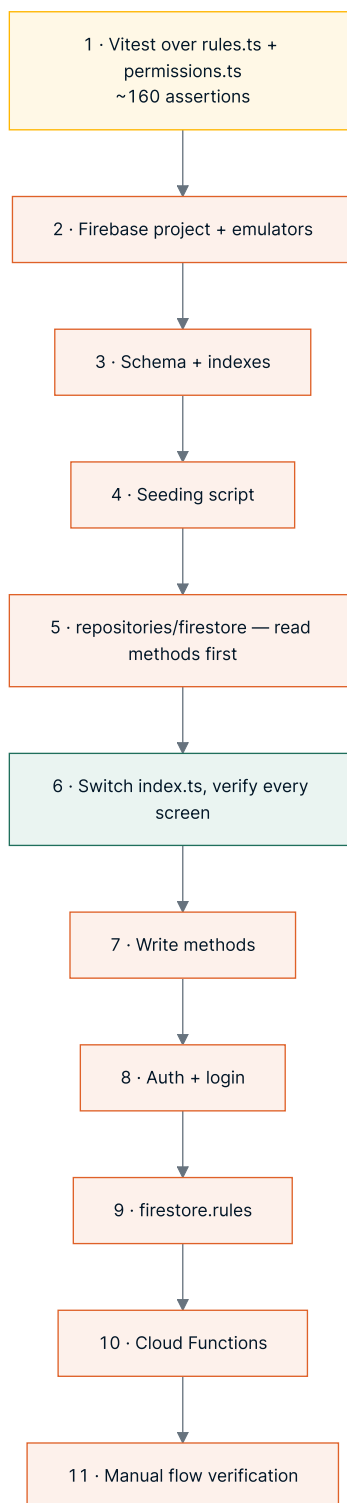
TASK	ESTIMATE
Firebase project, config, emulators	0.5 d
Firestore schema + composite indexes	1 d
Seeding script from <code>hotels.data.ts</code> + generated data	1 d
<code>repositories/firestore/**</code> — 11 repositories	4–5 d
Auth + login + session wiring	1.5 d
<code>firestore.rules</code> from the matrix	2 d
Cloud Functions (roll-ups, invoices, commissions)	2 d
Vitest over <code>rules.ts</code> + <code>permissions.ts</code>	1 d
Migration of the three hard cases (§14.3, §14.7, §14.9)	2 d
Manual verification of the 12 unverified flows	1 d
Total	~16–17 days

⚠ This assumes [ADR-06](#) held — that no repository method acquired capabilities Firestore lacks. Verify before committing to the estimate:

```
grep -rn "\.filter(\|\)\.sort(\|\)\.reduce(" src/data/repositories/mock/index.ts | wc -l
```

Client-side array operations inside a repository are fine where they mirror what the client will do post-fetch, and a problem where they assume the server did the work.

14.2 Order of work



Step 1 first, before any Firebase work. The rules layer is now stable, it is the highest- value code in the project, and tests written against it will not churn. It is also the safety net for everything that follows. [ADR-24](#) explains why it was not done in Phase 1 and why it should be done now.

Step 6 is the checkpoint. Reads only, everything else untouched. If all 34 screens render correctly against Firestore, the interface held and the rest is mechanical. If they do not, stop and fix the interface rather than patching screens.

14.3 Hard case 1 — pagination

The problem. `paginate()` is offset-based; Firestore is cursor-based.

```
// Phase 1
export function paginate<T>(records: T[], page = 1, pageSize = 25): ListResult<T> {
  const start = (page - 1) * pageSize;
  return { items: records.slice(start, start + pageSize), total: records.length, page, pageSize };
}
```

Firestore cannot skip to page 6 without reading pages 1–5, and cannot cheaply produce `total`.

Options

OPTION	FOR	AGAINST
Cursor pagination, Prev/Next only	Native, cheap, scales	Loses numbered pages. <code>Pagination</code> component and the <code>page</code> URL parameter both change
Keep offsets, read-and-discard	No UI change	Reads $N \times \text{pageSize}$ documents to reach page N. Costs real money
Distributed counter for <code>total</code> , cursors for movement	Keeps the "of 1,100" readout	A counter document per collection, maintained by a Function
Move list queries to Algolia/Typesense	Full-text and offset pagination	Another service, another sync

Recommendation: cursor pagination with Prev/Next, plus distributed counters for `total` where the readout is worth keeping (reservations, customers, invoices).

UI impact: `Pagination` loses its numbered window; `useListState` swaps `page` for a cursor token. Both are single-file changes — the component is used through one interface everywhere.

14.4 Hard case 2 — direct `db` reads in screens

Eleven places read `db` synchronously for derived counts, bypassing the repository layer. Each must become a real aggregate.

SCREEN	READS	BECOMES
<code>InvoicesPage</code>	<code>db.invoices</code> for outstanding/overdue totals	<code>financeRepo.invoiceTotals()</code>
<code>PaymentsPage</code>	<code>db.payments</code> for received/unreconciled	<code>financeRepo.paymentTotals()</code>
<code>RunsPage</code>	<code>db.automationRuns</code> for success rate	<code>automationRepo.runStats()</code>
<code>AuditLogPage</code>	<code>db.auditLogs</code> for counts	<code>adminRepo.auditStats()</code>
<code>UsersPage</code>	<code>db.users</code> for role counts	<code>adminRepo.userStats()</code>
<code>RolesPage</code>	<code>db.users</code> for per-role counts	same
<code>CompanyDetailPage</code>	<code>db.customers</code> , <code>db.reservations</code>	<code>companiesRepo.contacts()</code> , <code>.reservations()</code>
<code>HotelDetailPage</code>	<code>db.reservations</code> for property bookings	<code>hotelsRepo.reservations()</code>
<code>CustomerFormPage</code>	<code>db.customers</code> for duplicate check	<code>customersRepo.checkDuplicate()</code>
<code>ImportPage</code>	<code>db.customers</code> for duplicate check	same
<code>TemplatesPage</code>	<code>db.automationWorkflows</code>	<code>automationRepo.workflowForEvent()</code>

Find them all:

```
grep -rn "from \@/data/repositories\" src/features --include=*.tsx | grep "db"
```

⚠ The duplicate checks are the interesting ones. Scanning every customer client-side does not scale. In Firestore this becomes an indexed equality query on a normalised field — which means **adding `emailNormalised` and `phoneNormalised` to the customer document**, written on save, because you cannot index a computed value:

```
emailNormalised: email.trim().toLowerCase(),
phoneNormalised: phone.replace(/\D/g, "").slice(-10),
```

This is a schema change driven by the migration and should be made in the seeding script from the start.

14.5 Hard case 3 — roll-up counters

`customer.totalReservations`, `customer.totalRevenue`, `company.totalReservations`, `company.totalRevenue`, `company.creditUsed` are maintained by the seed today.

🔗 In Phase 2 they must be maintained by a Cloud Function on reservation write, **not** by client code. Client-maintained counters drift the moment two writes race, and the drift is silent.

```
// functions/src/onReservationWrite.ts - sketch
export const onReservationWrite = onDocumentWritten("reservations/{id}", async (event) => {
  const before = event.data?.before.data() as Reservation | undefined;
  const after = event.data?.after.data() as Reservation | undefined;

  const counts = (r: Reservation | undefined) =>
    !r || r.status === "cancelled" || r.status === "draft" ? 0 : 1;

  const deltaCount = counts(after) - counts(before);
  const deltaValue = (counts(after) ? after!.totalAmount : 0)
    - (counts(before) ? before!.totalAmount : 0);

  if (deltaCount === 0 && deltaValue === 0) return;

  const customerId = after?.customerId ?? before?.customerId;
  if (customerId) {
    await db.doc(`customers/${customerId}`).update({
      totalReservations: FieldValue.increment(deltaCount),
      totalRevenue: FieldValue.increment(deltaValue),
    });
  }
  // ...same for company
});
```

`FieldValue.increment` is atomic, which is the whole reason this belongs server-side.

Note the exclusion rule matches the seed exactly: cancelled and draft reservations do not count toward revenue. Volume VII §7.6.

14.6 The `TODAY` constant

43 references across the codebase. All must become `new Date()`.

```
grep -rn "TODAY" src --include=*.ts --include=*.tsx | wc -l
```

LOCATION	CHANGE
<code>data/seed/index.ts</code>	Deleted with the file
<code>repositories/mock/index.ts</code>	Deleted with the file
<code>features/**</code> (dashboard, calendar, inventory, reports)	Import from a <code>lib/clock.ts</code> returning <code>new Date()</code>

🔗 **Recommendation:** introduce `src/lib/clock.ts` now, exporting `now()`. It makes the demo date injectable, which is useful for testing date-dependent logic in Phase 2 — and the seed can keep a fixed clock for the test suite.

14.7 Hard case 4 — the merge operation

The single hardest operation to port. It currently touches three collections in one synchronous pass:

```
merge: (survivorId, absorbedIds, patch, actor) =>
  write(() => {
    for (const r of db.reservations) { /* re-point */ }
    for (const inv of db.invoices) { /* re-point */ }
    Object.assign(survivor, patch);
    survivor.totalReservations = ...; survivor.totalRevenue = ...;
    db.customers = db.customers.filter((c) => !absorbedIds.includes(c.id));
    recordAudit({ action: "merged", ... });
  });
```

In Firestore

It must become a **Cloud Function** using batched writes, because:

- It spans three collections and must be atomic.

- A customer with many reservations can exceed Firestore's **500-write batch limit**, so it needs chunking.
- Doing it client-side would leave partially-merged data if the tab closes mid-operation.

```
// Sketch
export const mergeCustomers = onCall(async ({ data, auth }) => {
  assertPermission(auth, "merge", "customer");
  const { survivorId, absorbedIds, patch } = data;

  const survivor = await db.doc(`customers/${survivorId}`).get();

  for (const collection of ["reservations", "invoices"]) {
    for (const absorbedId of absorbedIds) {
      let last: QueryDocumentSnapshot | undefined;
      while (true) {
        let q = db.collection(collection)
          .where("customerId", "==", absorbedId)
          .limit(400); // under the 500 batch limit
        if (last) q = q.startAfter(last);
        const snap = await q.get();
        if (snap.empty) break;

        const batch = db.batch();
        snap.docs.forEach((d) => batch.update(d.ref, {
          customerId: survivorId,
          customerName: survivor.data().fullName, // denormalised copy must follow
        }));
        await batch.commit();
        last = snap.docs[snap.docs.length - 1];
      }
    }
  }
  // ..patch survivor, recompute roll-ups, delete absorbed, write audit
});
```

⚠ Order remains load-bearing: re-point before delete, and update the denormalised `customerName` alongside `customerId`. Volume VIII §8.6.

14.8 `firestore.rules` — generate, do not hand-write

The permission matrix must be restated as security rules. Hand-writing them creates two sources of truth that will drift.

🔧 **Recommendation:** generate `firestore.rules` from `MATRIX` at build time.

```
// scripts/generate-rules.ts - sketch
import { ROLES, RESOURCES, grantsFor } from "../src/lib/permissions";

const RESOURCE_COLLECTIONS: Partial<Record<Resource, string>> = {
  customer: "customers", company: "companies", reservation: "reservations",
  hotel: "hotels", invoice: "invoices", payment: "payments", /* ... */
};

function rolesWith(resource: Resource, action: Action): Role[] {
  return ROLES.filter((role) => grantsFor(role, resource).includes(action));
}

// emits:
// allow read: if hasAnyRole(['super_admin', 'admin', ...]);
```

Then the hand-written part is only the scoping and the business rules, which are few:

```
match /reservations/{id} {
  allow read: if isSignedIn() && (
    hasAnyRole(['super_admin', 'admin', 'sales_manager', 'finance', 'support', 'viewer'])
    || (hasRole('salesperson') && resource.data.ownerId == request.auth.uid) // BR-05
    || (hasRole('hotel_manager') && resource.data.hotelId == userHotelId()) // BR-05
  );

  allow create: if hasPermission('reservation', 'create');

  allow update: if hasPermission('reservation', 'edit')
    && !(resource.data.status in ['completed', 'cancelled', 'no_show']); // BR-03

  allow delete: if false; // BR-01
}

match /rate_plans/{id} {
  allow read: if isSignedIn();
  allow write: if hasPermission('rate', 'edit') && !hasRole('hotel_manager'); // BR-04
}

match /audit_logs/{id} {
  allow read: if hasPermission('audit_log', 'view');
  allow create: if isSignedIn();
  allow update, delete: if false; // BR-08
}
```

`allow delete: if false` on reservations, and `update, delete: if false` on audit logs, are the only places BR-01 and BR-08 are *truly* enforced. Everything in the client is a courtesy.

14.9 Hard case 5 — search

`matchesSearch()` scans named fields client-side. Firestore has no substring search.

OPTION	FOR	AGAINST
Prefix search via <code>>=</code> / <code><=</code> on a normalised field	Free, native	Prefix only — searching "peerless" would not match "The Peerless Inn"
Array of search tokens + <code>array-contains</code>	Native, word-level	Document bloat; no partial-word matching
Algolia / Typesense	Real search, typo tolerance, ranking	Another service, another sync path, cost
Keep client-side over a fetched page	Zero work	Only searches what is already loaded — misleading

Recommendation: prefix search on normalised fields for the smaller collections (hotels, users, companies), and Typesense for reservations and customers once they pass ~10,000 documents.

The interface does not change either way — `ListQuery.search` stays as it is.

14.10 Composite indexes

Firestore requires a composite index for any query combining a filter with a sort. From the current screens:

COLLECTION	INDEX
<code>reservations</code>	<code>ownerId ASC, checkIn DESC</code>
<code>reservations</code>	<code>hotelId ASC, checkIn DESC</code>
<code>reservations</code>	<code>status ASC, totalAmount DESC</code>
<code>reservations</code>	<code>status ASC, checkIn ASC</code>
<code>reservations</code>	<code>channel ASC, checkIn DESC</code>
<code>customers</code>	<code>ownerId ASC, lastActivityAt DESC</code>
<code>customers</code>	<code>status ASC, lastActivityAt DESC</code>
<code>companies</code>	<code>ownerId ASC, totalRevenue DESC</code>
<code>companies</code>	<code>tier ASC, totalRevenue DESC</code>
<code>invoices</code>	<code>status ASC, dueDate ASC</code>

COLLECTION	INDEX
<code>invoices</code>	<code>companyId ASC, issueDate DESC</code>
<code>auditLogs</code>	<code>entityType ASC, at DESC</code>
<code>auditLogs</code>	<code>entityId ASC, at DESC</code>

Derive the full set from the `FILTER_KEYS` and `defaultSortBy` of each list screen — every combination the UI can produce needs one, and Firestore will tell you at runtime with a console link.

14.11 Data types at the boundary

PHASE 1	FIRESTORE
<code>IsoDateTime</code> (string)	<code>Timestamp</code>
<code>IsoDate</code> (string)	Keep as string
<code>number</code>	<code>number</code>
<code>string[]</code>	<code>array</code>
Nested objects (<code>rooms</code> , <code>lines</code>)	<code>map[]</code>

⚠️ **Keep `IsoDate` as a string.** Date-only values (`checkIn` , `checkOut` , `issueDate`) must not become `Timestamp` , or timezone conversion will shift them by a day. `yyyy-MM-dd` also sorts and range-filters correctly as a string, which is exactly what the queries rely on.

Convert `IsoDateTime` at the repository boundary:

```
function fromDoc<T>(snap: QueryDocumentSnapshot): T {
  const data = snap.data();
  return {
    ...data,
    id: snap.id,
    createdAt: (data.createdAt as Timestamp).toDate().toISOString(),
    updatedAt: (data.updatedAt as Timestamp).toDate().toISOString(),
  } as T;
}
```

Doing it at the boundary means **nothing above the repository changes**. `src/lib/format.ts` keeps taking strings.

14.12 Auth and the role switcher

```
// Phase 1
export const useSession = create<SessionState>()({
  persist((set) => ({ role: "sales_manager", setRole: (role) => set({ role }) }),
    { name: "fidato.session" }),
});
```

Phase 2 feeds the same store from Firebase Auth custom claims:

```
onAuthStateChanged(auth, async (fbUser) => {
  if (!fbUser) return useSession.setState({ user: null, role: null });
  const token = await fbUser.getIdTokenResult();
  useSession.setState({
    user: { id: fbUser.uid, name: fbUser.displayName ?? "", email: fbUser.email ?? "" },
    role: token.claims.role as Role,
    hotelId: token.claims.hotelId as string | undefined,
  });
});
```

✂️ **Keep the role switcher, gated to development.** It remains the fastest way to review the permission model:

```
{import.meta.env.DEV && <RoleSwitcher />}
```

`useScope()` , `useActor()` and `useCurrentUser()` keep their signatures, so no screen changes.

14.13 What must not change

If any of these change, the migration has gone wrong:

MUST NOT CHANGE	WHY
<code>src/data/types.ts</code> shapes	The screens are written against them
Repository method signatures	The seam depends on them
<code>ListQuery</code> / <code>ListResult</code>	Every list screen consumes them
<code>src/lib/permissions.ts</code>	Becomes the source for security rules
<code>src/lib/rules.ts</code>	The business rules do not change with the backend
<code>src/lib/format.ts</code>	Pure formatting
Any component in <code>src/components/</code>	—
Any screen in <code>src/features/</code>	Except where §14.4 replaces a <code>db</code> read

14.14 Phase 2 definition of done

#	CRITERION
1	All 38 routes render against Firestore
2	Login works; the role switcher is dev-only
3	<code>firestore.rules</code> generated from the matrix, with rule tests
4	Vitest covers <code>rules.ts</code> and <code>permissions.ts</code>
5	Roll-ups maintained by Cloud Functions
6	Merge runs as a Cloud Function with chunked batches
7	Composite indexes deployed
8	The 12 flows in Volume XIII §13.4 verified by hand
9	Seeding script populates a fresh project from <code>hotels.data.ts</code>
10	Emulator suite runs the whole app offline

14.15 What Phase 1 left for Phase 3 and 4

Phase 3 — n8n. Every workflow is already defined with trigger, conditions, ordered steps and the target n8n node per step. The webhook path is displayed on each workflow's detail page. The definitions in this system *are* the specification.

Phase 4 — final testing. Load, security review, accessibility audit with a real screen reader, acceptance testing against the business rules.

VOLUME XVI

Phase 2 as built

Volume XIV is the Phase 2 *plan*, written before the build. This volume is what was actually constructed, and where the two disagree, **this one is what shipped**.

Read Volume XIV for the reasoning that preceded the work and the effort estimates. Read this for the system that exists.

16.1 The constraint that shaped everything: Spark

The project runs on the Firebase **Spark** plan. Volume XIV assumed Cloud Functions. There are none, and there is no Admin SDK.

Four of the handover's recommendations had to be redesigned before any of them could be built:

VOLUME XIV PROPOSED	WHY IT IS IMPOSSIBLE	WHAT WAS BUILT
Cloud Function roll-up counters	No functions	Counters updated inside the client transaction that creates the booking
Server-side customer merge	No functions	A transactional merge in the repository layer
Server-issued invoice numbers	No functions	A <code>counters</code> document incremented transactionally
Custom claims carrying the role	Claims need the Admin SDK	The role lives in <code>users/{uid}.role</code> ; every rule re-reads it
Immutable audit via a function	No functions	Rules permit create and refuse update and delete

Three further consequences follow from the same constraint and appear throughout the system:

Every Firestore read must be bounded. Spark allows 50,000 reads a day. A report that reads every reservation to aggregate costs one read per row *per view*. `getCountFromServer` for counts, `limit()` everywhere else. This is Constraint 5.

No server means no secrets. Anything the app knows, a user can read out of the bundle. That is why automation goes through a queue and a webhook rather than the app calling Drive or WhatsApp directly, and why the register's publishable Supabase key grants nothing on its own.

No server means no scheduled work. Nothing runs unless a browser or n8n makes it run.

16.2 Identity

The chain

```
LoginPage
├─ signInWithEmailAndPassword      firebase/auth
│   └─ onAuthStateChanged         src/lib/session.ts
│       └─ getDoc(users/{uid})    ← the role lives here
│           └─ useSession.role
│               └─ navigationFor(role) → what the sidebar shows
│                   └─ <Guard>      → what the router opens
```

⚠️ **The `users` document id must be the auth uid.** Rules resolve a role with `get(/databases/{db}/documents/users/{request.auth.uid})`, and **rules cannot query** — they can only fetch a document by a path they can construct. A `users` collection keyed by anything else could not be read by a rule at all.

Volume XIV proposed keying users by email. That would not have worked, and the reason is worth holding on to: it is not a preference, it is a hard property of the rules language.

Invitations, and why they are a separate collection

An invited person has no uid until they sign up, so they cannot have a `users` document. The invitation therefore lives in `invitations/{lowercased-email}`.

The deterministic id is also what makes the invitation readable by the person it belongs to and nobody else:

```
allow read: if request.auth.token.email.lower() == invitationId;
```

A random id would have required a query, which rules cannot do.

Bootstrap is a deliberate chicken-and-egg

Only an Owner or Admin may create an invitation. With no Admin SDK, no script can create the first one. **The first Owner is written by hand in the Firebase console**, which bypasses rules entirely. `docs/RUNBOOK.md` carries the procedure.

⚠️ Rules explicitly refuse `role: automation` on an invitation, so the service account n8n uses can never be handed to a person by mistake.

Claiming an invitation onto an existing account

A late addition, after a real failure. Someone could hold a Firebase Auth account with no `users` document — signed in, and roleless, with no way forward. `claimInvitation` repairs that state by writing the `users` document for an already-authenticated uid.

16.3 The permission model, in two halves

Both halves exist and neither is optional.

HALF	FILE	DECIDES	IF IT IS WRONG
Client matrix	<code>src/lib/permissions.ts</code>	What renders — navigation, buttons, routes	A screen that opens and then fails, or a capability nobody can find
Rules	<code>firestore.rules</code>	What returns	Actual exposure

⚠️ **A grant in the matrix is not a permission.** `firestore.rules` is the only real boundary. The matrix merely decides what a user is invited to try.

They cannot see each other, so they drift. `src/lib/permissions.test.ts` exists solely to assert they agree, resource by resource, and its comments name the rule each case mirrors. Two resources had already drifted for Finance when that test was written — the grants had been widened without touching the rules, which bought two broken navigation entries rather than any additional access.

The roles

ROLE	SHAPE
owner	Everything. The only role that may create, promote or demote another Owner
admin	Everything except owner-escalation. Full user administration
crs_manager	The central desk. Every account, unscoped. No commission terms, no user administration
manager	Amends the book, does not delete from it
salesperson	Raises leads and bookings; cannot amend them. Scoped to their own records
finance	Invoices and payments. ⚠️ Deliberately no commission and no audit log
viewer	Read only
hotel_manager, support	Dormant — empty grant maps
automation	System. n8n only. Touches the queue and its write-back fields

⚠️ **Finance is excluded from commission on purpose.** They handle what Fidato *bills*; commission is what Fidato *earns*, and that is a negotiated commercial term. Granting it in the matrix would not have widened access — the rules refuse it — it would have produced a menu entry that opened and failed.

⚠️ **Only an Owner may create or promote an Owner.** Without that, an Admin escalates in two moves: invite an Owner at an address they control, then claim it.

Row-level scoping

A salesperson sees their own book. The rule and the query must agree exactly: `scopeConstraints` narrows the query to `ownerId == me`, and the rule permits the same. A rule and a query that disagree do not return a subset — **the query fails outright**.

16.4 Data, and the shapes Firestore refuses

`undefined` is not a value

Firestore rejects a document containing `undefined` anywhere, and the error does not name the field. This surfaced as *"Could not create / Nothing was saved"* on reservation creation.

Every denormalised field is now coalesced at the call site:

```
customerId: customer.id,
customerName: customer.fullName ?? "",
hotelId: hotel.id,
hotelName: hotel.name ?? "",
hotelCity: hotel.city ?? "",
```

`toDoc()` strips `undefined` as a second line of defence, and the collection defaults were backfilled so a document that predates a field still reads.

Transactions: all reads before all writes

Firestore aborts a transaction the moment it reads after writing — it must, because the write was based on a snapshot and a later read could not be given a consistent view of it.

```
const snap = await tx.get(customerRef);           // ┐ every read
const cSnap = companyRef ? await tx.get(companyRef) : null; // ┐ first
tx.set(ref, reservation);                       // ┐ then every
tx.update(customerRef, { ...rollups });          // ┐ write
if (companyRef) tx.update(companyRef, { ...rollups }); // ┐
```

⚠ **The bug this replaced was intermittent by data shape, not by luck.** The company read sat below `tx.set`. A booking for an unattached customer has only one read, so the ordering happens to be legal and it worked. A booking for a company-linked customer threw. `tests/rules/transactions.test.ts` pins both orderings against the real engine.

Queries silently exclude documents missing the sort field

A Firestore `orderBy` drops any document that lacks the field. A queue sorted by a field the queue rows did not carry returned nothing at all — no error, an empty list.

Composite index direction must match exactly

An index built ASC does not serve a DESC query. Both are generated from the code by `scripts/build-indexes.mjs`, so the index set cannot drift from the queries.

16.5 Commission, and why it is a subcollection

⚠ **Firestore has no field-level read security.** A field on a document is readable by everyone who can read that document, whatever the interface renders.

Commission therefore cannot live on `hotels/{id}`. It lives in `hotels/{id}/private/commercial`, guarded by its own rule admitting only Owner and Admin.

Hiding it in the UI would have done nothing at all — the SDK and the REST API do not consult the interface. This is Constraint 3, and it is the clearest example in the system of the difference between *not shown* and *not readable*.

16.6 Import

CSV and Excel, both, with the descriptor as the single source of truth.

```
parse → guessMapping → validateRows → commit
```

⚠ **Nothing is written until the final button.** Parsing, mapping, validation and duplicate detection all happen in the browser against a file that never leaves the machine. An import that half-succeeds and leaves you guessing which half is worse than one that refuses to start.

The template is generated from the descriptor, so it cannot drift out of date — a change to the validation changes the template in the same commit.

Validation runs in two passes: per-field, and then `checkRow` for constraints that span columns. The second exists because a bank branch with no account number is two individually valid cells that are jointly useless — the voucher prints a bank block only when it has both an account name and a number, so a half-filled row imports cleanly and then silently prints nothing.

`xlsx` is lazily imported, so a 424 kB parser stays out of every other route.

16.7 Where the build differs from Volume XIV

VOLUME XIV	AS BUILT	WHY
Users keyed by email	Keyed by auth uid	Rules cannot query; they can only fetch by a constructible path

VOLUME XIV	AS BUILT	WHY
Cloud Functions for roll-ups, merge, numbering	All client-side and transactional	Spark
Custom claims for roles	Firestore document read by rules	No Admin SDK
CSV import	CSV and Excel	Requested during the build
Approvals workflow	Dropped	It modelled a step Fidato does not perform
—	CRS Manager role added	A central desk that works every account, unscoped
—	Hotel confirmation number, all five meal plans	Real operational requirements

16.8 What Phase 2 verified

Rules tests	101, against the real rules engine in the emulator
Unit tests	153
Mutation check	Weakening the commission rule fails exactly three tests

⚠️ **What none of it proves:** that a client sends the query a rule expects. The rules tests exercise the engine; the unit tests exercise pure logic. The interactive flows — wizard, approve/cancel, merge, import commit — remain click-tested by hand only.

VOLUME XVII

Automation and n8n

How a booking becomes a voucher in a guest's inbox, a file in Drive and a message on WhatsApp — without the application ever holding a credential for any of them.

17.1 The rule

React UI → repository layer → Firestore → automationQueue → n8n → the outside world

The React app never talks to Drive, WhatsApp, email, AI, accounting or marketing.

This is not a preference. Spark has no server, so anything the app can do, it does in a browser with code the user can read. A Drive credential in the bundle is a published Drive credential. The app writes a row to `automationQueue` and posts a webhook; n8n holds every secret and makes every outbound call.

Anything that appears to break this rule is a defect. It is Constraint 2.

17.2 The two records, and why there are two

Creating a reservation produces **two** independent records of the same intent:

RECORD	WHERE	PURPOSE	WRITTEN BY
The queue row	<code>automationQueue/{id}</code>	The durable record that this event happened	The app, always
The webhook POST	n8n's endpoint	The optimistic fast path	The app, fire and forget

```
reservationsRepo.create(...)
├─ runTransaction → the booking and its roll-ups
├─ recordAudit
└─ void pushToN8n("reservation.created", ...)
    ├─ setDoc(automationQueue/{id}) ← durable
    ├─ fetch(webhook.url) ← optimistic
    └─ .then(outcome => updateDoc(reservation, { automation: outcome }))
```

⚠ `pushToN8n` is deliberately not awaited. A booking must not fail because a webhook is slow or an n8n instance is down. The guest's reservation is the thing that matters; delivery is a consequence of it.

⚠ Which means a successful save says nothing about delivery. That is why the outcome is written back onto the reservation and rendered as a visible status, rather than assumed.

17.3 The payload: two bodies

The webhook carries two objects, and they are not interchangeable.

BODY	HOLDS	CONSUMED BY
<code>data.voucher</code>	The rendered voucher — everything a guest should see	Email, Drive, WhatsApp
<code>data.reservation</code>	The booking record — everything the business stores	The register insert

They were separated after an attempt to serve both from one shape. A voucher is a document addressed to a guest; a reservation is a row addressed to a database. Fields that belong in one are wrong in the other — internal notes, owner ids, roll-up counters — and merging them meant either leaking internals onto a guest's document or omitting fields the register needs.

17.4 The voucher

Three renderings of one source, in `src/features/reservations/`.

RENDERING	FILE	USED FOR
HTML	<code>voucher.ts</code> — <code>renderVoucherHtml</code>	On-screen preview, and the email body
PDF	<code>voucherPdf.ts</code> — jsPDF, vector	Download, WhatsApp, Drive

RENDERING	FILE	USED FOR
Sample	<code>voucherSample.ts</code>	Tests and the "Send test" button

What the outside world forced

Gmail strips `<svg>` and blocks `data: URIs`. The inline SVG logo — perfect on screen — vanished in email. The email uses a PNG hosted on `fidatohotels.com`, served from the sending domain so it is not treated as a remote tracking pixel.

The email is the voucher, not a covering note. A plain-text mail with an attachment reads as a system notification. The body is the branded document.

⚠️ **jsPDF's built-in fonts are WinAnsi only.** ₹, en dashes and middots are not representable. `ascii()` strips them at render, because a blank box on a document about money reads as a fault in the business, not in the font. Currency is written as `INR`.

Bank details print only when both an account name and a number are present. A bare IFSC, or an account number with nobody to pay, is worse than no bank block — a guest may attempt a transfer against it. IFSC is uppercased at render as well as at entry, because the form only protects what the form wrote.

A QR code links to the Fidato site, generated at render.

The rejected approaches

ATTEMPT	WHY IT FAILED
HTML voucher in the email body only	No attachment to keep; the logo stripped
Gotenberg container rendering HTML → PDF	The n8n host had no Docker access
Base64 PDF over the webhook	Payload size, and n8n handled it poorly
n8n rendering the PDF from HTML	Added a rendering dependency for a document the app can already draw

What settled: the app draws the PDF as vectors — 18 kB, one page, about 270 ms — and n8n receives it ready to send.

17.5 Delivery

Email

From `crs@fidatohotels.com`, over SMTP on the company domain.

⚠️ **Deliverability is a DNS problem, not a code problem.** Until SPF, DKIM and DMARC exist for `fidatohotels.com` and align with the sender, vouchers land in spam. No amount of work in this repository changes that.

Google Drive

A copy of every voucher, filed automatically.

WhatsApp — Meta Cloud API

An approved utility template, with the PDF **uploaded to obtain a media id** rather than linked.

⚠️ **Uploading avoids publishing.** A link would require the voucher to sit at a public URL — a guest's name, itinerary and the property's bank details, reachable by anyone with the address.

⚠️ **Named and positional template parameters are not interchangeable.** A template approved with named parameters rejects positional ones and vice versa; the error is not obvious.

Interakt was implemented first and superseded when the Meta app and its approved template became available. The Interakt workflow remains in `docs/n8n/` as a fallback.

The register insert

An HTTP node writes the reservation into the booking register's Supabase project.

⚠️ **It uses HTTP, not the Supabase node.** The Supabase node depended on n8n *Variables*, which is a licensed feature and simply undefined on the community edition — expressions referencing `$vars` evaluated to nothing and the node failed with no useful message.

⚠️ **The credential must hold the `service_role` key.** With the publishable key the insert is refused by RLS with error 42501. This is currently outstanding.

17.6 Failure, and making it visible

Automation that fails silently is worse than none, because the booking looks complete and the guest has nothing.

On the reservation

Every reservation carries the outcome of its push:

```
{ "status": "sent", "detail": "Delivered", "withPdf": true,
  "at": "2026-08-14T10:02:04.405Z", "durationMs": 670 }
```

The health banner

`AutomationHealthBanner` counts **recent bookings that did not reach n8n**.

⚠️ **Counted from reservations, not from the queue.** The queue is readable only by Owner and Admin, so a count taken from there would be blank for the salesperson who made the booking — the one person who needs to know their guest was never emailed.

It is silent when everything is fine. A banner that is always on screen stops being read.

The switched-off webhook

The webhook can be configured and disabled at once, and that state produced a memorable waste: bookings queued correctly, nothing sent, and n8n showed no executions at all.

⚠️ **Worse, "Send test" hardcoded `enabled: true`.** The test passed, proving nothing about the path real bookings take. A standing banner now reports a configured-but-disabled webhook, and the test result says explicitly that it does not reflect live behaviour.

17.7 The gap that remains

⚠️ **Nothing ever writes back to an `automationQueue` row.**

Observed before the launch wipe: six rows at `pending` with `attempts: 0`, while all three reservations recorded `automation: sent · Delivered`. **Delivery genuinely worked.** The queue is simply write-only from the application's side, and the n8n workflow does not close the rows either.

The consequence is a contradiction between two screens:

SCREEN	REPORTS
Reservations	Delivered
Automation → Run history	0% success, pending count only grows

`AutomationHealthBanner` is unaffected, because it counts reservations rather than the queue — a decision made for a different reason that happens to save it here.

The fix is one line. `pushToN8n` already knows the outcome and writes it to the reservation; it could write it to the queue row too. Or n8n could close the row it consumed. It is untouched because it decides *who owns the queue's lifecycle*, and that is a design question rather than a bug fix.

17.8 Workflow files

In `docs/n8n/`. Import from file; credentials are never in the JSON.

FILE	PATH
<code>fidato-reservation-meta.json</code>	The current workflow — 10 nodes, Meta Cloud API, HTTP register insert
Earlier variants	Interakt, SMTP-only, and no-converter versions retained as fallbacks

⚠️ **Nodes must not reference other nodes by name.** `$( 'Reservation confirmed' )` breaks the moment anyone renames the trigger, with the error *"Referenced node doesn't exist"*. The Code nodes resolve their input by walking the incoming items and falling back through candidate names.

⚠️ **`$vars` is a licensed feature.** On the community edition it is undefined, and expressions using it fail quietly. Values it once held are now literals or credentials.

VOLUME XVIII

The booking register

The historical record: 6,626 rows carried across from `FH Booking Register.xlsx`, in their own database, behind their own access control.

18.1 What it is, and what it is not

The register is the spreadsheet Fidato ran on before this platform existed — years of bookings, maintained by hand by whoever took them.

It was digitised rather than abandoned because it is the one thing a new system cannot replace by going forward. Rates, corporate relationships and seasonal patterns all live in it.

⚠️ **It is a separate Supabase project and shares no domain model with the CRS.**

	REGISTER ENTRY	CRS RESERVATION
Is	A historical record of something that happened	A live booking under management
Lives in	Postgres, Supabase project <code>vghughipfjltjmelumdo</code>	Firestore, <code>crstest-9a0c5</code>
Shape	33 columns, all nullable but two	A typed domain object with rules
Edited	To correct the past	To manage the present

Conflating them would produce two systems that disagree about the same stay. The register module in `src/features/register/` imports nothing from `@/data/types` for exactly this reason.

Access: Owner and CRS Manager only.

18.2 The data, honestly described

The source is a hand-maintained spreadsheet, and the schema reflects that: **every column is nullable except `sheet_name` and `excel_row_num`**.

FACT	CONSEQUENCE
1,657 of 6,626 rows are entirely blank	Spreadsheet padding. Shown by default, with an opt-out, because the row count is expected to match the file
563 cancellations spelled four ways	"Cancelled", "Cancellec", "cancelled"... A folded column normalises them for filtering while preserving what was typed
<code>payment_status</code> holds 868 distinct values	Free text — <i>"Payment Received on 02 April (RTGS-...)"</i> . Not a status. Deliberately not a dropdown; the search box covers it
<code>commission_amount</code> empty on all 6,626 rows	Never tracked. Filled in by hand through this screen
A quarter have no check-in date	Sorting puts nulls last, always

The column that is not money

⚠️ `amount_received` holds bank and UTR reference numbers alongside real payments.

Summing it produced **1.37 quadrillion** against 7.2 crore of revenue. 143 values exceed a crore; 302 are more than five times their own booking's revenue; the same number repeats down several rows where the spreadsheet was filled down.

It is never presented as money. Values larger than the booking they belong to — with 5% slack for rounding and small overpayments — are counted as *suspect* and surfaced as a data-quality warning that disappears as the column is corrected. The fourth totals card shows **Cancelled** instead, from a clean column.

This is Constraint 10.

18.3 Access, decided twice

⚠️ **Two independent systems decide whether you see this screen and whether it has anything on it, and they cannot see each other.**

DECIDES	WHERE	IF YOU ARE MISSING FROM IT
Whether the screen renders	<code>src/lib/permissions.ts</code>	No navigation entry, no route
Whether rows come back	<code>register_access</code> in Supabase	A working screen over an empty table

Somebody named in one and not the other gets no error — just nothing. `docs/supabase/register-security.md` documents the model.

How a Firebase user authenticates to Postgres

```
createClient(url, publishableKey, {
  accessToken: async () => (await auth.currentUser?.getIdToken()) ?? null,
})
```

⚠️ **The token authorises, not the key.** The publishable key ships in the browser bundle and grants nothing on its own — every policy requires a valid Firebase ID token *and* an allowlist entry.

⚠️ **accessToken is called before every request, not once at startup.** Firebase ID tokens expire after an hour and `getIdToken()` refreshes on demand. Caching the string would work for an hour and then fail with an empty table until a reload.

⚠️ **Policies target anon, not authenticated.** Supabase's `authenticated` role requires a `role: "authenticated"` claim in the JWT, which Firebase does not mint and which — with no Admin SDK on Spark — nothing can add. The policies therefore attach to `anon` and authorise on the contents of `auth.jwt()`.

⚠️ **A SECURITY DEFINER function is required to read the allowlist from inside a policy,** since `register_access` is itself RLS-protected and a policy cannot read a table it is not permitted to read.

The Supabase-side step nothing in this repository can do

Supabase must be told to trust the Firebase project: **Authentication → Third-Party Auth → Firebase, project `crstest-9a0c5`**. Until that exists, Supabase rejects the token before RLS runs and everything returns nothing.

18.4 The finding that generalises: views bypass RLS

A Postgres view runs as its *owner*, not its caller, and bypasses RLS entirely — unless it is created with `security_invoker = true`.

The lockdown was tested by querying the `bookings` table in SQL. It was correctly protected. The test passed.

The browser queries the **view**. Signed out, it received all 6,626 rows.

Both views now set `security_invoker = true`, re-verified end to end from a signed-out browser rather than from SQL. It is Constraint 12, and it must be applied to any future view over register data.

The general lesson is larger than Postgres: a test that exercises a different path from production proves nothing about production. The same shape appears three more times in this manual — `tsc --noEmit` compiling nothing, "Send test" forcing the webhook enabled, and the fixture that lied about the voucher payload.

18.5 Reporting, and the row cap

Coverage-driven, not a fixed list

Charts and filters render for a column only where `filled > 0`, read from `register_field_coverage`.

`commission_amount` is empty on every row today and is filled in by hand through this very screen. A hardcoded chart list would show an empty commission chart forever, or omit it forever. Enter one figure and the chart appears, with no code change and no deploy.

⚠️ **This is also what produced the worst failure on the screen.** The coverage view runs with `security_invoker`, so it counts what the *caller* can see. A caller the register has not admitted receives a perfectly successful response listing all 18 fields with `filled: 0` — indistinguishable, to the client, from genuinely empty columns.

Every filter and chart being gated on that, an access failure silently removed the Properties, Booker and Companies dropdowns and all four charts, and the Reports tab then explained that nothing had been entered in any of the 18 columns yet. Confident, and wrong.

The screen now reports that it has no rows to show and names all three possible causes — genuinely empty, project not trusted, address not on the allowlist — **without claiming which**, because from the client they are indistinguishable. An earlier version asserted an access problem and would have accused the owner of one the moment they cleared the register on purpose.

PostgREST caps every response, silently

⚠️ **Supabase ships with "Max rows" at 1,000. Ask for 10,000 and you receive 1,000, with a 200, and nothing in the response says it was cut.**

PostgREST has no `DISTINCT` and no `SUM`, so every dropdown and every chart here is folded in the browser. A cap therefore does not shorten a list — **it makes the numbers wrong**.

BEFORE	SYMPTOM
<code>fetchDistinct</code> asked for 7,000	The properties dropdown listed a fraction of the 82; companies a fraction of the 533
No <code>ORDER BY</code>	A <i>different</i> fraction each load
Aggregations asked for 10,000	Revenue was the sum of the first 1,000 of 6,626 rows

Now every scan pages with `.range()`, ordered by `excel_row_num`, using the true count from the `Content-Range` header — which reports the real total even when the body was capped, so a capped page cannot be mistaken for the last one.

⚠️ **The `ORDER BY` is load-bearing, not tidiness.** Paging without one lets Postgres order two pages differently, so rows arrive twice or never.

Consolidated at the same time. The Reports tab issued six independent full scans — totals, monthly, and one per bar chart — which paging would have turned into forty-two requests. It now reads the filtered register once and folds it six ways, on the same query key as the totals cards. The three dropdowns share one scan instead of three.

The folds are pure functions, so `registerRepo.test.ts` tests them directly and runs the paging against a fake PostgREST that caps its responses the way the real one does — asserting a complete result at caps of 1,000, 400 and 250, with contiguous ranges.

The better fix, not yet done: aggregate in Postgres with views or an RPC, `security_invoker` set. It needs DDL access.

18.6 Reads and writes go to different objects

OPERATION	TARGET
Read	<code>register_bookings</code> — the view: folded status, blank-row flag
Write	<code>bookings</code> — the table

⚠️ **The view is not updatable and must never become so.** `booking_status_normalised` is derived; writing it back would silently discard what a person actually typed.

`updated_at` is maintained by a trigger. Nothing in the client sets it — doing so would fight the database for the timestamp.

18.7 New bookings file into the register

A reservation created in the CRS is inserted into the register by n8n, so the historical record continues rather than stopping at the platform's launch date.

⚠️ **The idempotency guard uses a plain unique index, not a partial one.** `ON CONFLICT` cannot infer a partial index — Postgres raises error 42P10. The partial clause proved unnecessary in any case, because NULLs are distinct in a unique index. Retrying a delivery is now a no-op that preserves any manual edits made in between.

18.8 Operating it

TASK	HOW
Load data	Supabase → Table Editor → <code>bookings</code> → Insert → Import data from CSV. Template: <code>docs/supabase/register-import-template.csv</code>
Clear it	<code>docs/supabase/clear-register.sql</code> , from the SQL editor. ⚠️ The app cannot: RLS scopes DELETE to rows the caller can see, and the browser key sees none
Grant access	Add the address to <code>register_access</code> and confirm the role in <code>permissions.ts</code>

⚠️ **Only `sheet_name` and `excel_row_num` are NOT NULL.** Dates must be `YYYY-MM-DD`; money must be bare numbers; `id`, `created_at` and `updated_at` are filled by the database.

⚠️ **There is no undo.** Point-in-time recovery is a paid Supabase feature and is not enabled. Take a backup table first.

VOLUME XIX

Launch and operations

Taking the platform live, and running it afterwards.

Volume XI is the repair section — symptom to cause to fix. This volume is the procedures: what to do on purpose, in what order, and what cannot be undone.

19.1 State at the point of launch

14 August 2026. Every business collection cleared at the owner's instruction so the platform starts on real data.

COLLECTION	DOCUMENTS
<code>users</code>	1 — the Owner. ⚠️ The only way into the application
<code>settings</code>	2 — <code>org</code> and <code>webhook</code>
<i>everything else</i>	0

Cleared: reservations, customers, companies, hotels, roomTypes, seasons, inventory, invoices, payments, commissions, automationQueue, automationRuns, auditLogs, notifications, counters, importJobs, mergeJobs, and all pending invitations.

⚠️ `counters` went with the rest, so booking references restart at `FH-2026-00001`.

⚠️ `settings` was deliberately kept. It is configuration, not data: `org` holds the company details printed on every voucher — brand name, legal name, GSTIN, registered address, support email and phone — and `webhook` holds the n8n URL, secret and enabled flag. Deleting it would have erased the letterhead and disarmed the automation on launch day.

⚠️ The Owner account was kept for a reason that is not sentiment. It was the only `users` document. Deleting it locks the product out of itself: with no Admin SDK on Spark, the way back in is hand-writing a document in the Firebase console.

19.2 The launch sequence

Order matters. Each step depends on the one before it.

1 — Clear the register

The launch wipe covered Firestore only. The register still holds its rows.

Run `docs/supabase/clear-register.sql` from the Supabase SQL editor: it counts first, offers a backup table, then truncates.

⚠️ The application cannot do this. RLS scopes `DELETE` to rows the caller can see, and the browser's publishable key sees none. That is the lockdown working correctly.

2 — Clear the external blockers

None of these are code, and none can be fixed from the repository.

BLOCKER	WHERE	UNTIL THEN
Firebase not trusted by Supabase	Supabase → Authentication → Third-Party Auth → project <code>crstest-9a0c5</code>	The register returns nothing for everyone
Address not on the allowlist	<code>register_access</code> in Supabase	That person sees an empty register
No SPF / DKIM / DMARC	DNS for <code>fidatohotels.com</code>	Vouchers land in spam
n8n holds the publishable key	The n8n Supabase credential needs <code>service_role</code>	New bookings are refused by the register with error 42501

3 — Invite the team

Admin → Users. Every previous invitation was cleared in the wipe.

⚠️ Only an Owner may invite an Owner. Rules refuse any write that sets or clears that role from anyone else — this is what stops an Admin escalating in two moves.

⚠️ `role: automation` cannot be invited at all. It exists for n8n.

4 — Import the data, in this order

From **Import** in the sidebar. **Properties → Companies → Customers**.

The order is not cosmetic: customers reference companies by name, and reservations reference properties. Loading customers first leaves them unlinked.

Download the template for each — it is generated from the same descriptor the importer validates against, so it cannot drift out of date.

⚠️ **Nothing is written until the final button.** The whole file is judged before any of it is committed.

5 — Raise one booking and watch it land

Before anyone raises a real one.

CHECK	WHERE
The reservation saved	Reservations
Automation reported <code>sent</code>	The reservation's own page
The voucher arrived	The guest address you used — check spam
The PDF is right	Open it. ⚠️ No ₹ — currency prints as <code>INR</code>
It reached the register	The Register screen

⚠️ **Use a customer that belongs to a company.** A customer with no company exercises only one read inside the transaction, and the ordering bug that shape hides was a real defect.

19.3 Day-two operations

The webhook kill switch

`settings/webhook.enabled = false` stops delivery immediately without touching n8n. Reservations still queue.

Use it when automation misbehaves and the platform must keep working.

⚠️ **A green "Send test" proves less than it appears to** — the test path forces the webhook enabled, so it succeeds while real bookings queue and never send. A standing banner reports the configured-but-disabled state.

Deploying

```
npm run build
npx firebase deploy --only hosting --project crstest-9a0c5
```

With rules, when rules changed:

```
npx firebase deploy --only hosting,firestore:rules --project crstest-9a0c5
```

⚠️ **Never deploy rules without `npm run test:rules` passing first.** Nothing deploys them automatically, and the window between a schema change and a rules deploy is a window in which the database is open. Constraint 6.

⚠️ **Never edit `firestore.rules` with anything that writes a BOM.** The emulator reports `token recognition error at: ''` on line 1 and a real deploy fails the same way.

Watching for trouble

SIGNAL	MEANS
The red banner on Reservations	Bookings did not reach n8n. Those guests have no voucher
Automation → Run history, 0% success	⚠️ Expected and misleading — see §19.5
The register says it has no rows	Genuinely empty, or you are not admitted. It cannot tell which
Screens empty after launch	Correct until data is imported

19.4 What cannot be undone

⚠️ **Four things here roll back differently, and two do not roll back at all.**

LAYER	REVERSIBLE	HOW
Application code	✓	<code>git revert</code>
Hosting	✓	Firebase console → Hosting → Release history
Firestore rules	✓	Redeploy the previous file from git
Firestore data	✗ No	Spark has no backups
Register data	⚠️ By hand only	Point-in-time recovery is a paid feature

This is why Constraint 15 says cancel, pause or disable rather than delete. A `git revert` restores the code that deleted the records; it does not restore the records.

⚠️ **Roll hosting and rules back together or not at all.** An old client against new rules shows as permission errors on screens that worked yesterday. When data is already gone, what remains is limited: `auditLogs` says what was there and who removed it; `automationQueue` holds the payload of anything that reached n8n; the register holds reservations created since the wiring; and `FH Booking Register.xlsx` is the source for the historical rows.

⚠️ **Export before any bulk delete.** Every list screen has one.

Full procedures: `docs/ROLLBACK.md`.

19.5 Known issues at launch

Recorded so they are recognised rather than diagnosed from scratch.

Automation → Run history reports 0% success

⚠️ **Expected. It is not measuring what it appears to measure.**

Nothing ever writes back to an `automationQueue` row. Rows stay at `pending` with `attempts: 0` while the reservations they belong to correctly record `sent` · `Delivered`. Delivery works; the queue is write-only from the application's side.

Trust the reservation, and the health banner — which counts reservations, not the queue. Volume XVII §7 has the fix and why it was not taken.

"Duplicates" is offered to people who cannot use it

The navigation entry is gated on customer access but the screen needs the `merge` action, so a salesperson is shown a link to a screen they cannot use. The same fault was found and fixed for Import; the mechanism to fix it — `NavItem.canSee` and `Guard.allow` — already exists.

Interactive flows are not click-tested

The wizard, approve/cancel, merge and import commit are covered at the logic layer and have never been driven end to end by an automated browser. Verify them by hand after any change that touches them.

19.6 The operating documents

The manual is the reference. These are the working files, and they are kept current.

FILE	ANSWERS
<code>docs/HANDOVER.md</code>	Where things stand right now. Rewritten every session
<code>docs/CONSTRAINTS.md</code>	What must never happen, and what is settled
<code>docs/FLOW.md</code>	What calls what, in what order
<code>docs/DECISIONS.md</code>	Why, not just what
<code>docs/TEST-CHECKLIST.md</code>	What "done" means — commands and expected output
<code>docs/ROLLBACK.md</code>	How to undo a change
<code>docs/traces/</code>	One file per bug, start to finish
<code>docs/RUNBOOK.md</code>	Deploy, rebuild, recover, and the Owner bootstrap

`docs/FIELD-GUIDE.md` indexes them.

⚠️ **Documentation that is trusted and wrong is worse than none.** `CLAUDE.md` described an undeployed project for weeks after it shipped. State now lives in one file that is rewritten every session, and `CLAUDE.md` keeps only what ages slowly.

VOLUME XV

Glossary and index

15.1 Domain terms

TERM	MEANING IN THIS SYSTEM
ADR	Enterprise: Average Daily Rate. In this manual: Architectural Decision Record (Volume III). Where the hotel metric is meant, it is written <i>average rate</i>
AP	American Plan — room plus all meals
Blocked room	Held out of sale — maintenance, an event hold, an owner's use. Deducted from availability but not counted as booked
Channel	How a booking arrived: direct sales, corporate, travel agent, website, phone, walk-in
Commission	What Fidato earns on a booking. 8–18% by property, on gross booking value including tax
CP	Continental Plan — room plus breakfast
CRS	Central Reservation System — books across many properties. What this system is
EP	European Plan — room only
Folio	The itemised charge sheet for a stay. Frozen once the stay completes
GSTIN	Indian tax registration number. Appears on invoices
In house	A guest currently staying: check-in ≤ today < check-out
Key account	The highest company tier — largest volume, best negotiated terms
MAP	Modified American Plan — room, breakfast and one other meal
No show	Confirmed booking, guest never arrived. Terminal status
Occupancy	⚠️ Here: the property's occupancy across all channels, from the inventory model. Not Fidato's bookings ÷ total rooms (ADR-19)
PMS	Property Management System — runs one hotel. Each partner has their own; this is not one
Rate plan	A price for one room type, one meal plan, one season
Room mix	A property's breakdown by room type, e.g. <i>5 Suite / 115 Deluxe / 30 Business Class</i>
Room night	One room for one night. The unit of hotel inventory
Fidato room nights	The subset of room nights this platform booked, as opposed to all channels
Season	A date window a rate plan applies to
Shoulder season	Between peak and off-peak
Terminal status	Completed, cancelled or no-show. Locked against edits (BR-03)
Voucher	The document sent to the guest on confirmation

15.2 Technical terms

TERM	MEANING HERE
Barrel	An index file re-exporting a folder — <code>src/components/ui/index.ts</code>
Denormalisation	Copying a field onto a related document to avoid a join. Firestore has no joins
Deterministic seed	Data generated from a fixed PRNG seed, identical on every run
Guard	The route-level permission check in <code>routes.tsx</code>
Hairline	A 1 px border used instead of a shadow (ADR-16)
Invalidation	Marking a TanStack Query cache entry stale so it refetches

TERM	MEANING HERE
Query key	The array identifying a cached query. Includes the scope, so role changes invalidate automatically
Roll-up	A counter stored on a parent document (<code>totalRevenue</code>) rather than computed
Row-level scoping	Filtering <i>which records</i> an actor sees, distinct from <i>what actions</i> they may take
Scroll lock	Radix's <code>pointer-events: none</code> on <code><body></code> while a modal is open. Source of D-02
Seam	<code>repositories/index.ts</code> — the one file Phase 2 changes
Slot	Radix's <code>asChild</code> mechanism. Accepts exactly one child (D-01)
Tabular figures	Fixed-width digits so numbers align in columns
Tone	A semantic colour role — success, warning, danger, info, accent, neutral

15.3 The eight business rules

#	RULE	ENFORCED IN
BR-01	A reservation is never deleted, only cancelled	<code>canCancelReservation()</code> + no delete path
BR-02	Bookings $\geq$ ₹50,000 require approval	<code>requiresApproval()</code>
BR-03	Completed / cancelled / no-show are locked	<code>canEditReservation()</code>
BR-04	Hotel managers cannot edit rate plans	<code>canEditRates()</code>
BR-05	A salesperson sees only their assigned accounts	<code>scopeRecords()</code>
BR-06	Customer email and phone must be unique	<code>isDuplicateEmail()</code> / <code>isDuplicatePhone()</code>
BR-07	Merging moves all children to the survivor	<code>customersRepo.merge()</code>
BR-08	Every change is written to the audit log	<code>recordAudit()</code>

15.4 The twenty-four decisions

#	DECISION	COST ACCEPTED
01	Frontend before backend	Seed engine is throwaway
02	React + Vite over Next.js	No SSR
03	Tailwind v4	Long class strings
04	Radix, not shadcn/ui	28 components to write
05	TanStack Query with no server	A dependency doing "nothing" in Phase 1
06	Firestore-shaped repositories	Some screens less efficient than SQL would allow
07	120–400 ms simulated latency	The app feels slower than it is
08	Deterministic seed, resets on refresh	Work lost on reload
09	Real property data	An extraction pass over 32 PDFs
10	Role switcher, no login	Not how the real product behaves
11	Forbidden, not 404	Reveals that a resource exists
12	Show blocked actions	More markup and copy
13	Permissions $\neq$ scoping	Two concepts to understand
14	Inter, not Georgia + Arial	Documented brand deviation
15	Added a success colour	Documented brand deviation
16	Hairlines, not shadows	Flatter than fashion

#	DECISION	COST ACCEPTED
17	URL owns list state	Verbose query strings
18	Zustand, not Context	One more dependency
19	Occupancy from inventory	Two metrics to explain
20	Cancel, never delete	No delete anywhere
21	Duplicates warn, never block	Duplicates can exist temporarily
22	Assistant computes, not generates	Not a real model
23	Recharts	104 kB gzipped
24	No test framework	No regression safety net

15.5 The ten defects

#	DEFECT	CLASS
D-01	Radix <code>Slot</code> crash blanked the app	Integration
D-02	Stranded scroll lock made the app unclickable	Integration
D-03	Unbounded date range reported +757% growth	Silent
D-04	Occupancy from the wrong denominator	Silent
D-05	Seed too small — "today" looked dead	Silent
D-06	Reservations created in the future	Silent
D-07	Icon-only buttons had no accessible name	Accessibility
D-08	<code>undefined</code> from missing documents	Contract
D-09	Unsubstituted merge token	Data
D-10	Two primary buttons on the same view	Design system

Zero were caught by TypeScript or the build. Four produced confidently wrong answers, and one was invisible to every structural check — it needed a rendered screenshot.

15.6 File index

Configuration

FILE	PURPOSE
<code>vite.config.ts</code>	Aliases, manual chunks
<code>tsconfig.app.json</code>	<code>strict</code> , <code>noUnusedLocals</code> , <code>noUncheckedIndexedAccess</code>
<code>.npmrc</code> + <code>win-ca-bundle.pem</code>	Local CA trust for this machine's TLS proxy
<code>.claude/launch.json</code>	Dev server definition

Core

FILE	LINES	PURPOSE
<code>src/main.tsx</code>	~40	Providers, router, query client
<code>src/routes.tsx</code>	~280	38 routes + the permission guard
<code>src/styles/theme.css</code>	359	Every design token

Library

FILE	PURPOSE
src/lib/cn.ts	clsx + tailwind-merge
src/lib/format.ts	20 formatting functions — money, dates, text
src/lib/permissions.ts	Roles, resources, actions, matrix, scoping
src/lib/rules.ts	The 8 business rules
src/lib/session.ts	Zustand: role + UI state

Data

FILE	PURPOSE
src/data/types.ts	46 domain types
src/data/seed/random.ts	Deterministic PRNG
src/data/seed/names.ts	Name and copy pools
src/data/seed/hotels.data.ts	32 real properties — survives into Phase 2
src/data/seed/index.ts	The generator
src/data/repositories/index.ts	The seam
src/data/repositories/mock/store.ts	db, latency, query pipeline
src/data/repositories/mock/index.ts	11 repositories

Components

FOLDER	CONTENTS
src/components/ui/	28 primitives across 13 files + barrel
src/components/app/	AppShell, Sidebar, TopBar, RoleSwitcher, CommandPalette, AiPanel, navigation

Features

FOLDER	SCREENS
src/features/dashboard/	1
src/features/reservations/	5
src/features/crm/	8
src/features/hotels/	4
src/features/finance/	4
src/features/reports/	6 + shell
src/features/automation/	3
src/features/notifications/	2
src/features/ai/	1 + responses
src/features/admin/	5
src/features/design-system/	1
src/features/shared/	Forbidden, NotFound, RouteFallback, useListState

15.7 Command reference

```
npm install                # from PowerShell on this machine
npm run dev                # http://localhost:5173
npm run build
npx tsc --noEmit -p tsconfig.app.json
npm run dev -- --port 5174 # if 5173 is taken
```

15.8 Constants worth knowing

CONSTANT	VALUE	WHERE
APPROVAL_THRESHOLD	50_000	lib/rules.ts
SEED	20260728	seed/index.ts
TODAY	2026-07-28	seed/index.ts
MIN_LATENCY / MAX_LATENCY	120 / 400 ms	mock/store.ts
Default page size	25	mock/store.ts
GST bands	12% below ₹7,500, 18% at or above	reservationsRepo.quote()
House radius	10px	theme.css
Body text	14px / 22px	theme.css
Transition	150-200 ms ease-out	theme.css
Page max width	1600px	Page

15.9 Where to find things

QUESTION	VOLUME
What is this system for?	I
How is it structured?	II
Why was X chosen over Y?	III
What colour / size / spacing should I use?	IV
How does component X work?	V
What fields does entity X have?	VI
Where does the data come from?	VII
How do queries work?	VIII
Who can do what?	IX
What does screen X do?	X
Something is broken	XI
What went wrong during the build?	XII
What was actually tested?	XIII
How do I take this to Firebase?	XIV

15.10 Open questions

Carried forward, not resolved.

#	QUESTION	OWNER
1	Logo wordmark is #142B3A ; the guide says #031728 . Which is authoritative?	Brand team
2	Is the Inter substitution for Georgia + Arial approved?	Brand team
3	Is the added success green #1F6F5C approved?	Brand team
4	Should /reservations gain bulk actions, or should the copy be amended?	Product
5	Is offset pagination worth preserving via distributed counters, or is Prev/Next acceptable?	Product + engineering
6	Will the platform ever serve more than one timezone?	Product
7	Should commission be calculated on gross (current) or net of tax?	Finance

APPENDIX A

Complete data dictionary

Every field of every collection. Volume VI explains the *design*; this is the reference.

Legend — R required · O optional · D denormalised copy of another document's field · ↑ roll-up maintained by writes · 🔒 immutable after creation

A.1 Auditable — mixed into 15 collections

FIELD	TYPE	NOTES
createdAt	IsoDateTime	R 🔒
createdBy	string	R 🔒 User id
updatedAt	IsoDateTime	R Set on every write
updatedBy	string	R User id

A.2 hotels — 32 documents

FIELD	TYPE	NOTES
id	string	R 🔒 <code>htl-{city}-{slug}</code>
name	string	R Full property name
shortName	string	R For tables and chips
city	string	R Also used as the grouping key in the occupancy report
state	string	R
address	string	R
category	HotelCategory	R <code>business</code> · <code>resort</code> · <code>heritage</code> · <code>beach</code> · <code>hill_station</code> · <code>banquet</code>
status	HotelStatus	R <code>active</code> · <code>onboarding</code> · <code>paused</code> . Paused blocks new bookings
starRating	number	R 3–5
totalRooms	number	R Real. Portfolio range 17–236
description	string	R Fact-sheet marketing copy
roomMix	string[]	R Verbatim, e.g. "DeLuxe Room - 115 Rooms"
features	string[]	R
facilities	string[]	R
amenities	string[]	R
thingsToDo	string[]	R Nearby attractions
distances	HotelDistance[]	R <code>{ label, km }</code>
contacts	HotelContact[]	R <code>{ name, designation, email, phone }</code>
managerId	string	O The pinned hotel-manager user
commissionPercent	number	R 8–18
onboardedAt	IsoDate	R

Indexes needed (Phase 2): `status ASC, name ASC` · `city ASC, name ASC`

A.3 roomTypes — ~120 documents

FIELD	TYPE	NOTES
<code>id</code>	<code>string</code>	R 🔒
<code>hotelId</code>	<code>string</code>	R 🔒 Scoping key
<code>hotelName</code>	<code>string</code>	R D
<code>name</code>	<code>string</code>	R "Deluxe Room"
<code>code</code>	<code>string</code>	R "DR" — used to build rate-plan codes
<code>description</code>	<code>string</code>	R
<code>totalRooms</code>	<code>number</code>	R Caps the wizard's quantity stepper
<code>maxOccupancy</code>	<code>number</code>	R
<code>baseRate</code>	<code>number</code>	R Fallback when no rate plan matches
<code>extraAdultRate</code>	<code>number</code>	R Not yet used by the quote engine
<code>amenities</code>	<code>string[]</code>	R
<code>sizeSqft</code>	<code>number</code>	R

A.4 ratePlans — ~370 documents

FIELD	TYPE	NOTES
<code>id</code>	<code>string</code>	R 🔒
<code>hotelId</code> / <code>hotelName</code>	<code>string</code>	R / R D
<code>roomId</code> / <code>roomTypeName</code>	<code>string</code>	R / R D
<code>name</code>	<code>string</code>	R "Full Board — Shoulder Season"
<code>code</code>	<code>string</code>	R "DR-AP"
<code>mealPlan</code>	<code>MealPlan</code>	R <code>EP</code> · <code>CP</code> · <code>MAP</code> · <code>AP</code>
<code>rate</code>	<code>number</code>	R Per room per night, before tax
<code>validFrom</code> / <code>validTo</code>	<code>isoDate</code>	R The season window
<code>minNights</code>	<code>number</code>	R Not yet enforced by the wizard
<code>cancellationPolicy</code>	<code>string</code>	R Editable in the rate dialog
<code>isActive</code>	<code>boolean</code>	R

⚠️ `minNights` is stored and displayed but **not enforced**. Recorded as a Phase 2 item.

A.5 inventory — derived, not stored

FIELD	TYPE	NOTES
<code>id</code>	<code>string</code>	R <code>inv-{roomId}-{date}</code>
<code>hotelId</code> / <code>roomId</code>	<code>string</code>	R
<code>date</code>	<code>isoDate</code>	R
<code>totalRooms</code>	<code>number</code>	R From the room type
<code>booked</code>	<code>number</code>	R All channels, not just Fidato
<code>blocked</code>	<code>number</code>	R Maintenance / holds

FIELD	TYPE		NOTES
<code>available</code>	<code>number</code>	R	<code>totalRooms</code> - booked - blocked
<code>rate</code>	<code>number</code>	R	Weekend uplift ×1.15

A.6 companies — 40 documents

FIELD	TYPE		NOTES
<code>id</code>	<code>string</code>	R 🔒	<code>cmp-NNN</code>
<code>name</code>	<code>string</code>	R	Trading name
<code>legalName</code>	<code>string</code>	R	Appears on invoices
<code>tier</code>	<code>CompanyTier</code>	R	<code>key_account</code> · <code>corporate</code> · <code>sme</code> · <code>travel_agent</code>
<code>status</code>	<code>CompanyStatus</code>	R	<code>active</code> · <code>prospect</code> · <code>dormant</code>
<code>industry</code>	<code>string</code>	R	
<code>gstIn</code>	<code>string</code>	R	On invoices
<code>city</code> / <code>state</code> / <code>address</code>	<code>string</code>	R	
<code>website</code> / <code>phone</code> / <code>email</code>	<code>string</code>	R	
<code>ownerId</code>	<code>string</code>	R	Scoping key (BR-05)
<code>ownerName</code>	<code>string</code>	R D	
<code>creditLimit</code>	<code>number</code>	R	
<code>creditUsed</code>	<code>number</code>	R ↑	Drives the utilisation bar
<code>paymentTermDays</code>	<code>number</code>	R	Sets invoice <code>dueDate</code>
<code>contractStart</code> / <code>contractEnd</code>	<code>IsoDate</code>	O	
<code>negotiatedDiscountPercent</code>	<code>number</code>	R	Applied by the quote engine
<code>totalReservations</code>	<code>number</code>	R ↑	
<code>totalRevenue</code>	<code>number</code>	R ↑	
<code>lastActivityAt</code>	<code>IsoDateTime</code>	R	
<code>notes</code>	<code>string</code>	R	

A.7 customers — 180 documents

FIELD	TYPE		NOTES
<code>id</code>	<code>string</code>	R 🔒	<code>cus-NNNN</code>
<code>firstName</code> / <code>lastName</code>	<code>string</code>	R	
<code>fullName</code>	<code>string</code>	R	Stored — the search and sort field
<code>email</code>	<code>string</code>	R	Unique (BR-06)
<code>phone</code>	<code>string</code>	R	Unique on last 10 digits (BR-06)
<code>status</code>	<code>CustomerStatus</code>	R	<code>active</code> · <code>lead</code> · <code>inactive</code>
<code>source</code>	<code>CustomerSource</code>	R	7 values
<code>companyId</code> / <code>companyName</code>	<code>string</code>	O / O D	Absent for individuals
<code>designation</code>	<code>string</code>	O	
<code>city</code> / <code>state</code>	<code>string</code>	R	

FIELD	TYPE	NOTES	
<code>ownerId</code>	<code>string</code>	R	Scoping key
<code>ownerName</code>	<code>string</code>	R D	
<code>preferences</code>	<code>string[]</code>	R	Shown in the wizard, sent to the property
<code>vip</code>	<code>boolean</code>	R	Property notified before arrival
<code>totalReservations</code>	<code>number</code>	R ↑	
<code>totalRevenue</code>	<code>number</code>	R ↑	
<code>lastStayAt</code>	<code>isoDateTime</code>	O	Set on completion
<code>lastActivityAt</code>	<code>isoDateTime</code>	R	
<code>notes</code>	<code>string</code>	R	Internal only

✂ Phase 2 adds `emailNormalised` and `phoneNormalised` — Volume XIV §14.4.

A.8 reservations — 1,100 documents

Identity

FIELD	TYPE	NOTES	
<code>id</code>	<code>string</code>	R 🔒	<code>res-NNNN</code>
<code>reference</code>	<code>string</code>	R 🔒	<code>FH-2026-04821</code> — quoted to guests
<code>status</code>	<code>ReservationStatus</code>	R	7 values
<code>channel</code>	<code>BookingChannel</code>	R	6 values

Parties — all denormalised

FIELD	TYPE	NOTES	
<code>customerId</code> / <code>customerName</code>	<code>string</code>	R / R D	⚠ Both updated on merge
<code>companyId</code> / <code>companyName</code>	<code>string</code>	O / O D	
<code>hotelId</code>	<code>string</code>	R	Scoping key
<code>hotelName</code> / <code>hotelCity</code>	<code>string</code>	R D	
<code>ownerId</code>	<code>string</code>	R	Scoping key
<code>ownerName</code>	<code>string</code>	R D	

Stay

FIELD	TYPE	NOTES	
<code>checkIn</code> / <code>checkOut</code>	<code>isoDate</code>	R	Occupies nights <code>checkIn</code> → <code>checkOut</code> - 1
<code>nights</code>	<code>number</code>	R	Stored so lists need no computation
<code>rooms</code>	<code>ReservationRoom[]</code>	R	Line items
<code>guests</code>	<code>ReservationGuest[]</code>	R	
<code>totalRooms</code> / <code>totalAdults</code> / <code>totalChildren</code>	<code>number</code>	R	

Money — every component stored

FIELD	TYPE	NOTES	
<code>roomCharges</code>	<code>number</code>	R	Before discount and tax

FIELD	TYPE	NOTES
<code>extrasCharges</code>	<code>number</code>	R Reserved
<code>discountAmount</code>	<code>number</code>	R From the company's negotiated rate
<code>taxAmount</code>	<code>number</code>	R GST at 12% or 18%
<code>totalAmount</code>	<code>number</code>	R The approval threshold tests this

Approval and cancellation

FIELD	TYPE	NOTES
<code>requiresApproval</code>	<code>boolean</code>	R
<code>approvedBy</code> / <code>approvedAt</code> / <code>approvalNote</code>		O Written only on <code>pending_approval</code> → <code>confirmed</code>
<code>cancelledAt</code> / <code>cancelledBy</code> / <code>cancellationReason</code>		O Written only on cancellation

Notes and links

FIELD	TYPE	NOTES
<code>specialRequests</code>	<code>string</code>	R Sent to the property
<code>internalNotes</code>	<code>string</code>	R Never leaves the platform
<code>invoiceId</code>	<code>string</code>	O Set when the invoice is raised

ReservationRoom

FIELD	TYPE	NOTES
<code>roomId</code> / <code>roomTypeName</code>	<code>string</code>	
<code>ratePlanId</code> / <code>ratePlanName</code>	<code>string</code>	
<code>mealPlan</code>	<code>MealPlan</code>	
<code>quantity</code>	<code>number</code>	
<code>ratePerNight</code>	<code>number</code>	 Snapshot — rate changes do not affect existing bookings
<code>adults</code> / <code>children</code>	<code>number</code>	Per room

A.9 invoices — ~450 documents

FIELD	TYPE	NOTES
<code>id</code> / <code>number</code>	<code>string</code>	R  <code>INV-2607-0193</code>
<code>status</code>	<code>InvoiceStatus</code>	R 6 values
<code>reservationId</code> / <code>reservationReference</code>	<code>string</code>	R / R D
<code>customerId</code> / <code>customerName</code>	<code>string</code>	R / R D
<code>companyId</code> / <code>companyName</code>	<code>string</code>	O / O D Billed to the company when present
<code>hotelId</code> / <code>hotelName</code>	<code>string</code>	R / R D
<code>issueDate</code>	<code>isoDate</code>	R
<code>dueDate</code>	<code>isoDate</code>	R <code>issueDate + paymentTermDays</code>
<code>lines</code>	<code>InvoiceLine[]</code>	R  Frozen at issue
<code>subtotal</code> / <code>taxAmount</code> / <code>totalAmount</code>	<code>number</code>	R
<code>amountPaid</code>	<code>number</code>	R ↑

FIELD	TYPE	NOTES
<code>amountDue</code>	<code>number</code>	R ↑ Stored so it can be filtered
<code>notes</code>	<code>string</code>	R

`InvoiceLine`: `description` · `quantity` · `unitPrice` · `amount` · `taxPercent`

A.10 payments — ~380 documents

FIELD	TYPE	NOTES
<code>id</code> / <code>reference</code>	<code>string</code>	R 🔒
<code>invoiceId</code> / <code>invoiceNumber</code>	<code>string</code>	R / R D
<code>customerId</code> / <code>customerName</code>	<code>string</code>	R / R D
<code>amount</code>	<code>number</code>	R
<code>method</code>	<code>PaymentMethod</code>	R <code>bank_transfer</code> · <code>upi</code> · <code>card</code> · <code>cash</code> · <code>cheque</code>
<code>receivedAt</code>	<code>IsoDateTime</code>	R
<code>reconciled</code>	<code>boolean</code>	R Matched to a bank statement
<code>note</code>	<code>string</code>	R

A.11 commissions — ~800 documents

FIELD	TYPE	NOTES
<code>id</code>	<code>string</code>	R 🔒
<code>reservationId</code> / <code>reservationReference</code>	<code>string</code>	R / R D
<code>hotelId</code> / <code>hotelName</code>	<code>string</code>	R / R D
<code>ownerId</code> / <code>ownerName</code>	<code>string</code>	R / R D Attribution follows the account
<code>bookingValue</code>	<code>number</code>	R Gross, including tax
<code>percent</code>	<code>number</code>	R 🔒 Snapshot of the hotel's rate at accrual
<code>amount</code>	<code>number</code>	R
<code>status</code>	<code>CommissionStatus</code>	R <code>accrued</code> → <code>approved</code> → <code>paid</code>
<code>periodMonth</code>	<code>string</code>	R "2026-07"

A.12 users — 24 documents

FIELD	TYPE	NOTES
<code>id</code>	<code>string</code>	R 🔒 Becomes the Firebase Auth uid
<code>name</code> / <code>email</code> / <code>phone</code>	<code>string</code>	R
<code>role</code>	<code>Role</code>	R Becomes a custom claim
<code>hotelId</code> / <code>hotelName</code>	<code>string</code>	O Present only for hotel managers
<code>department</code>	<code>string</code>	R
<code>isActive</code>	<code>boolean</code>	R
<code>lastSeenAt</code>	<code>IsoDateTime</code>	R
<code>avatarColor</code>	<code>string</code>	R Assigned, not hashed

A.13 auditLogs — ~2,600 documents · append-only

FIELD	TYPE	NOTES
id	string	R 🔒
entityType	union	R 🔒 reservation · customer · company · hotel · invoice · user · rate
entityId	string	R 🔒
entityLabel	string	R 🔒 D So the log renders without reading the record
action	AuditAction	R 🔒 10 values
summary	string	R 🔒 One line
detail	string	O 🔒
actorId / actorName / actorRole		R 🔒
at	IsoDateTime	R 🔒

Not Auditable — an audit entry that could be updated would defeat its purpose.

A.14 notifications and notificationTemplates

notifications — 40 documents

FIELD	TYPE	NOTES
id / title / body	string	R
category	NotificationCategory	R reservation · approval · payment · system · customer · automation
channel	NotificationChannel	R in_app · email · whatsapp · sms · push
isRead	boolean	R
actorName	string	O
link	string	O Deep link
at	IsoDateTime	R

notificationTemplates — 12 documents

FIELD	TYPE	NOTES
id / name	string	R
event	string	R Matches an automation trigger
channel	NotificationChannel	R
subject	string	R Email only
body	string	R Contains {{tokens}}
isActive	boolean	R
variables	string[]	R Declared tokens

A.15 automationWorkflows and automationRuns

automationWorkflows — 12 documents

FIELD	TYPE	NOTES
id / name / description	string	R
trigger	string	R Becomes the n8n webhook path

FIELD	TYPE	NOTES
<code>triggerDetail</code>	<code>string</code>	R
<code>conditions</code>	<code>string[]</code>	R All must hold
<code>steps</code>	<code>AutomationStep[]</code>	R Ordered
<code>status</code>	<code>AutomationStatus</code>	R <code>active</code> · <code>paused</code> · <code>draft</code>
<code>runsLast30Days</code>	<code>number</code>	R
<code>successRate</code>	<code>number</code>	R Percentage
<code>averageDurationMs</code>	<code>number</code>	R
<code>lastRunAt</code>	<code>IsoDateTime</code>	O

`AutomationStep` — `id` · `order` · `kind` (8 values) · `label` · `detail` · `n8nNode?` ← the Phase 3 contract

`automationRuns` — ~340 documents

`id` · `workflowId` · `workflowName` D · `status` (`success` · `failed` · `running`) · `startedAt` · `durationMs` · `trigger` · `entityLabel` D · `error?` · `stepsCompleted` · `stepsTotal`

A.16 integrations and orgSettings

`integrations` — 14 documents

`id` · `name` · `category` (7 values) · `description` · `status` (`connected` · `available` · `error`) · `connectedAt?` · `lastSyncAt?` · `viaN8n` ← marks the Phase 3 set

`orgSettings` — singleton

`legalName` · `brandName` · `gstIn` · `registeredAddress` · `supportEmail` · `supportPhone` · `currency` · `timezone` · `financialYearStart` · `approvalThreshold` · `defaultCommissionPercent`

⚠ `approvalThreshold` exists here **and** as `APPROVAL_THRESHOLD` in `rules.ts`. The constant is authoritative in Phase 1; the settings field is display-only. 🛠 Phase 2 must make the document authoritative and delete the constant, or they will drift.

A.17 Enum reference

TYPE	VALUES
<code>HotelStatus</code>	<code>active</code> <code>onboarding</code> <code>paused</code>
<code>HotelCategory</code>	<code>business</code> <code>resort</code> <code>heritage</code> <code>beach</code> <code>hill_station</code> <code>banquet</code>
<code>MealPlan</code>	<code>EP</code> <code>CP</code> <code>MAP</code> <code>AP</code>
<code>CompanyTier</code>	<code>key_account</code> <code>corporate</code> <code>sme</code> <code>travel_agent</code>
<code>CompanyStatus</code>	<code>active</code> <code>prospect</code> <code>dormant</code>
<code>CustomerStatus</code>	<code>active</code> <code>lead</code> <code>inactive</code>
<code>CustomerSource</code>	<code>direct</code> <code>referral</code> <code>website</code> <code>ota</code> <code>corporate</code> <code>walk_in</code> <code>campaign</code>
<code>ReservationStatus</code>	<code>draft</code> <code>pending_approval</code> <code>confirmed</code> <code>checked_in</code> <code>completed</code> <code>cancelled</code> <code>no_show</code>
<code>BookingChannel</code>	<code>direct_sales</code> <code>corporate</code> <code>travel_agent</code> <code>website</code> <code>phone</code> <code>walk_in</code>
<code>InvoiceStatus</code>	<code>draft</code> <code>sent</code> <code>partially_paid</code> <code>paid</code> <code>overdue</code> <code>void</code>
<code>PaymentMethod</code>	<code>bank_transfer</code> <code>upi</code> <code>card</code> <code>cash</code> <code>cheque</code>
<code>CommissionStatus</code>	<code>accrued</code> <code>approved</code> <code>paid</code>
<code>AuditAction</code>	<code>created</code> <code>updated</code> <code>status_changed</code> <code>cancelled</code> <code>approved</code> <code>merged</code> <code>exported</code> <code>viewed</code> <code>note_added</code> <code>email_sent</code>
<code>NotificationChannel</code>	<code>in_app</code> <code>email</code> <code>whatsapp</code> <code>sms</code> <code>push</code>
<code>NotificationCategory</code>	<code>reservation</code> <code>approval</code> <code>payment</code> <code>system</code> <code>customer</code> <code>automation</code>

TYPE	VALUES
AutomationStatus	active paused draft
AutomationStepKind	generate_pdf send_email send_whatsapp update_record notify_user webhook wait condition
RunStatus	success failed running
Role	super_admin admin sales_manager salesperson hotel_manager finance support viewer

APPENDIX B

Complete component props reference

Every exported component, every prop. Volume V explains the *design*; this is the lookup.

Legend — R required · default in `code`

B.1 Button.tsx

Button

PROP	TYPE	DEFAULT	
variant	"primary" "secondary" "ghost" "danger" "link"	secondary	
size	"sm" "md" "lg" "icon"	md	
loading	boolean	false	Spinner replaces <code>leadingIcon</code> ; disables
asChild	boolean	false	Renders the child (router <code>Link</code>)
leadingIcon	ReactNode	—	
trailingIcon	ReactNode	—	Suppressed while loading
className	string	—	Merged via <code>tailwind-merge</code>
...native	ButtonHTMLAttributes	—	

⚠ With `asChild` , `disabled` is **not** forwarded — invalid on an anchor.

B.2 Input.tsx

Field

PROP	TYPE		
label	ReactNode	R	
children	(args) => ReactNode	R	Render prop
required	boolean		Adds marker + <code>aria-required</code>
hint	ReactNode		Replaced by <code>error</code> when present
error	string		Sets <code>aria-invalid</code>
className	string		

Render-prop args: { id: string; describedBy: string | undefined; invalid: boolean }

Input

PROP	TYPE	
invalid	boolean	Red border + ring
numeric	boolean	Applies <code>.tabular</code>
leadingIcon	ReactNode	Inside the field, left
...native	InputHTMLAttributes	

Textarea

`invalid` · plus native `TextareaHTMLAttributes` . Default `rows={3}` .

NativeSelect

`invalid` - plus native `SelectHTMLAttributes` . Renders its own chevron.

B.3 Combobox.tsx

PROP	TYPE	
<code>value</code>	<code>string</code>	R
<code>onChange</code>	<code>(value: string) => void</code>	R
<code>options</code>	<code>ComboboxOption[]</code>	R
<code>placeholder</code>	<code>string</code>	Shown when nothing selected
<code>searchPlaceholder</code>	<code>string</code>	Also the search input's <code>aria-label</code>
<code>emptyMessage</code>	<code>string</code>	Shown when the filter matches nothing
<code>footer</code>	<code>ReactNode</code>	Below the list — e.g. "Create a new customer"
<code>disabled</code> / <code>invalid</code>	<code>boolean</code>	
<code>id</code> / <code>className</code>	<code>string</code>	

`ComboboxOption` : { `value`: `string`; `label`: `string`; `description?`: `string`; `disabled?`: `boolean` }

Filters on label **and** description; caps at 100 rendered results.

B.4 DatePicker.tsx

DateRangePicker

PROP	TYPE		
<code>from</code> / <code>to</code>	<code>string undefined</code>	R	ISO dates
<code>onChange</code>	<code>(range: { from?: string; to?: string }) => void</code>	R	
<code>minDate</code> / <code>maxDate</code>	<code>Date</code>		
<code>id</code> / <code>className</code>	<code>string</code>		

B.5 DataTable.tsx

DataTable<T>

PROP	TYPE	DEFAULT	
<code>columns</code>	<code>Column<T>[]</code>	R	
<code>rows</code>	<code>T[]</code>	R	
<code>rowKey</code>	<code>(row: T) => string</code>	R	
<code>loading</code>	<code>boolean</code>		→ <code>SkeletonTable</code>
<code>error</code>	<code>unknown</code>		→ <code>ErrorState</code>
<code>onRetry</code>	<code>() => void</code>		
<code>onRowClick</code>	<code>(row: T) => void</code>		Makes rows focusable + Enter-activated
<code>sortBy</code>	<code>string</code>		
<code>sortDir</code>	<code>"asc" "desc"</code>		
<code>onSort</code>	<code>(key: string) => void</code>		Enables sort buttons

PROP	TYPE	DEFAULT
<code>empty</code>	<code>ReactNode</code>	Shown when the dataset is empty
<code>hasFilters</code>	<code>boolean</code>	⚠ Switches empty → no-results
<code>onClearFilters</code>	<code>() => void</code>	
<code>stickyHeader</code>	<code>boolean</code>	<code>true</code>
<code>className</code>	<code>string</code>	

Column<T>

PROP	TYPE	
<code>key</code>	<code>string</code>	R Also the sort key
<code>header</code>	<code>ReactNode</code>	R
<code>cell</code>	<code>(row: T, index: number) => ReactNode</code>	R
<code>numeric</code>	<code>boolean</code>	Right-align + <code>data-numeric</code>
<code>sortable</code>	<code>boolean</code>	Needs <code>onSort</code>
<code>width</code>	<code>string</code>	Tailwind width utility
<code>hideBelow</code>	<code>"sm" "md" "lg" "xl"</code>	Responsive dropping
<code>className</code>	<code>string</code>	

Pagination

`page` R · `pageSize` R · `total` R · `onPageChange` R · `className`

Returns `null` when `total === 0`.

B.6 FilterBar.tsx

PROP	TYPE	
<code>search</code>	<code>string</code>	R
<code>onSearchChange</code>	<code>(value: string) => void</code>	R
<code>searchPlaceholder</code>	<code>string</code>	
<code>filters</code>	<code>FilterDef[]</code>	
<code>values</code>	<code>Record<string, string></code>	
<code>onFilterChange</code>	<code>(key: string, value: string) => void</code>	
<code>onClear</code>	<code>() => void</code>	

`FilterDef`: { `key`: string; `label`: string; `options`: { `value`: string; `label`: string }[] }

⚠ `"all"` is treated as *no filter* by `applyFilters`.

B.7 Card.tsx

COMPONENT	PROPS
<code>Card</code>	<code>className</code> + native div attributes
<code>CardHeader</code>	<code>title?: ReactNode</code> · <code>description?: ReactNode</code> · <code>actions?: ReactNode</code>
<code>CardBody</code>	<code>className</code> + native
<code>CardFooter</code>	<code>className</code> + native. Right-aligned action row

⚠️ `CardHeaderProps` uses `Omit<HTMLAttributes<HTMLDivElement>, "title">` so `title` can be a `ReactNode` rather than the native `string`.

B.8 PageHeader.tsx

Page

`children` `R` · `className`. Applies `px-5 py-6 sm:px-7 sm:py-8 max-w-[1600px] mx-auto`.

PageHeader

PROP	TYPE	
<code>title</code>	<code>ReactNode</code>	<code>R</code>
<code>description</code>	<code>ReactNode</code>	
<code>breadcrumbs</code>	<code>{ label: string; to?: string }[]</code>	Last entry is the current page
<code>badge</code>	<code>ReactNode</code>	Beside the title
<code>actions</code>	<code>ReactNode</code>	Right-aligned; wraps on narrow
<code>children</code>	<code>ReactNode</code>	Filter row below the header

Section

`title?` · `description?` · `actions?` · `children` `R` · `className`

B.9 StatusPill.tsx

PROP	TYPE	DEFAULT	
<code>tone</code>	<code>Tone</code>	<code>neutral</code>	6 values
<code>children</code>	<code>ReactNode</code>	<code>R</code>	
<code>dot</code>	<code>boolean</code>	<code>true</code>	Set <code>false</code> for categorical labels
<code>className</code>	<code>string</code>		

`Tone` = `success` · `warning` · `danger` · `info` · `accent` · `neutral`

Exported tone maps

`RESERVATION_TONES` · `INVOICE_TONES` · `HOTEL_TONES` · `CUSTOMER_TONES` · `COMPANY_TONES` · `AUTOMATION_TONES` · `RUN_TONES` · `COMMISSION_TONES` · `INTEGRATION_TONES`

All `Record<string, Tone>`. Always use with a fallback:

```
<StatusPill tone={RESERVATION_TONES[r.status] ?? "neutral"}>
```

B.10 States.tsx

COMPONENT	PROPS
<code>EmptyState</code>	<code>title</code> <code>R</code> · <code>description?</code> · <code>icon?</code> · <code>action?</code> · <code>compact?</code>
<code>NoResultsState</code>	<code>onClear?</code> · <code>title?</code> · <code>description?</code>
<code>ErrorState</code>	<code>onRetry?</code> · <code>title?</code> · <code>description?</code>
<code>Skeleton</code>	<code>className</code> — set width and height
<code>SkeletonTable</code>	<code>columns: number</code> <code>R</code> · <code>rows?</code> (default 8)
<code>SkeletonCards</code>	<code>count: number</code> <code>R</code>

⚠️ Skeletons must match the height of the content they replace, or the layout jumps.

B.11 Overlays.tsx

Dialog

EXPORT	NOTES
Dialog	Radix Root
DialogTrigger	Use with <code>asChild</code>
DialogClose	Use with <code>asChild</code>
DialogContent	<code>title</code> R · <code>description?</code> · <code>children</code> R · <code>footer?</code> · <code>size?</code> · <code>className?</code>

`size` : `sm` 420 · `md` 560 (default) · `lg` 760 · `xl` 980 px

DropdownMenu

`DropdownMenu` · `DropdownMenuTrigger` · `DropdownMenuContent` (`align?` `sideOffset?`) · `DropdownMenuItem` (`onSelect?` `disabled?` `danger?` `icon?`) · `DropdownMenuSeparator` · `DropdownMenuLabel`

⚠️ Opens on `pointerdown`, not `click`.

Tooltip

`TooltipProvider` (`delayDuration?`, app-wide 400) · `Tooltip` (`content` R · `children` R · `side?`)

⚠️ Wrap a **disabled** trigger in a `<span>` — disabled elements emit no pointer events.

Tabs

`Tabs` (`defaultValue` R) · `TabsList` · `TabsTrigger` (`value` R · `count?`) · `TabsContent` (`value` R)

`count` renders a subtle badge, used for row counts.

B.12 Toast.tsx

```
toast.success(title: string, body?: string): void
toast.warning(title: string, body?: string): void
toast.error(title: string, body?: string): void
toast.info(title: string, body?: string): void
```

`<Toaster />` is mounted once in `AppShell`. Bottom-right; 5 s (7 s for errors); max 3 stacked.

B.13 Misc.tsx

COMPONENT	PROPS
Avatar	<code>name</code> R · <code>color?</code> · <code>size?</code> (<code>xs</code> 20 · <code>sm</code> 24 · <code>md</code> 32 · <code>lg</code> 40 px)
StarRating	<code>value</code> R (1–5) · <code>className?</code>
ProgressBar	<code>value</code> R (0–100) · <code>tone?</code> (<code>accent</code> <code>success</code> <code>warning</code> <code>danger</code>) · <code>className?</code>
Segmented<T>	<code>value</code> R · <code>onChange</code> R · <code>options</code> R ({ <code>value</code> , <code>label</code> }[]) · <code>className?</code>
Stat	<code>label</code> R · <code>value</code> R · <code>hint?</code>
DetailList	<code>children</code> R — renders a <code><dl></code>
DetailRow	<code>label</code> R · <code>children</code> R
Checkbox	<code>label?</code> · <code>checked</code> R · <code>onCheckedChange</code> R · <code>disabled?</code>
Switch	<code>checked</code> R · <code>onCheckedChange</code> R · <code>aria-label</code> R · <code>disabled?</code>

⚠️ **Checkbox vs Switch.** Switch = takes effect immediately, no save. Checkbox = a form value, takes effect on submit. Using a Switch in a form implies a save that never happens.

B.14 App-level components

COMPONENT	PROPS	NOTES
AppShell	—	Renders <code><Outlet/></code> ; owns the scroll-lock guard
Sidebar	onNavigate?	Passed by the mobile drawer to close on navigate
TopBar	—	Search, actions, notifications, role switcher
RoleSwitcher	—	Reads and writes <code>useSession</code>
CommandPalette	—	⌘K; closes before navigating
AIPanel	—	Right drawer

B.15 Hooks

useListState(options?)

```
{
  filterKeys?: string[];      // ⚠️ must declare every filter key
  defaultSortBy?: string;
  defaultSortDir?: "asc" | "desc"; // default "desc"
  pageSize?: number;          // default 25
}
```

Returns:

MEMBER	TYPE	
search	string	Mirrored for instant typing
setSearch	(value: string) => void	
filters	Record<string, string>	
setFilter	(key: string, value: string) => void	
sortBy	string undefined	
sortDir	"asc" "desc"	
toggleSort	(key: string) => void	
page / setPage		
clear	() => void	
hasFilters	boolean	Drives the no-results state
query	ListQuery	Pass to the repository and the query key

Session hooks

HOOK	RETURNS
useSession(selector)	Zustand selector over <code>{ role, setRole }</code>
useCurrentUser()	The <code>User</code> for the current role
useScope()	<code>{ role, userId, hotelId? }</code> for repository calls
useActor()	<code>{ id, name, role }</code> for writes
useUi(selector)	Sidebar, mobile nav, palette, AI panel

B.16 Library functions

src/lib/format.ts

FUNCTION	EXAMPLE
<code>money(v)</code>	<code>₹1,20,000</code>
<code>moneyPrecise(v)</code>	<code>₹1,20,000.00</code>
<code>moneyCompact(v)</code>	<code>₹1.2L · ₹4.6Cr</code>
<code>number(v)</code>	<code>1,20,000</code>
<code>percent(v, digits = 1)</code>	<code>61.0%</code>
<code>delta(v, digits = 1)</code>	<code>+12.4% · -3.1%</code>
<code>dateShort(v)</code>	<code>28 Jul 2026</code>
<code>dateCompact(v)</code>	<code>28 Jul</code>
<code>dateLong(v)</code>	<code>Tuesday, 28 July 2026</code>
<code>dateTime(v)</code>	<code>28 Jul 2026, 3:45 PM</code>
<code>timeOnly(v)</code>	<code>3:45 PM</code>
<code>relative(v)</code>	<code>14 days ago</code>
<code>nights(in, out)</code>	<code>3</code>
<code>nightsLabel(in, out)</code>	<code>3 nights</code>
<code>isoDate(date)</code>	<code>2026-07-28</code>
<code>initials(name)</code>	<code>AB</code>
<code>truncate(v, max)</code>	<code>Turtle Beach...</code>
<code>phone(v)</code>	<code>+91 98765 43210</code>
<code>humanise(v)</code>	<code>pending_approval → Pending Approval</code>

src/lib/permissions.ts

`can(role, action, resource) · canAccess(role, resource) · grantsFor(role, resource) · permissionMatrix() · scopeRecords(ctx, records) ·`
`ROLES RESOURCES ACTIONS ROLE_LABELS ROLE_DESCRIPTIONS RESOURCE_LABELS ACTION_LABELS`

src/lib/rules.ts

`APPROVAL_THRESHOLD · BUSINESS_RULES · isTerminal(status) · canCancelReservation(role, r) · canEditReservation(role, r) · canEditRates(role)`
`· requiresApproval(total) · canApproveReservation(role) · labelFor(status) · nextStatuses(current) · isDuplicateEmail(email, existing,`
`ignoreId?) · isDuplicatePhone(phone, existing, ignoreId?)`